

Ben-Gurion University of the Negev

The Faculty of Engineering

The Department of Industrial Engineering and Management

Scheduling with Intervals

Thesis submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy

Yuval Itzhaki

Under the supervision of **Prof. Danny Hermelin** and **Prof. Dvir Shabtay**

March 2026



Ben-Gurion University of the Negev

The Faculty of Engineering

The Department of Industrial Engineering and Management

Scheduling with Intervals

Thesis submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy

Yuval Itzhaki

Under the supervision of **Prof. Danny Hermelin** and **Prof. Dvir Shabtay**

Signature of student: _____ Date: _____

Signature of supervisor: _____ Date: _____

Signature of supervisor: _____ Date: _____

Signature of chairperson of the
committee for graduate studies: _____ Date: _____

March 2026

Abstract

This thesis investigates the complexity of three interval scheduling problems. Interval scheduling is an important subarea of scheduling, often motivated by Just-in-Time manufacturing, a paradigm that aims to produce goods *exactly* when they are needed. In such problems, jobs are specified by time *intervals*, and feasible schedules correspond to selecting subsets of pairwise non-overlapping intervals subject to additional constraints. Despite their simple structure, many natural extensions of interval scheduling are computationally hard, including those studied in this thesis, which are NP-hard in their general form. We focus on the design of exact and deterministic algorithms, which naturally motivate the use of parameterized complexity as a refined framework for identifying tractable cases. Our goal is to design *fixed-parameter tractable* (FPT) algorithms, and to establish parameterized hardness results when the existence of such algorithms contradicts widely accepted complexity assumptions. The parameters we study arise naturally from the definition of interval scheduling and include, among others, the number of machines, processing-time bounds, and graph-theoretic properties of the job-conflict relation. The thesis includes the analysis of the three problems listed below.

Eligible Machines Interval Scheduling. In the first part of the thesis, we study a scheduling problem on a set of unrelated machines. The parameterized complexity of this classical problem with respect to the number of machines was included in Mnich and van Bevern's influential list of 15 open problems. We resolved this question negatively by proving its special case of eligible machines is W[1]-hard with respect to the number of machines. Further, we prove NP-hardness for the case that the interval lengths are bounded by a constant, and complement these results with an FPT algorithm parameterized by the combined parameter number of machines and maximum interval length.

Flexible Flow Shop Interval Scheduling. The second part of the thesis focuses on a two-stage flow shop scheduling problem, aiming to maximize the weighted number of Just-in-Time jobs, i.e., jobs completed exactly at their due dates. This problem has been shown in the literature to be NP-hard when each of the two stages is served by a single machine, even when all second-stage processing times are of unit length. We focus on the extended case, where the second stage involves a set of identical machines working in parallel. We investigate the computational complexity of this problem, presenting both hardness and algorithmic results. In particular, we show that the problem remains NP-hard even in the unweighted case with unary-encoded data and is W[2]-hard with respect to the number of machines. We then provide three pseudo-polynomial time dynamic programs, each of which is capable of solving a different special case of the problem. We also show how each of these algorithms can be converted into a fully polynomial-time approximation scheme (FPTAS) using the scaling and rounding technique. One improves upon the fastest known algorithm for the case that the second-stage is served by a single machine, while the other two algorithms are designed to efficiently solve instances where either the maximum second-stage processing time is bounded by a constant or the chromatic number of the

job-conflict graph is small. Additionally, we provide polynomial-time algorithms for two natural special cases with uniform first-stage processing times, the unweighted case and the case where no job execution interval properly contains another.

Fairness in Repetitive Interval Scheduling. In the last part of the thesis, we study a repetitive scheduling model in which the same set of clients submits interval jobs over a multi-day period. Each day constitutes an instance of interval scheduling on a single machine. We aim to ensure fair long-term service, where each client's Quality of Service (QoS) is measured by the total number of jobs executed for that client over the entire period. The objective is to maximize the QoS for the least satisfied client. We prove that the problem is NP-hard even under strong restrictions, including a constant number of days and uniform processing times. To identify tractable cases, we analyze the parameterized complexity of the problem with respect to parameters such as the number of days, the QoS, and the treewidth and chromatic number of the job-conflict graph, obtaining a detailed classification of its computational landscape.

The results presented in this thesis are based on [Her+24, Hee+25, Hee+26] and were obtained in joint work with Klaus Heeger, Hendrik Molter, Baruch Schieber, Danny Hermelin, and Dvir Shabtay.

Acknowledgements

First and foremost, I would like to thank my advisors, Danny Hermelin and Dvir Shabtay, not only for guiding the research in this thesis but also for shaping my development as an academic and as a member of the scientific community. Their mentorship extended far beyond the technical aspects of research; through countless discussions, encouragement, and thoughtful guidance, they helped me navigate the professional path of academia in a way that was both intellectually challenging and truly enjoyable.

I first met Danny rather serendipitously during my master's studies at Technische Universität Berlin. A short conversation quickly led to a collaboration, and that spirit has characterized our work together ever since. His door was always open, whether to discuss science, or sometimes, anything but science. I treasure Danny's remarkable ability to bring people together, maintaining connections across the community and ensuring researchers meet, collaborate, and stay connected. Danny led by example, while giving me the freedom and trust to find my own way as a researcher. His commitment to intellectual honesty and modesty pushed me to produce better work and, through that effort, become a better researcher and a better person.

Danny introduced me to Dvir when I left Berlin for Ben-Gurion University of the Negev. Dvir grounded my work in an Operations Research perspective that complemented Danny's theoretical foundations. Drawing on decades of experience in the field, he guided me with generosity, humor, and remarkable openness. Dvir cared genuinely about both my success and my well-being, and was always there, making sure I seized opportunities to grow as a researcher. Most of all, I value the way he shared his own perspective and experience openly, both professionally and personally. In very different ways, Danny and Dvir complemented each other perfectly as mentors. I could not have asked for better advisors.

I am grateful for the fruitful collaboration with Hendrik Molter, Klaus Heeger, and Baruch Schieber, for the many productive exchanges that shaped the work presented here, and for their generous support along the way. I would also like to thank my late Master's advisor, Rolf Niedermeier, whose uncompromising investment in young researchers set me on the path that eventually led to this thesis.

I would like to thank the academic community at Ben-Gurion University of the Negev, and in particular the departments of Computer Science, Mathematics, and my home department of Industrial Engineering and Management, for providing a stimulating research environment. Many conversations, seminars, workshops, and informal discussions helped

refine the ideas in this work. A special thank you goes to Limor Algabsi, the department's head of administration, who looked after my interests, sometimes even better than I did myself, and allowed me to focus on this work. I am also grateful to the members of my PhD committee for their support and for going out of their way to ensure that I had everything needed to succeed in my research.

I gratefully acknowledge the financial support provided by the Kreitman School of Advanced Graduate Studies, the Lubner Foundation, and the Center for Integration in Science of the Ministry of Aliyah and Integration, which supported me and our lab. The partial support of the Paul Ivanir Center for Production Management Research at Ben-Gurion University of the Negev is, as well, gratefully acknowledged.

I am grateful to the students I had the privilege to teach during my doctoral studies. Teaching the course Foundations of Algorithms and Complexity was one of the most enjoyable parts of my doctoral studies. I thank the students who, through their curiosity and sometimes painful scrutiny, made me a better teacher, communicator, and researcher. I would also like to thank the Garden Library, where I had the privilege of volunteering as a teacher at the Community Education Center and the Children and Youth Education Center, and the exceptionally motivated students I encountered there. The Garden Library is one of the most effective organizations I have had the fortune to work with. I am grateful for the chance to collaborate with dozens of dedicated teachers and leaders who influence the lives of many thousands of people. My work there strongly shaped my approach to teaching and community building.

Finally, I thank my friends and family whose support crossed borders and oceans and made my path possible. Their encouragement and patience accompanied me throughout the years of this work, and without them, none of it would matter.

Contents

Abstract **i**

1 Introduction **1**
1.1 Scheduling and Complexity 1
1.2 Just-in-Time Production 3
1.3 Just-in-Time Scheduling Models 5
1.4 Interval Scheduling 7
1.5 Three Interval Scheduling Problems 11
1.6 Related Work 15
1.7 Our Contribution 17

2 Preliminaries **23**

3 Interval Scheduling with Eligible Machines **27**
3.1 Hardness 29
3.1.1 The Number of Machines 29
3.1.2 Constant Processing Times 34
3.2 Positive results 38
3.2.1 Small Number of Machines and Small Maximum Processing Time 38
3.3 Discussion 42

4 Flexible Flow Shop Interval Scheduling **43**
4.1 Hardness 45
4.1.1 The Unweighted Case 45
4.2 Positive Results 53
4.2.1 A Dynamic Program for Fixed Number of Machines 53
4.2.2 A Dynamic Program for Small Chromatic Number 57
4.2.3 A Dynamic Program for Constant Processing Times 60
4.2.4 Algorithms for Uniform Preprocessing Times 64
4.2.5 Approximation Schemes 67
4.3 Discussion 68

5 Fair Repetitive Interval Scheduling **71**
5.1 Hardness 73

5.1.1 The Fairness Parameter and the Number of Days 73
5.1.2 The Treewidth of the Overall Conflict Graph 77
5.1.3 The Day-Independent Due Dates Case 83
5.2 Positive Results 84
5.2.1 One Easy Case for the Fairness Parameter 84
5.2.2 Day-Independent Due Dates and Processing Times 85
5.2.3 An Algorithm for Small Number of Days and Small Treewidth . . . 88
5.2.4 An Algorithm for Small Number of Clients 91
5.3 Discussion 92

Conclusion **95**

Bibliography **99**

Hebrew Summary **116**

Hebrew Abstract **121**

Chapter 1

Introduction

1.1 Scheduling and Complexity

"I think the next century will be the century of complexity."

— Stephen Hawking

Scheduling is a central research area at the intersection of operations research and computer science. It concerns the allocation of tasks to resources over time, and it is integral to many fields where stakes are high and resources are scarce. It is used for important operations like coordinating transportation [LC01], manufacturing goods [Pin05], arranging medical appointments [ZP14], and orchestrating operations on microprocessors [PS85]. The last example highlights that scheduling is not only an applied tool but also fundamental to computation itself. While the settings vary, the essence is the same — given limited resources and time, we must decide when and how to execute tasks to achieve our objectives.

Making good scheduling decisions is crucial. In fact, optimal resource allocation can potentially lower the costs of large infrastructure projects such as high-speed rails [AK01], accelerate the development of life-saving medicines [Har97], and increase the throughput of critical hospital wards [CDB10]. While the practice of scheduling dates back millennia — with early civilizations already organizing labor for construction and military campaigns [DT22] — its recognition as a scientific discipline only began in the 1950s. The rise of operations research and computer science brought formal models of resource allocation, turning scheduling into a subject of systematic study. The development of sophisticated scheduling techniques, such as the Program Evaluation and Review Technique (PERT) [Mal+59] and the Critical Path Method (CPM) [KW59], played a crucial role in large-scale projects whose scope and complexity would previously have rendered them inconceivable. Even today, ambitious undertakings such as semiconductor wafer fabrication depend heavily on intricate schedules to coordinate highly sensitive operations toward objectives like the maximization of production throughput [CJ09, Wei02].

At its core, in any scheduling problem, we are given a set of tasks, which we call *jobs*, and a set of resources, typically referred to as *machines*, that are required for their processing.

Additionally, we are given constraints such as 'a single machine can process at most a single task at a time'. The goal is to construct a schedule that optimizes a predefined performance criterion, such as minimizing the time required to complete all jobs, commonly known as the 'makespan minimization' objective.

Since both the input data and evaluation criteria are precisely specified, scheduling naturally invites algorithmic decision-making for finding optimal schedules. An algorithm is essentially a step-by-step sequence of precise instructions designed to process the input data to produce an output, in our case — a schedule. Each step can be translated into machine instructions, enabling computers to produce schedules at a speed and scale far beyond what could ever be achieved manually. Yet, not all algorithms are created equal — some are faster or more efficient than others, as algorithms themselves consume computational resources, namely, time and memory. In practice, we wish algorithms to produce high-quality schedules while using computational resources economically.

The problem in scheduling is that the best solutions are hiding within a finite but potentially enormous set of feasible and infeasible schedules. A schedule is a combinatorial object; it is a combination of jobs, resources and time, all given as input for the scheduling problem. Naturally, the number of possible combinations grows exponentially with the size of the input, creating a vast solution space for any algorithm to search. A central question is whether this search can be performed *efficiently*. If the answer is positive, an efficient algorithm provides the proof. But how can we certify a negative answer? This question is addressed by computational complexity theory. We rely on the notion of NP-hardness to identify problems for which no efficient algorithm is known [AB09]. Saying that a problem is NP-hard essentially means that there is currently no known algorithm that solves it in a reasonable amount of time as its input size grows. Such problems are typically classified using polynomial time reductions from other NP-hard problems (see Chapter 2 for details). A classification of a scheduling problem as NP-hard essentially rules out the existence of a general algorithm that can efficiently compute optimal schedules for all its instances. It means that for sufficiently large inputs, these problems become practically unsolvable, requiring a prohibitive amount of time to find optimal solutions, thereby placing optimal decision-making beyond our reach.

Coping with NP-hardness is a central goal in algorithmic research. It is believed that no NP-hard problem admits an exact deterministic algorithm that solves all its instances in efficient (polynomial) time. The approaches commonly used today for dealing with NP-hard problems typically trade off algorithm runtime for the certainty or the quality of the solution. For instance, exhaustively searching the solution space is guaranteed to always find an optimal solution at the cost of efficiency. Approximation algorithms relax the requirement for exactness and allow us to find solutions that are close to optimal [Hoc97]. Relaxing the requirement that algorithms be deterministic allows us to use randomized algorithms, which may produce an optimal solution with some probability and, with some probability, return a suboptimal one [Kar91]. Another approach is to identify and exploit special cases that admit efficient exact algorithms. This approach sacrifices the generality of a 'one-size-fits-all' solution in favor of achieving computational tractability for a targeted class of instances.

A more recent and refined perspective of this approach is the theory of parameterized complexity, which studies how the complexity of a problem depends not only on its overall input size but also on certain parameters of the input [Nie06, DF13, Cyg+15].

Given an NP-hard problem, we can use parameterized complexity to look for algorithms whose runtime is a polynomial in the input size, multiplied by some function of a chosen numeric parameter (for instance, the maximum processing time of a job). This function may grow quickly, but because it depends only on the parameter, the running time remains tractable whenever the parameter is significantly smaller than the overall input size. An algorithm of this type is said to be *fixed-parameter tractable* (FPT) with respect to the parameter. Yet, not all problems and not all parameters admit FPT algorithms — some are presumably inherently hard. The parameterized analogue of NP-hardness is W[1]-hardness, which rules out the existence of FPT algorithms under widely held assumptions. The Exponential Time Hypothesis (ETH) further supports these lower bounds and gives rise to the W-hierarchy, a collection of classes of parameterized problems that capture increasing levels of presumed hardness beyond FPT. To exclude membership in FPT it suffices to show a parameterized reduction from a W[1]-hard problem (see Chapter 2 for a full definition). In such cases, the best we can generally hope for is membership in the broader class XP, where problems admit polynomial-time algorithms only when the parameter is a fixed constant. In recent years, parameterized complexity has come to complement and refine many branches of complexity theory, and is used in combination with approximation algorithms [Lok+20, Cha+17], randomized algorithms [AYZ95, LRS18], and fine-grained complexity [GMN17, AWW16].

In this thesis, we primarily employ techniques from the field of parameterized complexity to map out the computational landscape of three scheduling problems. We will study the effect of each problem's parameters on its computational complexity, to identify instances that are computationally tractable. The thesis focuses on a particular family of scheduling problems known as *interval scheduling*. This family of problems is motivated by the Just-in-Time philosophy, a manufacturing paradigm renowned for its transformative impact on modern production systems.

1.2 Just-in-Time Production

Just-in-Time (JIT) production is one of the most influential operational philosophies of the last century. The JIT approach revolutionized manufacturing by advocating that components should be produced and delivered *exactly* when needed rather than being stored in costly inventories. Although early notions of inventory minimization can be traced back to Henry Ford's practices at the Ford Motor Company [FC22], JIT's global prominence is largely attributed to its systematic implementation in the Toyota Production System during the 1950s [Pet02, FC22, WJR90].

The Toyota Production System was a revolutionary management system developed in Toyota Motor Corporation under the leadership of executive Eiji Toyoda and engineer Taiichi

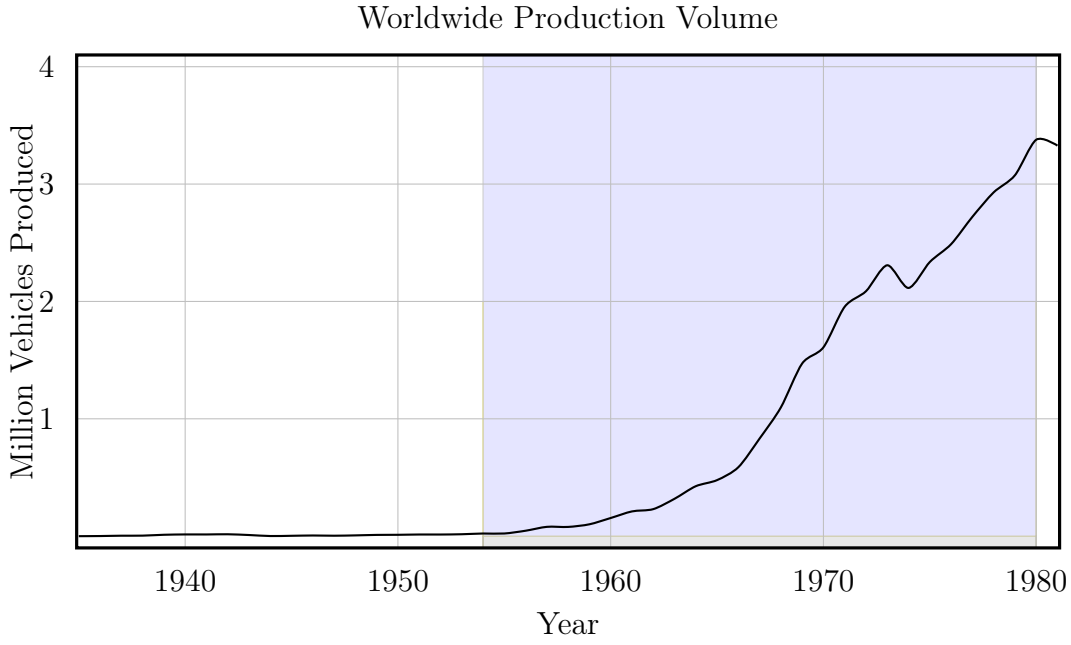


Figure 1.1: Worldwide Toyota vehicle production volume. The highlighted period (1954–1980) corresponds to the era identified by Toyota as the formative phase in which the Just-in-Time production system was systematically developed and implemented [Toy12].

Ohno. Their central insight was that efficiency could be achieved by tightly synchronizing production with actual demand, thereby minimizing the costs entailed by overproduction and unnecessary stockpiling. This shift led to the formulation of the *Just-in-Time* principle, where each component should be produced or delivered exactly when required, in the necessary quantity, and never earlier or later. In operational terms, JIT replaces the traditional *push* system, which produces in advance against forecast demand and pushes inventory downstream, with a *pull* system, in which actual consumption triggers production. The result is less waste and tighter temporal coordination throughout the process. The successful implementation of the JIT philosophy by Toyota Motor Corporation coincided with a significant increase in production volume (see Figure 1.1), and was arguably a major factor for the rise of the Japanese motor industry in the global market [OB88, Sug+77, Mon83].

Over time, the JIT principle extended far beyond Toyota and became a central component of what is now known as *Lean Manufacturing* [Näs08], shaping modern operations management across industries. A large body of work, ranging from early descriptive accounts to systematic surveys, has examined JIT implementation, with evidence pointing to significant benefits in productivity, cost efficiency, and scheduling performance [IL89, IM93, Inm+11, CFK00, Kub93, Sch82]. Decades after its introduction, JIT remains a central field in scheduling [RR11, ABF14, BS90, Bev+15, BE96, HLV02, LM96, Sha12, Spi99, SV05] and continues to attract considerable interest in application domains including logistics [Yan+21, Nug+20, Cho+23, CYC10, Agy+21, LC16], construction [HZ21, ATN20, Lyu+20, BLC19] and manufacturing [LTC96, Pin+18, Kum+22].

1.3 Just-in-Time Scheduling Models

From a scheduling perspective, the JIT philosophy makes timing a hard feasibility constraint. It recognizes that late completions disrupt subsequent operations, but also that early completions generate waste through unnecessary inventories. This contrasts with traditional criteria in scheduling (i.e. regular criteria) that are non-decreasing functions of job completion times, and therefore never penalize jobs for completing ahead of their due dates. An example of a regular scheduling criterion is $\sum w_j T_j$ — the minimization of the weighted total tardiness, where $T_j = \max\{0, C_j - d_j\}$ denotes the tardiness of job j, while C_j, d_j and w_j denote its completion time, due date, and weight, respectively.

Regular objectives essentially reduce a scheduling problem into a *sequencing problem*, the problem of finding for each machine an optimal *ordering* of job executions. An important consequence of regularity is that idle time is never beneficial. Therefore, it is unnecessary to specify the exact time at which the machine begins executing a job, as it optimally begins immediately upon the completion of the previous job in the sequence. Given a regular objective and an optimal job sequence, an optimal schedule can be thus trivially constructed by executing jobs consecutively without unnecessary idle time between them.

The JIT objectives fundamentally challenge this approach as they aim to avoid *both* earliness and tardiness, with penalties incurred for early completion as well. This makes JIT objectives *non-regular* and, therefore, typically harder to optimize. In particular, optimal JIT schedules may require the deliberate insertion of idle time, which significantly enlarges the solution space [BM99]. Furthermore, the theoretical frameworks developed for optimizing a regular scheduling criterion often fail to generalize to non-regular objectives [RS93].

Early works on JIT scheduling have assigned costs proportional to the extent of deviation from the due date, seeking to minimize the total weighted deviation of job completion times from their respective due dates [BS90, LW04]. This continues the traditional approach by minimizing $\sum_j (w_j^T T_j + w_j^E E_j)$, where $T_j = \max\{0, C_j - d_j\}$ and $E_j = \max\{0, d_j - C_j\}$ denote the tardiness and earliness of job j, respectively, and w_j^T and w_j^E are the corresponding penalty weights. Such models proved to be computationally hard to optimize [HP91a, HP91b] as they generalize the $\sum w_j T_j$ objectives, which typically do not enable efficient algorithms [LKB77, Law77, DL90, GJ78].

A more recent approach idealizes JIT through a binary performance measure, which we denote by Z_j [LM96, SDK15]. Under this criterion, a job j is considered successful only if it finishes *exactly* on its due date; in this case, $Z_j = 1$; otherwise $Z_j = 0$. The objective is then to maximize $\sum w_j Z_j$, the weighted number of jobs completed just in time. In this approach, a job that finishes early or late does not contribute to this objective and can equivalently be treated as *rejected*. This rejection-based interpretation underlies the binary performance measure Z_j and connects the JIT literature with the broader class of scheduling models, incorporating acceptance or rejection decisions [HS05, SGK13, COS12]. In a sense, just as regular objectives reduce the scheduling problem to a *sequencing* problem, this binary non-regular objective reduces the JIT scheduling problem to a *job-selection* problem, where the central task is to select a maximum-weight feasible subset of jobs.

job index j	processing time p_j	due date d_j	weight w_j
1	2	2	3
2	4	4	3
3	7	8	8
4	5	8	4
5	1	6	1
6	3	10	4
7	1	10	2

Table 1.1: A sample input for a scheduling problem.

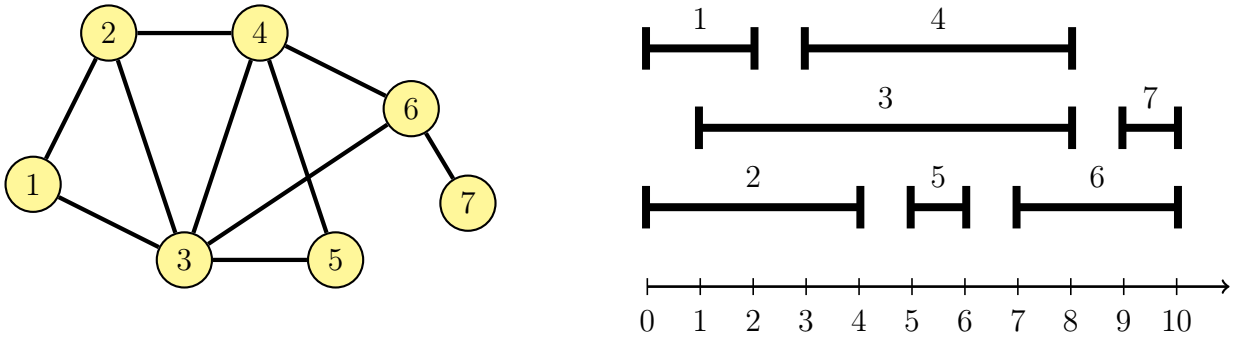
In what follows, we use the standard three-field notation $\alpha|\beta|\gamma$ of Graham [Gra69] to denote the scheduling problems that we consider. This compact notation is particularly useful for highlighting the nuanced differences between scheduling problems, which are typically very similar. The first field α specifies the machine environment, 1 in our case stands for a single machine, whereas P stands for parallel identical machines. The second field β encodes special restrictions, for instance $1|p_j = p|\sum w_j Z_j$ is the case where the processing times of all jobs are p units long. In such a case, we say that the processing times are *uniform*. The third field γ states the objective function. In $1||\sum w_j Z_j$, the objective is to maximize the weighted number of JIT jobs, which translates to maximizing $\sum w_j Z_j$ — a criterion known as the *throughput maximization* objective. This is, in fact, the most basic JIT scheduling problem we consider (see Table 1.1 for a sample instance):

$1||\sum w_j Z_j$:

Input: A set of n jobs $\mathcal{J} = \{1, \ldots, n\}$, a single machine and a target value $W^* \in \mathbb{N}$. Each job $j \in \mathcal{J}$ is characterized by a processing time $p_j \in \mathbb{N}$, a due date $d_j \in \mathbb{N}$, and a weight $w_j \in \mathbb{N}$.

Question: Is there a feasible schedule for $\mathcal{J}$ with $\sum w_j Z_j \geq W^*$?

Given a job's processing time and due date, and assuming no preemption, the requirement that it be completed Just-in-Time uniquely determines its start time. Each job can therefore be represented as a fixed time interval. A task that requires five hours of processing time, and whose delivery time is 8:00 AM, must be processed within the interval [03:00,08:00] (see job 4 in Figure 1.2 for visualization). Under this interpretation, we are given a set of jobs represented as intervals on the real line, and the goal is to find a maximum-weight subset of pairwise disjoint intervals, corresponding to a subset of jobs that can be processed on a single machine without conflict.



(a) Conflict graph representation.

(b) Interval representation.

Figure 1.2: Two representations of the job data of Table 1.1. The maximum independent set has size 3 (for instance $\{2, 5, 6\}$), whereas the maximum weighted independent set is $\{3, 7\}$ with total weight 10.

In this setting, a schedule is simply a subset of selected jobs, and it is feasible if the machine is not required to process more than a single job at any point. Equivalently, we aim to select a maximum-weighted set of *non-conflicting* jobs, where the conflict relation between jobs is precisely the intersection relation between their intervals. This perspective naturally invites graph theory, a branch of mathematics that provides a clean framework for studying objects and the relations between them. Over time, developments in graph theory have been closely intertwined with those in scheduling and operations research, offering powerful structural insights and algorithmic tools for scheduling problems.

1.4 Interval Scheduling

The emergence of interval scheduling, as a well-structured algorithmic problem, was not driven by a single discipline, but rather by the gradual convergence of ideas from operations research [DF54], graph theory [Haj57], order theory [Dil50], and even genetics [Ben59]. Essentially, interval scheduling seeks to identify maximum subsets of non-conflicting jobs — a problem that admits a natural graph-theoretic formulation. We are given a set of *vertices* V, each vertex corresponds to a job, and a set of *edges* E between pairs of vertices $E \subseteq V \times V$. The edges of the graph model the *conflict relation* between jobs. In graph-theoretical terms, the problem asks for the largest subset of vertices that are mutually non-adjacent, commonly known as the Independent Set problem.

The Independent Set problem was regarded as difficult well before the advent of computational complexity theory and appeared implicitly in early combinatorial puzzles. A prominent example is Max Bezzel's Eight Queens problem in 1848 [Bez48], which asks for a placement of eight non-attacking queens on a chessboard — an instance of Independent Set on the queen attack graph, where vertices represent board squares and edges connect mutually attacking squares (see Figure 1.3). This elegant puzzle attracted the interest of mathematicians such as Gauss and Berge [Cam77, Ber58].

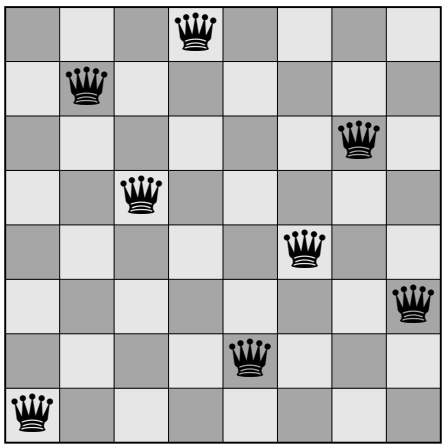


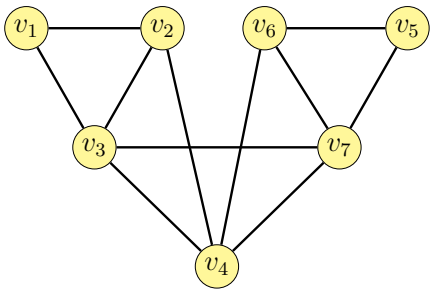
Figure 1.3: Posed by Bezzel in 1848 [Bez48], the Eight Queens problem asks for a placement of eight queens such that no pair of queens is in a position to attack each other. In graph terms, this problem asks for an 8-sized independent set in a graph whose vertices are the 64 squares of the chess board, and two vertices are adjacent if a queen can move between their corresponding squares in a single legal move.

The closely related Clique problem asks for a maximum subset of pairwise *adjacent* vertices, and appears in Karp's seminal list of 21 NP-complete problems [Kar72]. The NP-hardness of Independent Set follows immediately via a straightforward reduction from Clique using graph complementation. Therefore, both are unlikely to admit efficient algorithms. However, our special case of Independent Set admits a structural property we can exploit — the conflict relation between jobs corresponds precisely to the intersection relation between their intervals.

In a 1957 talk, the Hungarian mathematician György Hajós considered graphs whose vertices can be represented with intervals on the real line and with an edge between two vertices if and only if their corresponding intervals intersect [Haj57]. He raised the problem of recognizing such graphs and of reconstructing a corresponding *interval representation*. Fulkerson and Gross answered this question [FG65], by showing that interval graphs can be characterized by the consecutive-ones property of the vertex–maximal-clique incidence matrix (see Figure 1.4), yielding an $O(n^3)$-time algorithm for recognition and representation. This result was later improved to linear time by Booth and Lueker [BL76] using a new data structure called the PQ-tree. While this work did not address scheduling problems directly, it introduced the abstract graph-theoretic object underlying interval scheduling models.

Hajós's abstraction was formalized through a series of characterizations [LB62, GH64, FG65], revealing strong structural properties of the graph class. These works showed that the class of interval graphs is a proper subclass of *chordal graphs*, the class of all graphs where every cycle of four vertices or more contains a *chord* — an edge between non-consecutive vertices of the cycle. This classification allowed interval graphs to inherit then-recent results on chordal graphs [Ber61, HS58, Dir61, Ros70], and paved the way for efficient algorithms for the Independent Set problem on interval graphs [Gav72].

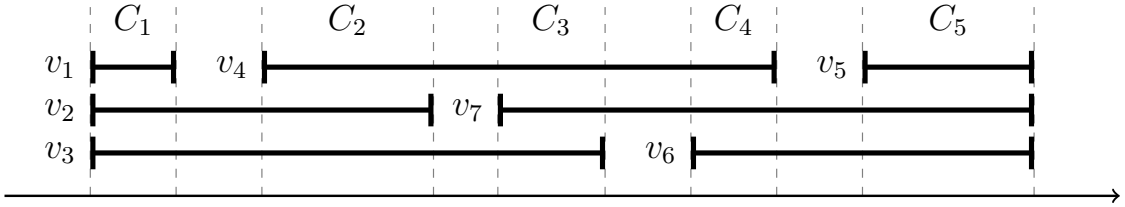
The class of chordal graphs is precisely the class of graphs that admit a *perfect elimina-*



(a) The graph G.

	C_1	C_2	C_3	C_4	C_5
v_1	1	0	0	0	0
v_2	1	1	0	0	0
v_3	1	1	1	0	0
v_4	0	1	1	1	0
v_5	0	0	0	0	1
v_6	0	0	0	1	1
v_7	0	0	1	1	1

(b) The Vertex–Maximal-Clique incidence matrix M.



(c) The graph G's interval representation.

Figure 1.4: An interval graph G, its interval representation and its Vertex–Maximal-Clique incidence matrix M, where $M[i, j] = 1$ if and only if the i-th vertex is a member of the j-th maximal clique. Observe that M has the consecutive-ones property. That is, there is a permutation of M's columns such that the ones of each row appear consecutively.

tion ordering. In such an ordering, the neighbors of every vertex v that appear later in the ordering form a clique. Fănică Gavril has used this property to develop an efficient algorithm for Independent Set on chordal graphs [Gav72]. To find the maximum independent set, pick the first vertex v in the ordering, remove it and its neighbors, and then again pick the first vertex in the remaining ordering. The first vertex in the ordering is *simplicial*, meaning its entire neighborhood forms a clique. Therefore at most one vertex from this clique can be included in the solution. Since removing a simplicial vertex and its neighbors preserves chordality, the same argument applies to the remaining graph and can be applied exhaustively. This algorithm was improved to linear time after faster methods to compute perfect elimination orderings were developed [RTL76]. Viewed through the scheduling lens, where jobs correspond to vertices and overlaps induce conflicts, ordering jobs by non-decreasing due dates yields a perfect elimination ordering.

We have seen that while the Independent Set problem is NP-hard on general graphs, its restriction to interval graphs becomes tractable. Gavril's use of perfect elimination orderings illustrates how the structural properties of interval graphs can be leveraged algorithmically. Another closely related and equally fundamental problem solved by Gavril's approach is *graph coloring*. A coloring of a graph is a partition of its vertices into color classes such that no two adjacent vertices receive the same color. Equivalently, each color class induces an independent set. Determining the minimum number of colors required to color a graph (i.e., determining its *chromatic number*), is a central theme in graph theory. Like the

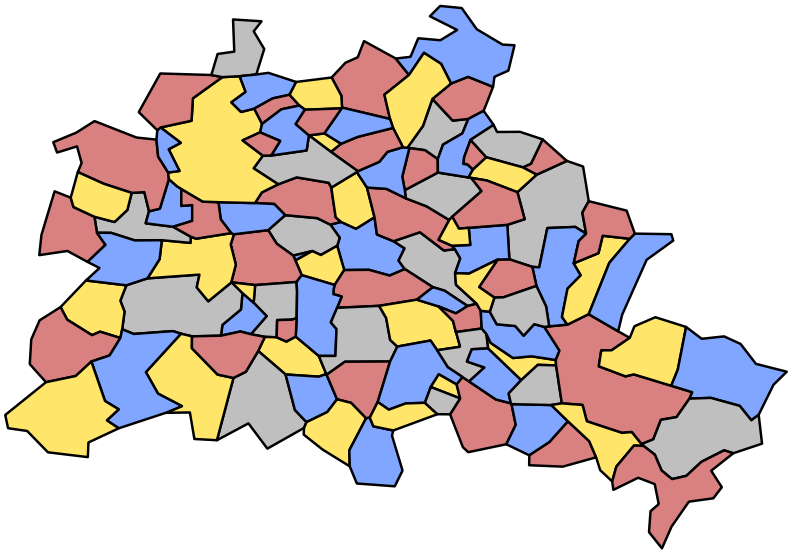


Figure 1.5: Graph coloring has attracted sustained interest in mathematics for over a century. Most famously, the Four Color Map problem, first posed in 1852 by Francis Guthrie [FF98], asks whether every planar map can be colored with at most four colors so that bordering regions receive different colors. In graph-theoretic terms, this is the question of whether every planar graph admits a coloring with at most four colors.

Independent Set problem, it has deep historical roots, with origins dating back to the nineteenth century (see Figure 1.5) and similarly, it was shown by Karp to be NP-hard on general graphs and was included in his list of 21 NP-complete problems [Kar72].

It has been known that chordal graphs are *perfect*, meaning their chromatic number equals the size of their maximum clique [Ber61], but no efficient coloring algorithm was known until Gavril. His algorithm for chordal graph coloring processes the vertices in reverse perfect elimination order, each vertex is assigned the lowest-indexed color not used by its already-colored neighbors. Since the latter neighbors of each vertex form a clique, this greedy rule never uses more colors than necessary, and the resulting coloring is optimal.

These graph-theoretic insights were subsequently translated back into the scheduling language. Coloring can be understood as scheduling on parallel identical machines, where each machine corresponds to a color and the jobs corresponding to the vertices of the color are non-conflicting as they form an independent set in the conflict graph. In this interpretation, determining whether all jobs can be scheduled on m machines corresponds to determining whether the associated interval graph is m-colorable. A closely related problem requires determining the maximum weighted number of JIT jobs executable on m identical machines working in parallel, known as Interval Scheduling on Parallel Machines (or $P||\sum w_j Z_j$ in Graham's notation):

$P||\sum w_j Z_j$:

Input: A set of n jobs $\mathcal{J} = \{1, \ldots, n\}$, a set of m machines and a target value $W^* \in \mathbb{N}$. Each job $j \in \mathcal{J}$ is characterized by a processing time $p_j \in \mathbb{N}$, a due date $d_j \in \mathbb{N}$, and a weight $w_j \in \mathbb{N}$.

Question: Is there a feasible schedule for $\mathcal{J}$ with $\sum w_j Z_j \geq W^*$?

In graph terms, $P||\sum w_j Z_j$ asks for the maximum m-colorable interval subgraph. This is a natural generalization of the classical graph coloring problem. Yannakakis and Gavril proved that finding a maximum m-colorable induced subgraph is NP-hard even when restricted to chordal graphs [YG87]; yet, in the same work, they showed that the special structure of interval graphs again enables efficient algorithms. Building on Gavril's earlier work, they presented an algorithm for finding a maximum m-colorable induced subgraph of an interval graph. Their approach solves the unweighted problem $P||\sum Z_j$ in $O(nm)$ time and the weighted $P||\sum w_j Z_j$ problem in polynomial time using a linear program with a totally unimodular constraint matrix [HK56, FG65].

These results were further refined in the scheduling literature. Arkin and Silverberg [AS87] gave an $O(n^2 \log n)$-time algorithm for maximizing the weighted number of JIT jobs on parallel identical machines, their approach was based on a linear time reduction to the minimum cost flow problem, which is solvable in $O(n^2 \log n)$ time [EK72]. Further refinements were developed by Carlisle and Lloyd [CL95], exploiting the interval structure to consider at each time step only the jobs whose intervals contain it, yielding an $O(mn \log n)$-time algorithm for $P||\sum w_j Z_j$.

Beyond these polynomial-time solvable cases, Arkin and Silverberg also examined a natural generalization that is central to this thesis. In their generalization, jobs are no longer eligible on every machine. Instead each job may be processed only on a specified subset of the machines. This eligibility constraint makes the problem NP-hard, motivating its examination through the lens of parameterized complexity. This is, in fact, the first problem studied in this thesis.

1.5 Three Interval Scheduling Problems

"Problems are treasures."

— Taiichi Ohno , *Toyota Motor Corporation*

This thesis investigates three generalizations of the interval scheduling problem, each motivated by practical scenarios, posing distinct theoretical challenges. The first problem, scheduling with eligible machines, involves throughput maximization in settings where each job can be processed only by a subset of the given machines. The second problem, interval scheduling in a flexible flow shop, likewise aims to maximize throughput, but in the context

of a production pipeline with two processing stages. The third, fair repetitive scheduling, addresses the problem of guaranteeing a level of service for a set of clients who repeatedly request the processing of their jobs over a long period.

All problems are non-preemptive, meaning that the processing of a job cannot be interrupted; it is either accepted and executed in its interval or rejected. The first and second problems are generalizations of scheduling intervals on parallel identical machines. The third is a generalization of the single machine interval scheduling using a different objective to incorporate fairness in a repetitive scheduling setting.

Typically, the input of a scheduling problem consists of a set of jobs, a set of machines, some constraints and an objective. We denote with $\mathcal{J}$ the set of jobs with labels $\{1, \ldots, n\}$ and with $\mathcal{J}_j = \{1, \ldots, j\}$ the subset of the first j jobs ordered by their due-dates. We denote with d_j the due date of job j, with p_j the processing time of job j, and with w_j the weight of job j. We denote the total weight of a set X of jobs by $w(X) = \sum_{j \in X} w_j$. We say a problem is *unweighted* when all weights are either uniform or irrelevant for the objective. Throughout this work we use the Just-in-Time performance criterion Z_j, which is 1 when a job j completes exactly at d_j and 0 otherwise.

A schedule is typically a mapping from jobs to times and machines, yet due to the Z_j performance criterion and since all considered problems in this work are non-preemptive (i.e., job executions cannot be interrupted once they begin), every schedule can be canonically described as a *selection* of jobs. We formally define a schedule for each problem individually.

Interval Scheduling with Eligible Machines

A natural and common way to generalize any scheduling problem in the parallel identical machines setting is to associate each job j with a set of *eligible machines* M_j [LLP11, LS08]. In this setting, job j can be scheduled on machine i only if $i \in M_j$. Such constraints arise naturally when machines differ in functionality, or capacity, or when certain jobs are compatible only with newer or specialized machines. We are naturally interested in maximizing the throughput of such systems. The decision version of this problem is presented below.

$P|M_j|\sum w_j Z_j$:

Input: A set of n jobs $\mathcal{J} = \{1, \ldots, n\}$, a set of m machines, and a target value $W^* \in \mathbb{N}$. Each job $j \in \mathcal{J}$ is characterized by a processing time $p_j \in \mathbb{N}$, a due date $d_j \in \mathbb{N}$, weight $w_j \in \mathbb{N}$ and a set of eligible machines $M_j \subseteq \{1, \ldots, m\}$.

Question: Is there a feasible schedule for $\mathcal{J}$ with $\sum w_j Z_j \geq W^*$?

Consider the following example. In hospital operating rooms, procedures must be scheduled within fixed time windows due to patient preparation, staff availability, and

post-operative care. While operating rooms may appear identical, some procedures require specialized equipment or certified staff available only in specific rooms. Consequently, a procedure may be schedulable only on a subset of rooms, even though its processing time is identical across all rooms. This naturally leads to an interval scheduling problem with eligible rooms and strict timing constraints.

Flexible Flow Shop Interval Scheduling

A second natural generalization of Interval Scheduling on Parallel Machines arises in production systems where jobs must pass through multiple ordered processing stages. Such settings are ubiquitous in manufacturing pipelines, logistics chains, and service systems, where a job must first undergo preparation or setup before being processed. In the flow shop model, each job is subject to precedence constraints that enforce completion of one processing stage before the next stage starts.

Flexible flow shops are multi-stage production systems in which one or more stages consist of parallel machines, allowing jobs to be assigned to different machines within a stage. This flexibility distinguishes them from classical flow shops and enables better load balancing, but it also significantly increases the complexity of scheduling decisions.

We arguably consider the simplest form of such systems, a two-stage flexible flow shop system, which we denote by $FF(1, m)$. The input consists of a set of n jobs that must be processed non-preemptively. Each job must undergo two stages of processing, the first has a single machine, while the second consists of m identical machines running in parallel. Each job $j \in \mathcal{J}$ requires two operations, with processing times $p_j \in \mathbb{N}$ and $q_j \in \mathbb{N}$, respectively. A feasible schedule assigns the first operation of each job to the single machine in the first stage and the second operation to one of the m parallel machines in the second stage, such that no two operations overlap on the same machine and each job's second operation begins only after its first operation has been completed. The objective is to maximize the weighted throughput, that is, the total weight of jobs completed exactly at their due dates. We hence use $FF(1, m)||\sum w_j Z_j$ to denote this problem, its decision version is defined as follows:

$FF(1, m)||\sum w_j Z_j$:

Input:	A set of n jobs $\mathcal{J} = \{1, \ldots, n\}$, and a target value W^*. Each job $j \in \mathcal{J}$ is characterized by a due date $d_j \in \mathbb{N}$, the processing time $p_j \in \mathbb{N}$ of its first-stage operation, the processing time $q_j \in \mathbb{N}$ of its second-stage operation, and a weight $w_j \in \mathbb{N}$.
Question:	Is there a feasible schedule for $\mathcal{J}$ with $\sum w_j Z_j \geq W^*$?

As an example application of $FF(1, m)||\sum w_j Z_j$, consider a small regional airport with m gates (analogous to machines) that primarily handles connecting flights arriving overnight. Each flight j has a scheduled departure time d_j, typically after 6:00 AM, and may

require fueling beforehand. A single fueling team, available from 6:00 AM, services all aircraft. Each plane has a unique fueling time p_j, based on its tank capacity and remaining fuel. For safety reasons, fueling must be completed before boarding, which takes an additional q_j time units depending on the aircraft and passenger load. The airport incurs a penalty w_j for any flight j that fails to depart exactly at time d_j. To maximize revenue, the airport must schedule fueling of planes and then assign departure slots to maximize the total weight of on-time departures $\sum w_j Z_j$.

Fair Repetitive Interval Scheduling

Our third problem departs from the throughput maximization objective and instead addresses long-term service guarantees in systems in which jobs recurrently arrive from a fixed set of clients. Such settings arise naturally in shared services, cloud platforms, maintenance operations, and public infrastructure, where clients repeatedly request processing, and fairness considerations are as important as efficiency.

Formally, let $p_{i,j}$ and $d_{i,j}$ denote the processing time and due date of the job of client $j \in \{1, \ldots, n\}$ on day $i \in \{1, \ldots, q\}$. A single machine that can process only one job at a time is available on each of the q days. Therefore, every pair of jobs whose intervals intersect are in conflict. A schedule σ_i for day $i \in \{1, \ldots, q\}$ is simply a subset of the jobs on that day, and it is said to be *feasible* if it contains no conflicting pair of jobs. A solution for all q days is therefore a tuple $(\sigma_1, \ldots, \sigma_q)$ of q schedules, one for each day, and it is said to be *feasible* if each σ_i is feasible. Given a feasible solution, let $Z_{i,j}$ denote a binary performance measure that equals 1 if the job of client j is scheduled on day i, and 0 otherwise. The objective is to maximize the service level of the least satisfied client, that is, to maximize the minimum sum $\sum_i Z_{i,j}$ over all j in $\{1, \ldots, n\}$. We call this problem Fair Repetitive Interval Scheduling and its decision version is defined as follows:

$1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$:

Input:	A set of n clients, where each client $j \in \{1, \ldots, n\}$ has a job with due date $d_{i,j} \in \mathbb{N}$ and processing time $p_{i,j} \in \mathbb{N}$ for each day $i \in \{1, \ldots, q\}$, and a target value $k \in \mathbb{N}$.
Question:	Are there q schedules, $\sigma_1, \ldots, \sigma_q$, such that σ_i is feasible on each day $i \in \{1, \ldots, q\}$, and $\sum_i Z_{i,j} \geq k$ for each $j \in \{1, \ldots, n\}$?

As an example application, consider a cloud provider offering infrastructure-as-a-service (IaaS) services to n clients in certain regions. Each of the clients wishes to deploy a server cluster on the infrastructure so that latency between the client and the cluster is minimized. Our infrastructure in the clients' region can only support one cluster at a time (acting as a single machine) and is requested by each client during a specific time interval each day. As the time intervals may intersect, on each of the days some jobs will have to be rejected.

Rejected jobs will be hosted on remote servers that suffer from increased latency. We wish to provide a fair schedule to the clients such that each client will have k of his jobs scheduled on the local (regional) infrastructure over a period of q days.

1.6 Related Work

Interval Scheduling with Eligible Machines

Arkin and Silverberg were the first to study the $P|M_j|\sum Z_j$ problem [AS87]. They proved that the problem is strongly NP-hard in general but solvable in $O(mn^{m+1})$ time, which is polynomial when m is fixed. The last algorithm can also be viewed as an XP-algorithm with respect to the number of machines (see Chapter 2 for more about XP and other parameterized complexity classes). The eligible machines setting serves as an intermediate model between the case of parallel identical machines and its generalization, the classic *unrelated machines* setting where jobs have different processing times on different machines [MW14]. Under the JIT objective, the unrelated machine interval scheduling problem is also solvable in $O(mn^{m+1})$ [SV05]. Yet, the question whether the problem is solvable in FPT time with respect to m remained open [MB18].

In interval graph terms, $P|M_j|\sum w_jZ_j$ is precisely an optimization version of the List Coloring problem on interval graphs. In List Coloring, a generalization of Coloring, every vertex is given a list of eligible colors, and we are asked whether the graph admits a proper coloring that assigns every vertex a color from its list. The correspondence is as follows. Every machine corresponds to a color. No two conflicting jobs can be assigned to the same machine, and thus their vertices cannot receive the same color. Finally, the set M_j of eligible machines of a job is exactly the color list of its vertex. The two problems differ only in that $P|M_j|\sum w_jZ_j$ does not require coloring every vertex; instead, the objective is to color a maximum-weight subset of vertices while respecting their eligibility lists.

Precoloring Extension is a special case of List Coloring. Some vertices have predetermined colors, equivalently, they have a color list that only contains one color. The remaining vertices have lists containing all colors. Since 1992, it is known that Precoloring Extension on unweighted interval graphs is NP-complete [BHT92], which implies that $P|M_j|\sum w_jZ_j$ is NP-complete even in the unweighted case when all eligible sets have size either 1 or m. In 2006, Marx [Mar06] strengthened this result by showing that Precoloring Extension is NP-complete even when restricted to *unit interval* graphs. Hence, we have that $P|M_j|\sum w_jZ_j$ is NP-complete even when all jobs have unit weight *and* all processing times are uniform; that is, all jobs have the same processing time p.

Two-Stage Flexible Flow Shop Interval Scheduling

The problem $P||\sum w_jZ_j$ [YG87, AS87, CL95] is clearly the special case of $FF(1,m)||\sum w_jZ_j$ when $p_j = 0$ for all jobs j. The first to investigate the JIT objective for a (non-flexible) flow

shop problem were Choi and Yoon [CY07]. Their research established that the problem is already strongly NP-hard for an unweighted three-stage flow shop with a JIT objective. They have also proved NP-hardness for the two-stage problem $FF(1,1)||\sum w_j Z_j$ even when all second-stage processing times are of unit length (*i.e*, the $FF(1,1)|q_j = 1|\sum w_j Z_j$ problem). This proof only showed *ordinary* NP-hardness, leaving open the question whether the problem $FF(1,1)||\sum w_j Z_j$ admits a *pseudo-polynomial time* algorithm (see Chapter 2 for formal definitions of these and other complexity notions used throughout). On the positive side, their work showed that the unweighted variant, $FF(1,1)||\sum Z_j$, can be solved in $O(n^4)$ time, a result which was later improved to $O(n^3)$ time [Sha12]. The question of pseudo-polynomial time solvability was later resolved by Shabtay and Bensoussan [SB12] who proved that $FF(1,1)||\sum w_j Z_j$ can be solved in $O(P \cdot n^2)$ time, where P is the total processing time of all first-stage operations. They also provided a fully polynomial time approximation scheme (FPTAS), which was later improved by Elalouf et al. [ELT13], resulting in an $O(n^3/\epsilon)$ time FPTAS. From a parameterized standpoint, the $FF(1,1)||\sum w_j Z_j$ problem was shown to be W[1]-hard with respect to the number of distinct due dates $d_\#$, but FPT when parameterized by the combination of $d_\#$ with either the number of distinct weights $w_\#$ or the number of distinct processing times $p_\#$ [HST19].

Fair Repetitive Interval Scheduling

The problem $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ is a case of a recently introduced framework for maximizing the *minimum quality of service per agent* — an *egalitarian*, or *max-min*, notion of fairness — in the context of *fair repetitive scheduling* [Hee+23, Her+25, SPF24, SFP25]. In this framework, we aim to serve a set of n agents over a period of q days. In the most basic case, each agent requests one job per day. There is a single machine available each day. A single objective function measures the Quality of Service (QoS) for all agents. Thus, one might think of the input in this setting as q instances of a certain single-machine scheduling problem, where each instance contains n jobs. The goal is to determine whether there exist q schedules, one for each day, where every client is guaranteed k job executions over the entire q-day scheduling period. We refer to k as the *fairness parameter*.

This egalitarian criterion, which follows the Rawlsian *maximin* social welfare function [Raw71, RL07], is one of several fairness notions studied in the operations research and social choice literature; prominent alternatives include utilitarian, proportional, and envy-based notions [Mou04]. Notably, the throughput objectives studied in the preceding subsections are utilitarian in nature — they maximize the total service provided, possibly at the expense of individual clients. The repetitive setting is precisely what renders the egalitarian alternative attainable. While conflicts prevent serving all clients on any single day, service can be balanced across the q-day horizon, as was done in max-min fair resource allocation for communication networks [NP08, Bon+06].

Heeger et al. [Hee+23] study the $1 \mid \text{rep} \mid \max_j \sum_i U_{i,j}$ problem in the fair repetitive scheduling framework, whose goal is to find q schedules, one for each day, where no client has his jobs tardy in more than k of the days. Hermelin et al. [Her+25] extend this framework with

three additional objectives: the $\sum_i C_{i,j}$ objective, which ensures that the total completion time of each client's jobs over all q days is at most k; the $\max_j \sum_i W_{i,j}$ objective, which ensures that no client's total waiting time exceeds k; and the $\sum_i L_{i,j}$ objective, which measures QoS by the total lateness of each client's jobs rather than their total waiting time.

1.7 Our Contribution

Before presenting our results for each of the three problems, we briefly describe the methodological themes that run through them. Our algorithmic results follow two main routes. The first is dynamic programming following Gavril's approach: our algorithms sweep the timeline, exploiting the geometry of intervals to consider only jobs whose intervals cross the sweep point. The sweep is designed so that the information the program must store at any moment remains bounded in terms of the relevant parameter. The second is reduction to well-understood combinatorial problems, whose known algorithms carry over to restricted variants of our problems. Our hardness results are (parameterized) reductions from hard computational problems, where the recurring challenge is to encode the combinatorial structure of the source problem into intervals while keeping the parameters of interest bounded, thereby establishing hardness even for these restricted cases. Finally, several of our pseudo-polynomial dynamic programs are converted into approximation schemes using the scaling-and-rounding technique.

Interval Scheduling with Eligible Machines

Recall that Arkin and Silverberg [AS87] showed that $P|M_j|\sum w_j Z_j$ is solvable in $O(n^{m+1})$-time, yet, they left open the question whether the problem admits an FPT-algorithm with respect to m. That is, whether the problem admits an algorithm with running time $f(m)\cdot(n)^{O(1)}$. Such an algorithm would be useful in most natural settings where the number of machines m is significantly smaller than the number of jobs to be processed. This question was included in Mnich and van Bevern's [MB18] influential list of major open problems in the parameterized complexity analysis of scheduling problems. Our first result provides a negative answer to this question.

Theorem 1.1 *$P|M_j|\sum w_j Z_j$ is strongly* W[1]*-hard when parameterized by the number of machines m.*

The proof of this theorem is provided via a parameterized reduction from the W[1]-hard problem Multicolored Clique [Fel+09]. While this is a standard approach for proving W[1]-hardness, the construction used here is rather involved, and relies on the job weights in the resulting $P|M_j|\sum w_j Z_j$ instance.

We next focus on studying the problem's complexity with respect to the processing-time values. Recall that Marx [Mar06] proved that $P|M_j|\sum w_j Z_j$ is NP-complete for the restrictive case where all jobs have uniform processing times and weights. However, although

the reduction of Marx constructs a unit interval graph, the uniform processing time p can be arbitrarily large. It therefore remains open whether the problem admits an FPT-algorithm with respect to p. Our next result rules out the existence of such an FPT-algorithm assuming that P $\neq$ NP. We show that $P|M_j|\sum w_jZ_j$ remains NP-hard even when unweighted and when all processing times are constant (this also applies to the general case where processing times may differ from job to job). To state our result, let $p_{\max}$ denote the maximum processing time of any job in a $P|M_j|\sum w_jZ_j$ instance. We prove the following theorem via a reduction from a variant of 3-SAT.

Theorem 1.2 *$P|M_j|\sum Z_j$ is* NP*-hard even when $p_{\max} \in O(1)$.*

Theorem 1.1 and Theorem 1.2 imply that we cannot find an optimal schedule efficiently even if the instance of $P|M_j|\sum w_jZ_j$ has either few machines or jobs with small processing times. However, we can show that when both these parameters are small, an efficient algorithm becomes possible. More precisely, we show that $P|M_j|\sum w_jZ_j$ is fixed-parameter tractable when parameterized by the combined parameter $m + p_{\max}$. The algorithm first applies a data reduction step that bounds the number of pairwise overlapping jobs in terms of m and $p_{\max}$, and then runs a dynamic program over the timeline of the reduced instance.

Theorem 1.3 *$P|M_j|\sum w_jZ_j$ is solvable in $O((m \cdot p_{\max})^{2m} \cdot mn)$ time, and is hence fixed-parameter tractable when parameterized by $m + p_{\max}$.*

With Theorem 1.1, we answer a fundamental, longstanding open question concerning the parameterized computational complexity of $P|M_j|\sum w_jZ_j$. Our results together with the XP-containment for the number of machines m from Arkin and Silverberg [AS87] essentially resolve the parameterized complexity classification of $P|M_j|\sum w_jZ_j$ for the number of machines m as a parameter, for the largest processing time $p_{\max}$, and for their combination. For full details of the results, see Chapter 3.

Flexible Flow Shop Interval Scheduling

As mentioned above, $FF(1,m)||\sum w_jZ_j$ is NP-hard already for $m = 1$ (and $q_j = 1$ for all jobs j) [CY07]. However, the unweighted $FF(1,1)||\sum Z_j$ problem is polynomial-time solvable [CY07, Sha12]. This raises the natural question of whether or not $FF(1,m)||\sum Z_j$ can be solved in polynomial-time. Our main result on this problem is a negative answer to this question.

Theorem 1.4 *$FF(1,m)||\sum Z_j$ is strongly* NP*-hard and strongly* W[2]*-hard with respect to the number of the second-stage machines m.*

We prove Theorem 1.4 using a reduction from the classical Hitting Set problem. Our reduction constructs an instance such that all numeric parameters are polynomial in the size of the input, thereby excluding pseudo-polynomial time algorithms unless P=NP. Moreover,

our reduction is also a *parameterized reduction* (see Chapter 2), ruling out the existence of an $f(m) \cdot n^{O(1)}$ time algorithm for any computable function f, assuming FPT $\neq$ W[2].

To complement the hardness result above, we present a dynamic programming approach which solves $FF(1,m)||\sum w_j Z_j$ in pseudo-polynomial time in the case of $m \in O(1)$ (constant number of second-stage machines). Since the problem is known to be only weakly NP-hard for $m = 1$ [CY07], the existence of a pseudo-polynomial-time algorithm is not ruled out. Let $W = \sum_j w_j$ denote the total weight of jobs in our input.

Theorem 1.5 *$FF(1,m)||\sum w_j Z_j$ is solvable in $O(W \cdot n^m)$ time.*

The result above improves the current fastest algorithm for $FF(1,1)||\sum Z_j$ [SB12] from runtime of $O(n^3)$ to $O(n^2)$. Moreover, it can be easily converted to an algorithm with a running time of $O(P \cdot n^m)$, where $P = \sum_j p_j$ is the total processing time of all first-stage operations in the input.

We proceed by presenting two dynamic programming algorithms, each designed to run efficiently under a different structural parameter. The first parameter, denoted by χ, is the chromatic number of the *second-stage* conflict graph, in which two jobs are considered conflicting if their *second-stage* operations overlap; see Chapter 4 for a formal definition. The second parameter, denoted by $q_{\max}$, is the maximum processing time of any second-stage operation, that is, $q_{\max} = \max_j q_j$. We establish the following result with respect to these two parameters.

Theorem 1.6 *$FF(1,m)||\sum w_j Z_j$ can be solved in:*

- *$O(W \cdot 2^{\chi} \cdot n)$ time, where χ is the chromatic number of the second-stage conflict graph.*
- *$O(W \cdot m^{q_{\max}} \cdot n)$ time, where $q_{\max} = \max_j q_j$ is the maximum processing time of any second-stage operation.*

Note that Choi and Yoon [CY07] showed that $FF(1,1)||\sum w_j Z_j$ is NP-hard already when $\chi = 1$, $m = 1$ and $q_{\max} = 1$. Thus, Theorem 1.6 shows that the problem is only weakly-NP-hard when $\chi \in O(\log n)$ or when $q_{\max} \in O(1)$.

Next, we consider the special case of uniform first-stage processing times, *i.e.* the case where $p_j = p$ for all jobs $j \in J$, the $FF(1,m)|p_j = p|\sum w_j Z_j$ problem. We provide two polynomial-time algorithms for this case: one for the unweighted setting, and another for the weighted setting, which applies only to *proper* instances, instances where no second-stage interval properly contains another. That is, for any pair of distinct jobs $i \neq j \in J$ it holds that $[d_i - q_i, d_i) \not\subseteq [d_j - q_j, d_j)$.

The unweighted case is solved by a greedy variant of the classical Moore–Hodgson algorithm [Moo68]: jobs are processed in nondecreasing order of starting times and, whenever a conflict arises, the job with the largest due date is discarded; a heap-based implementation yields the $O(n \log n)$ running time. For weighted proper instances, we formulate the problem as an integer linear program whose constraint matrix — owing to the properness of the instance — has the consecutive ones property, and is therefore totally unimodular; an optimal solution is thus obtained from the linear programming relaxation.

Theorem 1.7 *$FF(1,m)|p_j = p|\sum w_j Z_j$ is solvable in:*

- *$O(n \log n)$ time, when $w_j = 1$ for all $j \in J$, and*
- *$O(n^{3.5}(n + \log W))$ time, when $[d_i - q_i, d_i) \nsubseteq [d_j - q_j, d_j)$ for all $i \neq j \in J$.*

Finally, we show how to convert our dynamic programs for $FF(1,m)||\sum w_j Z_j$ into a fully polynomial-time approximation scheme (FPTAS) when $m \in O(1)$ or $q_{\max} \in O(1)$ or $\chi \in O(\log n)$. The scheme is based on the scaling and rounding technique which reduces the size of the dynamic programming tables computed in Theorem 1.5 and Theorem 1.6. By selecting an appropriate scaling factor, we obtain a feasible solution whose objective value is within a factor of $(1 - \varepsilon)$ of the optimum.

Theorem 1.8 *$FF(1,m)||\sum w_j Z_j$ admits an FPTAS when $m \in O(1)$, or $q_{\max} \in O(1)$, or when $\chi \in O(\log n)$.*

Full details of the results are found in Chapter 4.

Fair Repetitive Interval Scheduling

We investigate the computational complexity of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ across its natural restrictions and parameters, mapping the boundary between its tractable and intractable cases. Our first result in this regard gives a dichotomy of the NP-hardness of the problem under every possible value of the fairness parameter k. The hardness side of the dichotomy is established by a reduction from a variant of 3-SAT for the base case $k = 1$, $q = 3$, which we then extend to all remaining hard pairs (k, q) with $k \neq q - 1$ via two inductive arguments. For the tractable case $k = q - 1$, we give a polynomial-time reduction to 2-SAT.

Theorem 1.9 *The $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ problem is polynomial-time solvable when the fairness parameter $k \in \{0, q-1, q\}$, and otherwise it is* NP*-hard even if $p_{i,j} = 2$ for all i and j.*

Following Theorem 1.9, we proceed to examine $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ under further restrictions. The first restriction we consider is *day-independent processing times*, the case where the processing time of a job depends solely on its client and is identical for each of the q days. That is, for each client j we have $p_{i,j} = p_j$ for all days $i \in \{1, \ldots, q\}$. We denote this special case by $1 \mid \text{rep}, p_{i,j} = p_j \mid \min_j \sum_i Z_{i,j}$. As it turns out, the hardness proof of Theorem 1.9 applies to $1 \mid \text{rep}, p_{i,j} = 2 \mid \min_j \sum_i Z_{i,j}$ as well. However, when we further restrict the processing times to *unit*, the problem becomes polynomial-time solvable via reduction to Bipartite Maximum Matching. Note that the latter special case corresponds to scheduling scenarios with *slots* of fixed time duration for which clients can apply, as commonly used in public health systems [GD08, Car+11, Ala+22].

Theorem 1.10 *The $1 \mid \text{rep}, p_{i,j} = p_j \mid \min_j \sum_i Z_{i,j}$ problem is* NP*-hard. It is polynomial-time solvable when $p_j = 1$ for all $j \in \{1, \ldots, n\}$.*

We next consider the case of *day-independent due dates*, *i.e.*, the case where for each client j we have $d_{i,j} = d_j$ for all days $i \in \{1, \ldots, q\}$. This special restriction, denoted by $d_{i,j} = d_j$, is not only a natural theoretical restriction but also makes sense in practice, as clients often have a unique due date that is associated with *e.g.* a specific delivery time. This variant is NP-hard by the result of Marx [Mar06], and W[1]-hard with respect to q by our Theorem 1.1. However, we identify two further natural restrictions for which the problem becomes polynomial-time solvable, namely that either the number of days is constant or that the processing times are day-independent as well.

Theorem 1.11 *The* $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ *problem is* NP*-hard. It is solvable in polynomial time if either of the following additional restrictions holds:*

- *The number q of days is constant.*
- *The processing times are day-independent. That is, $p_{i,j} = p_j$ for all $i \in \{1, \ldots, q\}$ and $j \in \{1, \ldots, n\}$.*

The algorithm for a constant number of days is a dynamic program that processes clients in order of due dates, maintaining for each day only the last client scheduled on it. The algorithm for day-independent processing times rests on a perfect-graph argument: since the daily conflict graph is an interval graph, a k-fair schedule exists if and only if $k \cdot \chi \leq q$, where χ is the chromatic number of the conflict graph — a condition checkable in $O(n \log n)$ time.

Finally, we focus on natural parameters of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ and study how they affect the problem's complexity when they are limited in size. For the parameters k and q, we know that the problem is NP-hard even when both are constant due to Theorem 1.9. We show that this is not the case for the number of clients n: exploiting the fact that there are only $2^{O(n \log n)}$ distinct interval graphs on n vertices, we group the days by their conflict graphs and formulate the problem as an integer linear program whose number of variables is bounded in n, which can be solved using Lenstra's algorithm [Len83].

We then turn to a graph-decomposition approach, considering the *treewidth* ω (see Definition 3) of the overall conflict graph — the graph with a vertex for each client, in which two vertices are adjacent if their clients have conflicting jobs on some day. We show that the problem is NP-hard even for constant ω; since it is also NP-hard for constant q (Theorem 1.9), neither parameter alone can yield fixed-parameter tractability. For the combined parameter $\omega + q$, however, a dynamic program over a tree decomposition yields an FPT algorithm.

Theorem 1.12 *The* $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ *problem*

- *is* NP*-hard for $\omega \in O(1)$,*
- *admits an* FPT *algorithm with respect to parameter $\omega + q$, and*
- *admits an* FPT *algorithm with respect to n.*

Formal definitions of the *overall conflict graph*, *treewidth* and tree-decomposition appear in Chapter 5 (Definitions 2 and 3), where full details of the results are provided.

Chapter 2

Preliminaries

Computational complexity We assume basic familiarity with classical computational complexity theory and refer the reader to the textbook of Arora and Barak [AB09] for a comprehensive introduction.

A decision problem is a language $L \subseteq \Sigma^*$ over a finite alphabet Σ. We say an algorithm A *decides* language L if $A(x) = 1 \Leftrightarrow x \in L$. The class P consists of all problems that can be decided by a deterministic algorithm in polynomial time with respect to its input size. The class NP consists of all problems that can be decided by a nondeterministic polynomial-time algorithm. Equivalently, NP contains all problems for which there exists a deterministic polynomial time algorithm A where $A(x, y) = 1 \Leftrightarrow x \in L$ for some certificate y.

A polynomial time reduction from a problem $L \subseteq \Sigma^*$ to a problem $L' \subseteq \Gamma^*$ is a function $f : \Sigma^* \to \Gamma^*$ for which it holds:

1. f can be computed in polynomial time.
2. $x \in L \Leftrightarrow f(x) \in L'$ for every $x \in \Sigma^*$.

A problem L is said to be NP-*hard* if every problem in NP can be reduced to L by a polynomial-time reduction. If, in addition, $L \in$ NP, then L is called NP-*complete*. Polynomial-time reductions play a central role in classifying the computational difficulty of problems and in establishing conditional lower bounds under the widely believed assumption that P $\neq$ NP.

A distinction commonly made for decision problems with numerical input data is between *weakly* and *strongly* NP-hard problems. A problem is *weakly* NP-*hard* (or *ordinarily* NP-*hard*) if it is NP-hard and it admits a pseudo-polynomial time algorithm, i.e. a deterministic algorithm with running time polynomial in both the input size *and* the magnitude of the numeric input values. In contrast, a problem is *strongly* NP-*hard* if it remains NP-hard even when all numerical parameters in the input are encoded in unary. A strongly NP-hard problem does not admit a pseudo-polynomial time algorithm unless P = NP.

Parameterized complexity. We use basic notations of parameterized complexity and algorithmics [Cyg+15, DF13]. Formally, a *parameterized decision problem* is defined as a language $L \subseteq \Sigma^* \times \mathbb{N}$ where Σ^* is the set of all words over the finite alphabet Σ. Instances of L have the form $\mathcal{I} = (I, k) \in \Sigma^* \times \mathbb{N}$ in which k is the *parameter*.

A problem L is *fixed-parameter tractable* (FPT) with respect to k if there exists a deterministic algorithm that can decide whether $\mathcal{I}$ is a yes-instance within $f(k) \cdot |I|^{O(1)}$ time for some computable function f. If such an algorithm exists, then we say L belongs to the complexity class FPT. The problem L belongs to the class W[1] if there exists a nondeterministic algorithm that decides L in $|I|^{O(1)}$ time while using at most $f(k)$ nondeterministic choices, each drawn from polynomially many candidates, all occurring within the last $f(k)$ steps of the computation, for some computable function f [CFG03].

Naturally, enumerating all nondeterministic computations generally requires $|I|^{f(k)}$ time. Such time is also known as XP-time with respect to the parameter k, where XP is a class of parameterized problems that are solvable in polynomial time when the parameter is bounded by a constant. Finally, the class paraNP consists of all parameterized problems that can be decided by a nondeterministic algorithm in $f(k) \cdot |I|^{O(1)}$ time.

Clearly, the class FPT is contained within XP, and it is also known that this containment is strict, that is FPT $\subsetneq$ XP. Further, these complexity classes are members of a set of classes that form increasing levels of parameterized computational complexity and are related by a hierarchy of inclusions. In particular, FPT forms the lowest level of this hierarchy, while W[1], W[2], and, more generally, W[i], form progressively harder classes above it, collectively known as the W-hierarchy. Formally, this yields the following chain of inclusions:

$$\text{FPT} \subseteq \text{W[1]} \subseteq \text{W[2]} \subseteq \cdots \subseteq \text{XP} \subseteq \text{paraNP}.$$

It is widely conjectured that all of these inclusions are strict; in particular, it is conjectured that FPT $\neq$ W[1], which serves as the standard hardness assumption in parameterized complexity, analogous to P $\neq$ NP in classical complexity theory.

Just as *polynomial time reductions* are used to show membership or hardness for complexity classes of the polynomial time hierarchy, one can use *parameterized reductions* to show the parameterized hardness of a problem. Let L and L' be two parameterized problems. A parameterized reduction is a function $f : (I, k) \to (I', k')$ for which it holds:

1. $f(I, k)$ can be computed in $g(k) \cdot |I|^{O(1)}$ time for some computable function g.
2. $k' < h(k)$ for some computable function h.
3. $(I, k) \in L \Leftrightarrow (I', k') \in L'$.

A parameterized problem L is said to be *hard* for a parameterized complexity class $\mathcal{C}$ if every problem $L' \in \mathcal{C}$ can be reduced to L by a parameterized reduction. If L is both hard for $\mathcal{C}$ and belongs to $\mathcal{C}$, then L is *complete* for $\mathcal{C}$. A parameterized problem is *strongly* $\mathcal{C}$-hard if it remains $\mathcal{C}$-hard when all its input numeric values are encoded in unary.

For a more detailed introduction to this topic we refer to the work of Cygan et al. [Cyg+15], Downey and Fellows [DF13], Flum and Grohe [FG06], and Niedermeier [Nie06].

Approximation algorithms. We use basic notations in approximation algorithms [Vaz13, WS11]. Formally, an α-approximation algorithm for an optimization problem is a polynomial-time algorithm that, for any instance of the problem, returns an *approximate* solution, a solution whose value is within a factor of α of the value of the optimal solution. We call α the *approximation factor*. When a maximization problem has a $(1-\epsilon)$-approximation algorithm for every $\epsilon > 0$, we say it admits a *polynomial time approximation scheme* (PTAS). In addition, if its running time is also polynomial in both the input size and $\frac{1}{\epsilon}$ we call it a *fully polynomial time approximation scheme* (FPTAS).

Chapter 3

Interval Scheduling with Eligible Machines

> *"The master manufacturer, by dividing the work to be executed into different processes, each requiring different degrees of skill or force, can purchase exactly that precise quantity of both which is necessary for each process."*
>
> — Charles Babbage, *On the Economy of Machinery and Manufactures*

In this chapter, we study interval scheduling in a setting where machines are not interchangeable: each job is compatible only with a subset of them. Such eligibility constraints naturally arise when machines differ in functionality or capacity. Our objective is throughput maximization, i.e. maximizing the weighted number of JIT jobs that can be feasibly scheduled on the given machines.

$P|M_j|\sum w_j Z_j$:

Input:	A set of n jobs $\mathcal{J} = \{1, \ldots, n\}$, a set of m machines, and a target value $W^* \in \mathbb{N}$. Each job $j \in \mathcal{J}$ is characterized by a processing time $p_j \in \mathbb{N}$, a due date $d_j \in \mathbb{N}$, weight $w_j \in \mathbb{N}$ and a set of eligible machines $M_j \subseteq \{1, \ldots, m\}$.
Question:	Is there a feasible schedule for $\mathcal{J}$ with $\sum w_j Z_j \geq W^*$?

For the $P|M_j|\sum w_j Z_j$ problem, we define a schedule as a mapping from jobs to machines. Formally, a *schedule* is a function $\sigma : \mathcal{J} \to \{i_1, i_2, \ldots, i_m, \perp\}$. When we have $\sigma(j) = i$ (with $i \neq \perp$), then job j is scheduled to be processed on machine i, otherwise when $\sigma(j) = \perp$, job j is not assigned to any machine, i.e. rejected. We say that two jobs j, j' are *in conflict* if $[d_j - p_j, d_j) \cap [d_{j'} - p_{j'}, d_{j'}) \neq \emptyset$, that is, the processing time intervals corresponding to jobs j and j' overlap and they cannot be processed on the same machine. A schedule σ is *feasible* if every scheduled job is mapped to one of its eligible machines and there is no pair of conflicting jobs j, j' with $\sigma(j) = \sigma(j') = i \neq \perp$ that is, no pair of jobs are simultaneously scheduled on some machine. The goal of $P|M_j|\sum w_j Z_j$ is to find a feasible schedule σ that maximizes the weighted number of scheduled jobs $w(\sigma) = \sum_{\sigma(j)\neq\perp} w_j$.

We point out that $P|M_j|\sum w_jZ_j$ is known to be fixed-parameter tractable when parameterized by the number n of jobs [BBN19, Bev+15]. This can be shown with a simple reduction to Multicolored Independent Set on interval graphs parameterized by the number of colors, which is known to be fixed-parameter tractable [BBN19, Bev+15]. In Observation 1 we show that both $p_{\max}$ and m are upper-bounded by $O(2^n \cdot n)$, hence Theorem 1.3 shows that the aforementioned result can be improved (from a classification standpoint) by replacing the parameter number n of jobs with the combination of the number m of machines and the largest processing time $p_{\max}$.

We can make the following observation about the relation between the number of machines m and the number of jobs n.

Observation 1 *For every instance of $P|M_j|\sum w_jZ_j$ there exists an equivalent instance such that $m \leq 2^n \cdot n$.*

Proof. Note that for every instance of $P|M_j|\sum w_jZ_j$, we can define a set of eligible jobs for every machine i, which contains the jobs that have i in their eligible machine set. More formally, we define $J(i) = \{j \mid i \in M_j\}$ as the set of eligible jobs of machine i. Now assume we have more than n machines with the same eligible job set. Then we can safely remove one of the machines since we cannot schedule jobs on more than n different machines in total. It follows that we can assume, without loss of generality, that we have at most n machines for every set of eligible jobs. The observation now follows from the fact that there are at most 2^n different sets of eligible jobs. □

Furthermore, we can make the following observation about the start and end points of the intervals of the jobs.

Observation 2 *For every instance of $P|M_j|\sum w_jZ_j$ there exists an equivalent instance where for every job j we have $d_j \in \{1, \ldots, 2n\}$ and $p_j \in \{1, \ldots, 2n\}$.*

Proof. The intervals of the jobs naturally define an interval graph G, that models the conflict relation between the jobs. Note that there are at most n different interval start points $(d_j - p_j)$ and at most n different interval endpoints (d_j). Let $<_J$ denote a linear ordering of these start and end points such that $a <_J b$ implies $a \leq b$, where an end point precedes a start point of the same value (ties among points of the same kind are broken arbitrarily). Note that the edge set of G (and hence the conflict relation of the jobs) depends only on $<_J$, and not on the numeric values of the endpoints. Hence, we can produce an equivalent instance by replacing all interval endpoints by their ordinal positions in $<_J$. □

From Observation 2 we immediately get the following relation between the largest processing time $p_{\max}$ and the number n of jobs.

Corollary 3 *For every instance of $P|M_j|\sum w_jZ_j$ there exists an equivalent instance such that $p_{\max} \leq 2n$.*

From Observation 1 and Corollary 3, we obtain that both m and $p_{\max}$ are bounded by functions of n. Therefore, FPT-tractability with respect to $m+p_{\max}$ implies FPT-tractability with respect to n, yielding a strictly finer parameterization from a classification standpoint.

3.1 Hardness

We begin with the hardness proofs. The NP-hardness is established in [BHT92, Mar06]. We establish W[1]-hardness with respect to the number of machines m (Section 3.1.1) and paraNP-hardness with respect to the maximum processing time $p_{\max}$ (Section 3.1.2). Each proof includes a construction, i.e. a reduction from an instance of a hard problem to an instance of $P|M_j|\sum w_j Z_j$, followed by a proof of its correctness.

3.1.1 The Number of Machines

In this section, we prove Theorem 1.1. To this end, we present a parameterized polynomial-time reduction from Multicolored Clique parameterized by the number of colors to $P|M_j|\sum w_j Z_j$ parameterized by the number of machines m. In Multicolored Clique, we are given a k-partite graph $G = (V_1 \uplus V_2 \uplus \ldots \uplus V_k, E)$, we are asked whether G contains a clique of size k. The k vertex parts $V_1, V_2, \ldots, V_k$ are called *colors*. Multicolored Clique parameterized by k is known to be W[1]-hard [Fel+09].

Given an instance of Multicolored Clique, we construct an instance of $P|M_j|\sum w_j Z_j$ according to Construction 1:

Construction 1 *Let $G = (V_1 \uplus V_2 \uplus \ldots \uplus V_k, E)$ be a k-partite graph with n_G vertices. Assume we have some total ordering $<_\pi$ over $V := V_1 \uplus V_2 \uplus \ldots \uplus V_k$ such that for all $u \in V_\ell$ and $v \in V_{\ell'}$ we have that if $\ell < \ell'$ then $u <_\pi v$. Let $\pi(v)$ denote the ordinal position of $v \in V$ in the ordering $<_\pi$.*

In the following section, we describe the jobs and specify their processing times, deadlines, and weights. We then describe the machines and the eligible machine sets for the jobs. To describe the weights more easily, we introduce the following three values: $c_1 = n_G + 1$, $c_2 = (k-1)n_G c_1 + n_G + 1$, and $c_3 = (kn_G + k^2 n_G)n_G c_2 + 1$. We create the following jobs:

- *For each vertex $v \in V$, we create k* vertex jobs *$j_v^{(1)}, j_v^{(2)}, \ldots, j_v^{(k)}$, where one of the vertex jobs corresponds to the color of v and the $k-1$ other vertex jobs correspond to the other $k-1$ colors. Let $v \in V_\ell$. The processing time of $j_v^{(\ell)}$ (the vertex job corresponding to the same color as v) is $k+2$, the deadline of $j_v^{(\ell)}$ is $(k+2)\pi(v)+1$, and the weight of $j_v^{(\ell)}$ is 1. The processing time of $j_v^{(\ell')}$ with $\ell' \neq \ell$ (vertex jobs corresponding to a different color than v) is 1, the deadline of $j_v^{(\ell')}$ with $\ell' \neq \ell$ is $(k+2)\pi(v) - \ell'$, and the weight of $j_v^{(\ell')}$ with $\ell' \neq \ell$ is c_1.*

- *For each edge $e = \{u, v\} \in E$ with $u \in V_\ell$, $v \in V_{\ell'}$, and $\ell < \ell'$, we create one* edge job *j_e with processing time $(k+2)(\pi(v) - \pi(u)) - \ell + \ell'$, deadline $(k+2)\pi(v) - \ell - 1$, and weight $c_2(\pi(v) - \pi(u)) + c_3$.*

- *For each color combination ℓ, ℓ' with $\ell < \ell'$ we create $|V_\ell| + |V_{\ell'}|$* color combination jobs*, one for each $u \in V_\ell$ and one for each $v \in V_{\ell'}$. Let $u \in V_\ell$, we create a job $j_u^{(\ell,\ell')}$ with processing time $(k+2)\pi(u) - \ell' - 2$, deadline $(k+2)\pi(u) - \ell' - 1$, and weight $c_2\pi(u)$.*

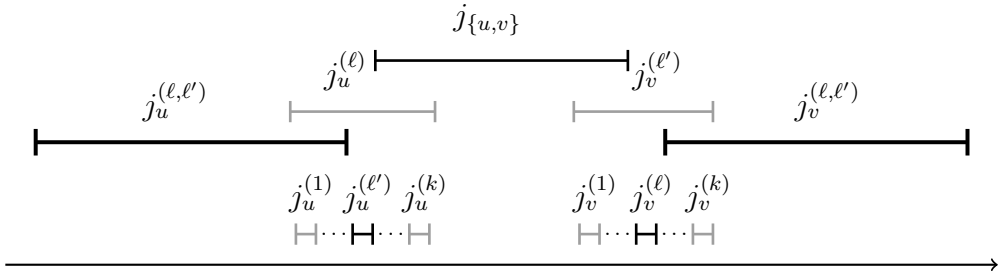


Figure 3.1: An illustration of the edge selection machine for color combination ℓ, ℓ' with $\ell < \ell'$. Depicted are intervals of jobs relating to $u \in V_\ell$, $v \in V_{\ell'}$, and $e = \{u, v\} \in E$. Gray intervals correspond to jobs that are not eligible on the machine.

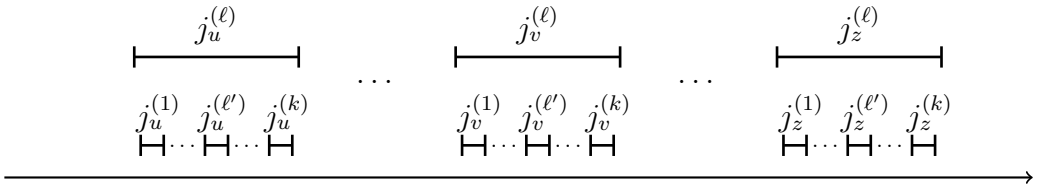


Figure 3.2: Illustration of the validation machine. Depicted are intervals of jobs corresponding to vertices $v, u, z \in V_\ell$ with $v <_\pi u <_\pi z$.

> *Let $v \in V_{\ell'}$, we create a job $j_v^{(\ell,\ell')}$ with processing time $(k+2)(n_G - \pi(v)) + \ell + 2$, deadline $(k+2)n_G + 2$, and weight $c_2(n_G - \pi(v))$.*

We create $m = \binom{k}{2} + 1$ machines $i_1, i_2, \ldots, i_{\binom{k}{2}+1}$. We call the first $\binom{k}{2}$ machines edge selection machines *(one machine for each color combination), and we call the remaining machine* validation machine*.*

Consider color combination ℓ, ℓ' with $\ell < \ell'$ and let i be the corresponding edge selection machine.

- *For each vertex $u \in V_\ell$ we add machine i to the set of eligible machines of job $j_u^{(\ell')}$ and of job $j_u^{(\ell,\ell')}$.*
- *For each vertex $v \in V_{\ell'}$ we add machine i to the set of eligible machines of job $j_v^{(\ell)}$ and of job $j_v^{(\ell,\ell')}$.*
- *For each edge $e = \{u, v\} \in E$ with $u \in V_\ell$ and $v \in V_{\ell'}$ we add machine i to the set of eligible machines of job j_e.*

We give an illustration of the edge selection machines in Figure 3.1. Finally, consider the validation machine $i_{\binom{k}{2}+1}$. We add the validation machine to the set of eligible machines of all vertex jobs. We give an illustration of the validation machine in Figure 3.2.

This finishes our construction of the $P|M_j|\sum w_j Z_j$ instance. We first show that given a clique of size k in G, we can create a feasible schedule for the constructed instance such that the total weight of scheduled jobs attains at least $W^* = \binom{k}{2}c_3 + \binom{k}{2}n_G c_2 + (k-1)n_G c_1 + k$.

Lemma 4 *Let G be an instance of* Multicolored Clique. *Let $\mathcal{I}$ be the $P|M_j|\sum w_jZ_j$ instance computed from G as specified by Construction 1. If G contains a clique of size k, then there is a feasible schedule σ for $\mathcal{I}$ such that for the total weight of scheduled jobs $w(\sigma)$ we have*

$$w(\sigma) \geq \binom{k}{2}c_3 + \binom{k}{2}n_Gc_2 + (k-1)n_Gc_1 + k.$$

Proof. Let $G = (V_1 \uplus V_2 \uplus \ldots \uplus V_k, E)$ be an instance of Multicolored Clique and consider the corresponding $P|M_j|\sum w_jZ_j$ instance specified by Construction 1. Assume there is a clique X of size k in G. Then we schedule the following jobs.

- For color combination ℓ, ℓ' with $\ell < \ell'$ let $\{u\} = X \cap V_\ell$ and $\{v\} = X \cap V_{\ell'}$. Since X is a clique in G, we know that $e = \{u, v\} \in E$. On edge selection machine i corresponding to color combination ℓ, ℓ' we schedule the following jobs: j_e, $j_u^{(\ell,\ell')}$, $j_v^{(\ell,\ell')}$, $j_u^{(\ell')}$, and $j_v^{(\ell)}$. By Construction 1, the intervals of the jobs are non-intersecting on machine i which is in the eligible set of all five jobs. Hence, scheduling these jobs on machine i yields a feasible schedule. Furthermore, it accounts for the weight $c_3 + n_Gc_2 + 2c_1$ of scheduled jobs per color combination. Summing over all color combinations, we obtain weight $\binom{k}{2}c_3 + \binom{k}{2}n_Gc_2 + k(k-1)c_1$. Note that for each $\{v\} = X \cap V_\ell$ we have scheduled all vertex jobs $j_v^{(\ell')}$ with $\ell \neq \ell'$.

- On the validation machine $i_{\binom{k}{2}+1}$ we schedule the following jobs. Let $\{v\} = X \cap V_\ell$, then we schedule vertex job $j_v^{(\ell)}$. Note that this job is only in conflict with vertex jobs $j_v^{(\ell')}$ with $\ell \neq \ell'$, which are scheduled on the edge selection machines. Furthermore, we schedule all jobs $j_u^{(\ell')}$ with $v \neq u \in V_\ell$ and $\ell \neq \ell'$. By Construction 1, all these jobs can be scheduled on machine $i_{\binom{k}{2}+1}$ without conflicts, and all these jobs have machine $i_{\binom{k}{2}+1}$ in their set of eligible machines.

  For all colors, this accounts for the weight $(n_G - k)(k-1)c_1 + k$ of scheduled jobs.

Clearly, we have that the constructed schedule is feasible. Furthermore, it is straightforward to check that the total weight of scheduled jobs in this constructed schedule is W^*.□

Before we show a similar statement for the opposite direction, we make an observation about feasible schedules in $P|M_j|\sum w_jZ_j$ instances from Construction 1. We show that we can assume that any feasible schedule where the total weight of scheduled jobs is at least $\binom{k}{2}c_3 + \binom{k}{2}n_Gc_2 + (k-1)n_Gc_1 + k$ schedules exactly one edge job on each edge selection machine.

Observation 5 *Let $\mathcal{I}$ be an instance of $P|M_j|\sum w_jZ_j$ resulting from applying Construction 1 to some k-partite graph G. Let σ be a feasible schedule such that for the total weight $w(\sigma)$ of scheduled jobs we have*

$$w(\sigma) \geq \binom{k}{2}c_3 + \binom{k}{2}n_Gc_2 + (k-1)n_Gc_1 + k.$$

Then exactly $\binom{k}{2}$ edge jobs are scheduled, one on each edge selection machine.

Proof. We first show that every schedule with weight $w(\sigma) \geq \binom{k}{2}c_3+\binom{k}{2}n_Gc_2+(k-1)n_Gc_1+k$ can schedule at most $\binom{k}{2}$ edge jobs. Let ℓ, ℓ' with $\ell < \ell'$ be a color combination. On the edge selection machine corresponding to color combination ℓ, ℓ' we have that all edge jobs corresponding to edges that do *not* connect vertices of colors ℓ and ℓ' are not eligible. Furthermore, all edge jobs corresponding to edges that connect vertices of colors ℓ and ℓ' are pairwise in conflict. It follows that on each edge selection machine, at most one edge job can be scheduled. Hence, any feasible schedule can schedule at most $\binom{k}{2}$ edge jobs, one on each edge selection machine.

We next show that any feasible schedule with $w(\sigma) \geq \binom{k}{2}c_3 + \binom{k}{2}n_Gc_2 + (k-1)n_Gc_1 + k$ needs to schedule at least $\binom{k}{2}$ edge jobs. Note that each edge job has a weight of at most $c_3 + n_Gc_2$, since $\pi(v) - \pi(u) \leq n_G$. Furthermore, there are exactly $(2k-1)n_G$ jobs that are not edge jobs, namely kn_G vertex jobs and $(k-1)n_G$ color combination jobs, and each of them has weight at most n_Gc_2. Let σ be a feasible schedule that schedules fewer than $\binom{k}{2}$ edge jobs, and let $w(\sigma)$ be the total weight of all jobs scheduled by σ. Let $\overline{W}$ denote the sum of weights of all jobs that are not edge jobs. Then we have

$$w(\sigma) \leq \left(\binom{k}{2} - 1\right)(c_3 + n_Gc_2) + \overline{W} \leq \left(\binom{k}{2} - 1\right)(c_3 + n_Gc_2) + (2k-1)n_G^2c_2 < \binom{k}{2}c_3,$$

where the last inequality holds since $c_3 = (kn_G + k^2n_G)n_Gc_2 + 1$ and $\binom{k}{2} - 1 < k^2 - k + 1 \leq (k^2 - k + 1)n_G$. Hence, the observation follows. □

Now we are ready to show how to construct a clique of size k from a feasible schedule where the total weight of scheduled jobs is at least $\binom{k}{2}c_3 + \binom{k}{2}n_Gc_2 + (k-1)n_Gc_1 + k$.

Lemma 6 *Let G be an instance of* Multicolored Clique. *Let $\mathcal{I}$ be the $P|M_j|\sum w_jZ_j$ instance computed from G as specified by Construction 1. If there is a feasible schedule σ for $\mathcal{I}$ such that for the total weight $w(\sigma)$ of scheduled jobs we have*

$$w(\sigma) \geq \binom{k}{2}c_3 + \binom{k}{2}n_Gc_2 + (k-1)n_Gc_1 + k,$$

then G contains a clique of size k.

Proof. Let $G = (V_1 \uplus V_2 \uplus \ldots \uplus V_k, E)$ be an instance of Multicolored Clique and consider the corresponding $P|M_j|\sum w_jZ_j$ instance specified by Construction 1. Assume we have a feasible schedule σ such that for the total weight $w(\sigma)$ of scheduled jobs we have $w(\sigma) \geq \binom{k}{2}c_3 + \binom{k}{2}n_Gc_2 + (k-1)n_Gc_1 + k$. We construct a clique of size k in G as follows.

By Observation 5 we know that σ schedules one edge job on each edge selection machine. We can also observe that on the validation machine, only vertex jobs are eligible and can be scheduled. Note that the sum of weights of all vertex jobs is $(k-1)n_Gc_1 + n_G$, which is strictly smaller than c_2.

Assume that edge job j_e is scheduled on the edge selection machine i corresponding to color combination ℓ, ℓ' with $\ell < \ell'$. Then by Construction 1, $e = \{u, v\} \in E$ with $u \in V_\ell$ and $v \in V_{\ell'}$. Now, by Construction 1, two color combination jobs can be scheduled on machine i, one for a vertex of color ℓ and one for a vertex of color ℓ'. To obtain a weight of scheduled jobs of at least $c_3 + n_G c_2$ it is necessary that jobs $j_u^{(\ell,\ell')}$ and $j_v^{(\ell,\ell')}$ are scheduled (note that the weights of j_e, $j_u^{(\ell,\ell')}$, and $j_v^{(\ell,\ell')}$ sum up to exactly $c_3 + n_G c_2$). Any other selection of color combination jobs to schedule either results in a weight that is lower by at least c_2 or in an infeasible schedule. Now, by Construction 1, the only further jobs that can be scheduled are $j_u^{(\ell')}$ and $j_v^{(\ell)}$. It follows that the maximum weight achievable on any edge selection machine is $c_3 + n_G c_2 + 2c_1$. Since $\binom{k}{2} 2c_1 < c_2$, it follows that for each edge selection machine corresponding to color combination ℓ, ℓ' with $\ell < \ell'$ we have the following: one edge job j_e for $e = \{u, v\}$ with $u \in V_\ell$ and $v \in V_{\ell'}$ is scheduled, and the two color combination jobs $j_u^{(\ell,\ell')}$ and $j_v^{(\ell,\ell')}$ are scheduled.

We can conclude that the jobs scheduled on all edge selection machines have a total weight of at least $\binom{k}{2} c_3 + \binom{k}{2} n_G c_2$. Hence, there are additional jobs scheduled that have a total weight of $(k-1) n_G c_1 + k$ on the validation machine.

Since no additional edge jobs or color combination jobs can be scheduled, we have that all $(k-1)n_G$ vertex jobs $j_v^{(\ell')}$ with $v \in V_\ell$ and $\ell \neq \ell'$ (having weight $c_1 > n_G$) are scheduled. Furthermore, at least k vertex jobs $j_v^{(\ell)}$ with $v \in V_\ell$ (having weight one) are scheduled.

Let X be the set of vertices in G such that if $v \in X$ and $v \in V_\ell$, the job $j_v^{(\ell)}$ is scheduled. We claim that X is a clique of size at least k in G.

By construction we have that $|X| \geq k$. Assume for contradiction that X is not a clique in G, then there are two vertices $u, v \in X$ such that $e = \{u, v\} \notin E$. Let $u \in V_\ell$ and $v \in V_{\ell'}$. Then, in particular, vertex jobs $j_u^{(\ell')}$ and $j_v^{(\ell)}$ cannot be scheduled on the validation machine, since they are in conflict with vertex jobs $j_u^{(\ell)}$ and $j_v^{(\ell')}$, respectively. However, as observed above, vertex jobs $j_u^{(\ell')}$ and $j_v^{(\ell)}$ can only be scheduled on the edge selection machine corresponding to color combination ℓ, ℓ' if there is edge job j_e with $e = \{u, v\}$ scheduled on that machine. In contradiction to the assumption that $e = \{u, v\} \notin E$. □

Finally, we have all the ingredients to prove Theorem 1.1.

Proof (Theorem 1.1). To prove Theorem 1.1, we show that Construction 1 is a parameterized polynomial-time reduction from Multicolored Clique parameterized by the number of colors to $P|M_j|\sum w_j Z_j$ parameterized by the number m of machines. First, it is easy to observe that given an instance of Multicolored Clique, the $P|M_j|\sum w_j Z_j$ instance specified by Construction 1 can be computed in polynomial time. Furthermore, if k is the number of colors in the Multicolored Clique instance, then the number of machines in the constructed $P|M_j|\sum w_j Z_j$ instance is $m = \binom{k}{2} + 1$. Lastly, observe that all weights in the constructed $P|M_j|\sum w_j Z_j$ instance are in $n_G^{O(1)}$ (where n_G is the number of vertices in the Multicolored Clique instance).

The correctness of the reduction follows from Lemmas 4 and 6. Since Multicolored Clique parameterized by the number of colors is W[1]-hard [Fel+09], Theorem 1.1 follows.□

3.1.2 Constant Processing Times

In this section, we prove Theorem 1.2. To this end, we present a polynomial-time many-one reduction from Exact (3,4)-SAT to $P|M_j|\sum w_jZ_j$ with unit weights and $p_{\max} \in O(1)$. In Exact (3,4)-SAT, we are given a Boolean formula ϕ in conjunctive normal form where every clause has exactly three literals and every variable appears in exactly four clauses, and are asked whether ϕ has a satisfying assignment. Exact (3,4)-SAT is known to be NP-hard [Tov84].

Construction 2 *Let ϕ be a Boolean formula in conjunctive normal form where every clause has exactly three literals and every variable appears in exactly four clauses. We construct an instance $\mathcal{I}$ of $P|M_j|\sum w_jZ_j$ as follows.*

- *For each variable x in ϕ, we create two machines i_x^T, i_x^F and one job j_x with deadline and processing time 25 that is eligible on machines i_x^T and i_x^F.*
- *For each clause c in ϕ we create three machines $i_c^{(1)}, i_c^{(2)}, i_c^{(3)}$ and in total nine jobs.*
- *We create three* literal jobs *$j_c^{(1)}, j_c^{(2)}, j_c^{(3)}$ all with processing time one, corresponding to the first, second, and third literal of c, respectively. Let the literal corresponding to $j_c^{(h)}$ be the k-th appearance of variable x. Then the deadline of $j_c^{(h)}$ is $2k + 8h - 8$. It is eligible on machines $i_c^{(2)}, i_c^{(3)}$ and, if x appears non-negated, then it is also eligible on machine i_x^T, otherwise it is also eligible on machine i_x^F.*
- *For each literal job, we create two* wrapper jobs*. Let $j_c^{(h)}$ be a literal job with deadline d. We create two jobs $j_{c,\alpha}^{(h)}, j_{c,\beta}^{(h)}$. Job $j_{c,\alpha}^{(h)}$ has deadline and processing time $d-1$. Job $j_{c,\beta}^{(h)}$ has deadline 25 and processing time $25-d$ (note that by construction $d \leq 24$). All wrapper jobs are eligible on machines $i_c^{(1)}, i_c^{(2)}, i_c^{(3)}$.*

All jobs have weight one. For an illustration of the literal and wrapper jobs, see Figure 3.3.

This finishes our construction of the $P|M_j|\sum w_jZ_j$ instance. We first show that given a satisfying assignment for ϕ, we can create a feasible schedule where all constructed jobs are scheduled.

Lemma 7 *Let ϕ be an instance of Exact (3,4)-SAT. Let $\mathcal{I}$ be the $P|M_j|\sum w_jZ_j$ instance computed from ϕ as specified by Construction 2. If ϕ is satisfiable, then there is a feasible schedule σ for $\mathcal{I}$ such that all jobs are scheduled.*

Proof. Let ϕ be an instance of Exact (3,4)-SAT and consider the corresponding $P|M_j|\sum w_jZ_j$ instance specified by Construction 2. Assume there is a satisfying assignment for ϕ. Then we schedule the jobs as follows.

Let x be a variable in ϕ. If x is set to true in the satisfying assignment, then we schedule job j_x on machine i_x^F. Otherwise, we schedule job j_x on machine i_x^T.

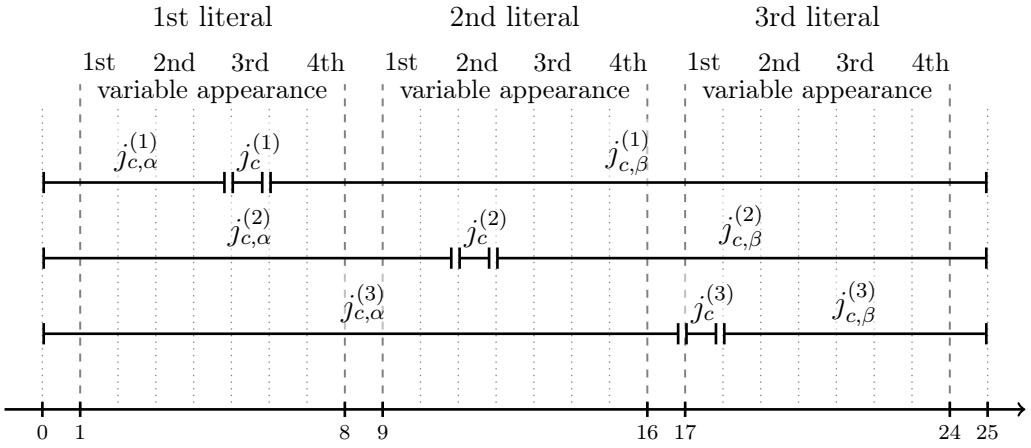


Figure 3.3: An illustration of Construction 2 when applied to the clause $c = (x \vee y \vee \neg z)$. Let c contain the third appearance of variable x, the second appearance of variable y, and the first appearance of variable z. The vertical dashed and dotted lines visualize the "slots" for the literal jobs, depending on which literal of the clause they represent and which appearance of the variable the literal represents. The eligible machines of $j_c^{(1)}$ are $\{i_x^T, i_c^2, i_c^3\}$, for $j_c^{(2)}$ these are $\{i_y^T, i_c^2, i_c^3\}$, and for $j_c^{(3)}$ the eligible machines are $\{i_z^F, i_c^2, i_c^3\}$ (as z appears negated in c).

Let c be a clause in ϕ that is satisfied by its h-th literal. Then, we schedule the nine jobs corresponding to clause c as follows. Let the other literals in c be the h'-th and h''-th literals; that is, $\{h, h', h''\} = \{1, 2, 3\}$.

- We schedule job $j_c^{(h')}$ and its two wrapper jobs $j_{c,\alpha}^{(h')}, j_{c,\beta}^{(h')}$ on machine $i_c^{(2)}$.
- We schedule job $j_c^{(h'')}$ and its two wrapper jobs $j_{c,\alpha}^{(h'')}, j_{c,\beta}^{(h'')}$ on machine $i_c^{(3)}$.
- We schedule the two wrapper jobs $j_{c,\alpha}^{(h)}, j_{c,\beta}^{(h)}$ of job $j_c^{(h)}$ on machine $i_c^{(1)}$.
- Let x be the variable appearing in the h-th literal of c. If x appears non-negated, we schedule $j_c^{(h)}$ on machine i_x^T, otherwise we schedule the job on machine i_x^F.

Note that the described schedule assigns every job to a machine in its eligible machine set. It remains to show that no two jobs that are assigned to the same machine are in conflict.

We can observe that by Construction 2, the intervals of a clause job $j_c^{(h)}$ and its two wrapper jobs $j_{c,\alpha}^{(h)}, j_{c,\beta}^{(h)}$ are pairwise non-overlapping. Hence, there are no conflicts on machines $i_c^{(1)}, i_c^{(2)}, i_c^{(3)}$ for all clauses c. In the remainder, we show that there are no conflicts on machines i_x^T, i_x^F for all variables x.

Assume there is a machine i_x^T for some variable x such that two jobs are assigned to i_x^T, which are in conflict (the case for i_x^F is analogous). Jobs eligible on machine i_x^T are j_x and jobs $j_c^{(h)}$ such that variable x appears non-negated in the h-th literal of clause c. First, note that by Construction 2, no two jobs $j_c^{(h)}, j_{c'}^{(h')}$ are in conflict, since $j_c^{(h)}$ and $j_{c'}^{(h')}$ correspond

to different appearances of x and hence have non-overlapping intervals. It follows that j_x is in conflict with some job $j_c^{(h)}$. Recall that clause c is satisfied by its h-th literal and that x appears in that literal non-negated. Hence, x is set to true in the satisfying assignment for ϕ. It follows that j_x is scheduled on machine i_x^F, a contradiction to j_x and $j_c^{(h)}$ being in conflict on machine i_x^T. □

Before we show a similar statement for the opposite direction, we make an observation about feasible schedules in $P|M_j|\sum w_j Z_j$ instances from Construction 2 that schedule all jobs.

Observation 8 *Let $\mathcal{I}$ be an instance of $P|M_j|\sum w_j Z_j$ resulting from applying Construction 2 to some instance ϕ of Exact (3,4)-SAT. Let σ be a feasible schedule such that all jobs are scheduled. Let c be a clause in ϕ. Then we have that for each $h \in \{1,2,3\}$, the two wrapper jobs $j_{c,\alpha}^{(h)}$ and $j_{c,\beta}^{(h)}$ are scheduled on the same machine. Furthermore, no three wrapper jobs are scheduled on the same machine.*

Proof. Recall that by Construction 2, the wrapper jobs $j_{c,\alpha}^{(h)}$ and $j_{c,\beta}^{(h)}$ with $h \in \{1,2,3\}$ are eligible on machines $i_c^{(1)}$, $i_c^{(2)}$, and $i_c^{(3)}$. First, note that no two jobs $j_{c,\alpha}^{(h)}$ and $j_{c,\alpha}^{(h')}$ can be scheduled on the same machine since their intervals intersect. Similarly no two jobs $j_{c,\beta}^{(h)}$ and $j_{c,\beta}^{(h')}$ can be scheduled on the same machine. From this it immediately follows that no three wrapper jobs can be scheduled on the same machine, which means that on each of the machines $i_c^{(1)}$, $i_c^{(2)}$, and $i_c^{(3)}$ exactly two wrapper jobs are scheduled.

By Construction 2 we have that for each $h \in \{1,2,3\}$, the sum of processing times of the two wrapper jobs $j_{c,\alpha}^{(h)}$ and $j_{c,\beta}^{(h)}$ is 24. Furthermore, the deadline of each wrapper job is at most 25. By Construction 2 we also have that the sum of processing times of any two wrapper jobs $j_{c,\alpha}^{(h)}$ and $j_{c,\beta}^{(h')}$ with $h \neq h'$ and non-overlapping intervals is at most 22. Moreover, we can observe that by Construction 2, the processing time of each wrapper job is odd. Hence, the sum of processing times of any two wrapper jobs is even.

From the above we have that if two wrapper jobs $j_{c,\alpha}^{(h)}$ and $j_{c,\beta}^{(h')}$ with $h \neq h'$ and non-overlapping intervals are scheduled on the same machine, say $i_c^{(1)}$, then wrapper jobs with a total processing time of at least 50 need to be scheduled on machines $i_c^{(2)}$ and $i_c^{(3)}$. Since the sum of processing times of two wrapper jobs is even, we have that on one machine two wrapper jobs with a total processing time of at least 26 need to be scheduled. However, this is not possible since the deadline of each wrapper job is at most 25. We can conclude that for each $h \in \{1,2,3\}$, the two wrapper jobs $j_{c,\alpha}^{(h)}$ and $j_{c,\beta}^{(h)}$ are scheduled on the same machine.

Now we are ready to show how to construct a satisfying assignment from a feasible schedule where all jobs are scheduled.

Lemma 9 *Let ϕ be an instance of Exact (3,4)-SAT. Let $\mathcal{I}$ be the $P|M_j|\sum w_j Z_j$ instance computed from ϕ as specified by Construction 2. If there is a feasible schedule σ for $\mathcal{I}$ such that all jobs are scheduled, then ϕ is satisfiable.*

Proof. Let ϕ be an instance of Exact (3,4)-SAT and consider the corresponding $P|M_j|\sum w_j Z_j$ instance specified by Construction 2. Assume we have a feasible schedule for the constructed instance such that all jobs are scheduled. We construct a satisfying assignment for ϕ as follows.

Let x be a variable in ϕ. Note that job j_x is only eligible on machines i_x^T and i_x^F. If j_x is scheduled on machine i_x^T, then we set x to false. Otherwise, we set x to true. We claim that this yields a satisfying assignment for ϕ.

Assume for contradiction that the above-described assignment is not satisfying. Then, there exists some clause c in ϕ that is not satisfied. Consider the nine jobs corresponding to clause c. Note that the wrapper jobs corresponding to the literals in c are only eligible on machines $i_c^{(1)}$, $i_c^{(2)}$, and $i_c^{(3)}$. By Observation 8, we have that for each $h \in \{1,2,3\}$, the wrapper jobs $j_{c,\alpha}^{(h)}$ and $j_{c,\beta}^{(h)}$ are scheduled on machine $i_c^{(\rho(h))}$ for some permutation ρ on $\{1,2,3\}$. Now consider the literal jobs $j_c^{(1)}$, $j_c^{(2)}$, and $j_c^{(3)}$. Note that all literal jobs are eligible on exactly three machines, including $i_c^{(2)}$ and $i_c^{(3)}$. In particular, no literal job is eligible on machine $i_c^{(1)}$. Furthermore, if $j_{c,\alpha}^{(h)}$ and $j_{c,\beta}^{(h)}$ are scheduled on machine $i_c^{(\rho(h))}$ for some $h \in \{1,2,3\}$ such that $\rho(h) \neq 1$, then only literal job $j_c^{(h)}$ can be scheduled on machine $i_c^{(\rho(h))}$ without creating a conflict.

It follows that the literal job $j_c^{(h)}$ with $\rho(h) = 1$ is not scheduled on any of the machines $i_c^{(2)}$ and $i_c^{(3)}$. Let variable x appear in the h-th literal of c. Assume x appears non-negated (the case where x appears negated is analogous). Then by Construction 2 we have that the literal job $j_c^{(h)}$ is eligible on machine i_x^T. Since we ruled out that job $j_c^{(h)}$ is scheduled on machine $i_c^{(2)}$ or $i_c^{(3)}$, it must be scheduled on machine i_x^T (it is only eligible on those three machines). However, then job j_x cannot be scheduled on machine i_x^T and must be scheduled on machine i_x^F. It follows that we set x to true. Since x appears non-negated in clause c, the clause is satisfied by the constructed assignment, a contradiction. □

Finally, we have all the ingredients to prove Theorem 1.2.

Proof (Theorem 1.2). To prove Theorem 1.2, we show that Construction 2 is a polynomial-time many-one reduction from Exact (3,4)-SAT to $P|M_j|\sum w_j Z_j$. First, it is easy to observe that given an instance of Exact (3,4)-SAT, the $P|M_j|\sum w_j Z_j$ instance specified by Construction 2 can be computed in polynomial time. Second, we can observe that all jobs in the constructed instance have weight one and a processing time of at most 25, that is, $p_{\max} = 25$. The correctness of the reduction follows from Lemmas 7 and 9. Since Exact (3,4)-SAT is NP-hard [Tov84], Theorem 1.2 follows. □

3.2 Positive results

3.2.1 Small Number of Machines and Small Maximum Processing Time

In this section, we provide an FPT-time algorithm for $P|M_j|\sum w_jZ_j$ with respect to the combined parameter $m + p_{\max}$, providing a proof for Theorem 1.3. The algorithm has two phases: a polynomial-time data reduction (Construction 3) that transforms every instance into an equivalent one in which every time step (d_j or $d_j - p_j$ for some $j \in \mathcal{J}$) is contained in the intervals of at most $O(m^2p_{\max}^2)$ jobs, followed by a dynamic program along the timeline that considers all possibilities of which jobs are currently processed on which machines.

The data reduction rests on the observation that some jobs are superior to others. Every job is fully characterized by its interval, weight, and set of eligible machines. Among jobs with identical intervals and identical sets of eligible machines, only the m heaviest can ever be scheduled simultaneously; we therefore keep those and discard the rest.

Construction 3 *Let $\mathcal{I}$ be an instance of $P|M_j|\sum w_jZ_j$. Recall that by Observation 2 we can assume without loss of generality that all interval start and endpoints are in $\{1,\dots,2n\}$. For every pair $(r,d) \in \{1,\dots,2n\}^2$ with $d - r \leq p_{\max}$ and every $i \in \{1,\dots,m\}$ let $S^{(i)}_{[r,d)}$ denote the set of the m highest-weighted jobs in J that have interval $[r,d)$ and are eligible on machine i. All jobs that are not contained in any set $S^{(i)}_{[r,d)}$ are discarded. Formally, the new instance $\mathcal{I}'$ has the following set of jobs from $\mathcal{I}$:*

$$\bigcup_{\substack{(r,d)\in\{1,\dots,2n\}^2\\ s.t.\ d-r\leq p_{\max},\\ i\in\{1,\dots,m\}}} S^{(i)}_{[r,d)}.$$

It is easy to see that Construction 3 is computable in polynomial time, as the set of jobs on every interval $[r,d)$ can be sorted in $O(n\log n)$ time and as there exist $O(n^2)$ pairs (r,d) with $r,d \in \{1,\dots,2n\}$. Furthermore, we observe that instances obtained from Construction 3 have the desired property regarding the number of jobs containing each time step (See Figure 3.4 for illustration).

Observation 10 *Let $\mathcal{I}$ and $\mathcal{I}'$ be two instances of $P|M_j|\sum w_jZ_j$ and let $\mathcal{I}'$ be obtained from $\mathcal{I}$ by applying Construction 3. It holds that in $\mathcal{I}'$ for every $k \in \{1,\dots,2n\}$ there exist $O(m^2p_{\max}^2)$ jobs whose interval contains k, for each of those we retain $O(m^2)$ top weighted jobs and discard the rest.*

Proof. Fix some $k \in \{1,\dots,2n\}$. For every $p \in \{1,\dots,p_{\max}\}$, there exist $O(p)$ intervals of length p that contain k; specifically the intervals $[k+i-p, k+i)$ with $i \in \{1,\dots,p\}$. Therefore there are $O(p_{\max}^2)$ different intervals that contain k. Furthermore, for every such interval and every machine i, there exist $O(m)$ jobs with the corresponding interval that can be scheduled on machine i. This follows directly from the construction of $\mathcal{I}$. Therefore, we

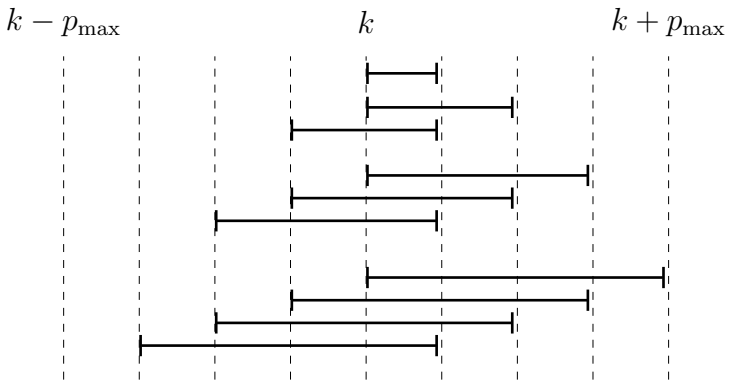


Figure 3.4: There exist $O(p_{\max}^2)$ distinct intervals that contain time step k.

have that the total number of jobs per interval is $O(m^2)$. Since there are $O(p_{\max}^2)$ intervals that contain k, the observation follows. □

We next show that a schedule σ^* is optimal for an instance produced by Construction 3 if and only if it is also optimal for the original input instance. In other words, we show that Construction 3 produces an equivalent instance.

Lemma 11 *Let $\mathcal{I}$ and $\mathcal{I}'$ be two instances of $P|M_j|\sum w_j Z_j$ and let $\mathcal{I}'$ be obtained from $\mathcal{I}$ by applying Construction 3. The instance $\mathcal{I}$ has an optimal schedule of weight W^* if and only if $\mathcal{I}'$ has an optimal schedule of weight W^*.*

Proof. Let σ^* be an optimal schedule for $\mathcal{I}'$. As the jobs in $\mathcal{I}'$ are a subset of the jobs in $\mathcal{I}$, we have that σ^* is a feasible schedule for $\mathcal{I}$. Assume toward a contradiction that σ^* is not optimal for $\mathcal{I}$. Then there exists a feasible schedule σ^{**} for $\mathcal{I}$ with $w(\sigma^{**}) > w(\sigma^*)$. Among all such schedules, choose σ^{**} to minimize the number of scheduled jobs that are not contained in $\mathcal{I}'$. The schedule σ^{**} must schedule a job j^* that belongs to $\mathcal{I}$ but not to $\mathcal{I}'$, otherwise σ^{**} would be feasible for $\mathcal{I}'$, in contradiction to the optimality of σ^* for $\mathcal{I}'$.

Suppose that $\sigma^{**}(j^*) = i$ and let the start and end point of the interval of job j^* be r and d respectively. By Construction 3, we know that j^* is at most the $m+1$ highest weighted job with interval $[r, d)$ that has i in its eligible machine set. It follows that there exists a job j^{**} in $\mathcal{I}'$ which is rejected by σ^{**}, has interval $[r, d)$, is eligible on machine i, and has weight greater or equal to w_{j^*}. Let σ^{***} denote the schedule where job j^* is replaced by j^{**}. Note that σ^{***} is feasible for $\mathcal{I}$ and note that $w(\sigma^{***}) \geq w(\sigma^{**})$. However, we also have that σ^{***} schedules fewer jobs that are not contained in $\mathcal{I}'$ than σ^{**}. This is a contradiction to our assumption on σ^{**}, and so the lemma follows. □

We present a dynamic program to compute an optimal schedule. The data reduction procedure (Construction 3) ensures that the size of the dynamic programming tables is FPT with respect to $m + p_{\max}$. At each time step k, we exhaustively check every possible configuration of jobs on the machine set, then look up in the dynamic program table for the optimal configuration of the previous time step that is compatible with the current one.

Following this strategy, we create a sequence of job *events* that we use to define sets of *currently relevant jobs* for some time step:

1. For each job j, we define the *start event* $d_j - p_j$ of j, and the *end event* d_j of j. This way, we have a total of $2n$ job events.

2. We sort all job events in non-decreasing order. If there are two jobs j and j' such that $d_j = d_{j'} - p_{j'}$, we let the end event of j be before the start event of j' in the ordering. We let $e_1, \ldots, e_{2n}$ denote the sequence of sorted events.

3. For each $k \in \{1, \ldots, 2n\}$, we create a set X_k of jobs *currently relevant* to the event e_k. Starting with $X_0 = \emptyset$, we let

   - $X_k = X_{k-1} \cup \{j\}$ if e_k is a start event for job j, and
   - $X_k = X_{k-1} \setminus \{j\}$ if e_k is an end event for job j.

In what follows, for a given $k \in \{1, \ldots, 2n\}$, we use $\mathcal{X}_k$ to denote $\bigcup_{k_0 \leq k} X_{k_0}$, *i.e*, the set of all current and previous relevant jobs. Thus, $\mathcal{X}_{2n}$ is the set of all jobs. For each $k \in \{1, \ldots, 2n\}$, we will compute a dynamic programming table T_k which stores the optimum weight of scheduled jobs in $\mathcal{X}_k$ under certain restrictions corresponding to the jobs in X_k.

Consider some $k \in \{1, \ldots, 2n\}$, and let X_k be the set of currently relevant jobs. We create a dynamic programming table $T_k : (X_k \cup \{\perp\})^m \to \mathbb{N}$ associated with X_k which is defined as follows. For an entry $\ell_1, \ldots, \ell_m \in X_k \cup \{\perp\}$, we require that $T_k[\ell_1, \ldots, \ell_m]$ equals the maximum weight of scheduled jobs in a schedule for $\mathcal{X}_k$ such that for each $i \in \{1, \ldots, m\}$:

- if $\ell_i \neq \perp$ then job ℓ_i is scheduled on machine i, and
- if $\ell_i = \perp$ then none of the jobs in X_k are scheduled on machine i.

The value of the optimal schedule will be the maximum value of any entry in T_{2n}. Note that some entries $T_k[\ell_1, \ldots, \ell_m]$ may be *invalid*: If for some $i \in \{1, \ldots, m\}$ we have that $\ell_i \neq \perp$ and $i \notin M_{\ell_i}$ (a job of X_k is scheduled on a machine that is not in its eligible machine set), or if for some $i \neq i'$ we have that $\ell_i = \ell_{i'} \in X_k$ (a job in X_k is scheduled on two different machines). We may discard such entries (or simply set their value to -1 in T_k); any other entry is a *valid entry*. As the base case of our dynamic program, we initialize the only valid entry of $X_0 = \emptyset$ by setting $T_0[\perp, \ldots, \perp] = 0$.

We next describe the recursive formula of our dynamic program. Let e_k be an event with $k \in \{1, \ldots, 2n\}$. We have two cases to consider, depending on whether the event e_k is a start event or an end event. Let $\ell_1, \ldots, \ell_m \in X_k \cup \{\perp\}$ be some valid entry.

- Suppose that e_k is a start event for some job j. Then we set

$$T_k[\ell_1, \ldots, \ell_m] = \begin{cases} T_{k-1}[\ell_1, \ldots, \ell_{i-1}, \perp, \ell_{i+1}, \ldots, \ell_m] + w_j & \text{If } \ell_i = j \text{ for some } i \in M_j, \\ T_{k-1}[\ell_1, \ldots, \ell_m] & \text{Otherwise.} \end{cases}$$

- Suppose that e_k is an end event for some job j. Then we set

$$T_k[\ell_1, \ldots, \ell_m] = \max\left\{T_{k-1}[\ell_1, \ldots, \ell_m], \quad \max_{\ell_i = \perp} T_{k-1}[\ell_1, \ldots, \ell_{i-1}, j, \ell_{i+1}, \ldots, \ell_m]\right\}.$$

Lemma 12 *The recursion above correctly computes $T_k[\ell_1, \ldots, \ell_m]$ for every $k \in \{1, \ldots, 2n\}$ and every valid entry $\ell_1, \ldots, \ell_m \in X_k \cup \{\perp\}$.*

Proof. The proof is via induction on k. For $k = 1$ the lemma statement clearly holds. Thus, let $k > 1$, and consider the job event e_k. Let j denote the job associated with e_k, and let $\ell_1, \ldots, \ell_m \in X_k \cup \{\perp\}$ be a valid entry.

First, assume that e_k is a start event (*i.e.* $e_k = d_j - p_j$). Recall that $X_k = X_{k-1} \cup \{j\}$ in this case. If $j \notin \{\ell_1, \ldots, \ell_m\}$, then all jobs $\{\ell_1, \ldots, \ell_m\} \cap X_k$ are also in X_{k-1}, and so, by induction, we have $T_k[\ell_1, \ldots, \ell_m] = T_{k-1}[\ell_1, \ldots, \ell_m]$. If $j = \ell_i$, then $i \in M_j$ otherwise $\ell_1, \ldots, \ell_m$ is invalid. Thus, we need to only consider schedules where j is scheduled on machine i. But in such schedules, no job from X_{k-1} can be scheduled on machine i, since all jobs in X_{k-1} are in conflict with j. We have by induction that $T_{k-1}[\ell_1, \ldots, \ell_{i-1}, \perp, \ell_{i+1}, \ldots, \ell_m]$ equals the maximum weight of jobs in $\mathcal{X}_{k-1}$ that can be scheduled under the condition that each job $\ell_{i'} \in \{\ell_1, \ldots, \ell_{i-1}, \ell_{i+1}, \ldots, \ell_m\}$ is scheduled on machine i' different from i. In any such schedule, we can safely schedule j on machine i, since any job of $\mathcal{X}_{k-1}$ that conflicts with j is in X_{k-1}. Scheduling job j on machine i gives an additional weight of w_j. Hence, by induction, our recursive formula is correct in this case as well.

Next, suppose that e_k is an end event (*i.e.* $e_k = d_j$). Recall that $X_k = X_{k-1} \setminus \{j\}$ in this case. If $\ell_1, \ell_2, \ldots, \ell_m \neq \perp$ then $\ell_1, \ldots, \ell_m \in X_{k-1}$, and so $T_k[\ell_1, \ldots, \ell_m] = T_{k-1}[\ell_1, \ldots, \ell_m]$ by induction. Thus, the recursion is correct in this case. If $\ell_i = \perp$ for some $i \in M_j$, then we must consider the possibility that job j is scheduled on machine i. By induction, $T_{k-1}[\ell_1, \ldots, \ell_{i-1}, j, \ell_{i+1}, \ldots, \ell_m]$ is the maximum weighted number of jobs that can be scheduled among the jobs that have a start event $k' \leq k - 1$ under the condition that each job $\ell_{i'} \in \{\ell_1, \ldots, \ell_{i-1}, \ell_{i+1}, \ldots, \ell_m\} \cap X_{k-1}$ is scheduled on machine $i' \neq i$, and job j is scheduled on machine $i \in M_j$. Hence, $\max_{\ell_i = \perp} T_{k-1}[\ell_1, \ldots, \ell_{i-1}, j, \ell_{i+1}, \ldots, \ell_m]$ gives us the maximum weighted number of scheduled jobs, assuming job j was previously scheduled on a currently available machine. If j is not scheduled, then $T_{k-1}[\ell_1, \ldots, \ell_m]$ gives us the maximum weighted number of scheduled jobs. Since all these cases are covered in the recursive formula above, the formula is correct in this case as well. □

Finally, we have all the ingredients to prove Theorem 1.3.

Proof (Theorem 1.3). Given an instance $\mathcal{I}$ of $P|M_j|\sum w_j Z_j$, apply Construction 3 to get $\mathcal{I}'$. By Lemma 11, we have that an optimal schedule for $\mathcal{I}'$ is also optimal for $\mathcal{I}$. By Lemma 12 we have that the dynamic programming algorithm described above is correct, and it can be used to compute an optimal schedule for $\mathcal{I}'$. To bound the running time of this algorithm, note that due to Observation 10 we have $|X_k| = O(m^2 p_{\max}^2)$ for every time step $k \in \{1, \ldots, 2n\}$. Thus, the size of table T_k is in $O((m \cdot p_{\max})^{2m})$. Since every table entry is computable in $O(m)$ time, it follows that we can compute the entire table in $O((m \cdot p_{\max})^{2m} \cdot mn)$ time. □

3.3 Discussion

We proved that $P|M_j|\sum w_jZ_j$ is W[1]-hard when parameterized by the number m of machines, answering an open question by Mnich and van Bevern [MB18]. Furthermore, we showed that the problem remains NP-hard even if the largest processing time $p_{\max}$ is constant. Complementing these two results, we proved that $P|M_j|\sum w_jZ_j$ is fixed-parameter tractable when parameterized by $m+p_{\max}$. Our work leaves a few natural research directions for the future, especially concerning the number m of machines as a parameter.

First, our W[1]-hardness reduction (Theorem 1.1) relies on using weights and non-uniform processing times. This raises the following two natural questions.

Open Question 1 *Is the unweighted $P|M_j|\sum Z_j$ in* FPT *when parameterized by m?*

Open Question 2 *Does $P|M_j, p_j = p|\sum w_jZ_j$ with uniform processing times admit an* FPT*-algorithm when parameterized by the number m?*

Secondly, in the reduction from Multicolored Clique that we use to prove Theorem 1.1, we construct a $P|M_j|\sum w_jZ_j$ instance with $m \in O(k^2)$ where k is the size of the required clique in the Multicolored Clique instance. This implies that assuming the Exponential Time Hypothesis (ETH) [IP01, IPZ01], there are no $f(m)(nm)^{o(\sqrt{m})}$ algorithms for $P|M_j|\sum w_jZ_j$ for any function f [Cyg+15]. The best known algorithms have a running time of $(nm)^{O(m)}$ [AS87, SV05]. Hence, there is still a gap between the upper and lower bound. Closing this gap requires either an algorithm running in $(nm)^{O(\sqrt{m})}$ time, or a reduction whose parameter blowup is linear (i.e., $m \in O(k)$), which would raise the lower bound to match the known $(nm)^{O(m)}$ upper bound.

Chapter 4

Flexible Flow Shop Interval Scheduling

"Just-in-Time means that, in a flow process, the right parts needed in assembly reach the assembly line at the time they are needed and only in the amount needed. A company establishing this flow throughout can approach zero inventory."

— Taiichi Ohno , *Toyota Motor Corporation*

In this chapter, we study interval scheduling in a two-stage flow shop setting, where each job must first be processed on a single shared machine and is then completed on one of several parallel machines. The completion of each job is required to occur exactly at its deadline, reflecting a JIT production paradigm. Such two-stage models arise naturally in manufacturing environments where a common preprocessing step is followed by parallel finishing operations, and timing requirements are tight. The objective is to select and schedule a subset of jobs, maximizing the weighted JIT throughput subject to the two-stage precedence of the flow shop.

$FF(1,m)||\sum w_jZ_j$:

Input:	A set of n jobs $\mathcal{J} = \{1, \ldots, n\}$, and a target value W^*. Each job $j \in \mathcal{J}$ is characterized by a due date $d_j \in \mathbb{N}$, the processing time $p_j \in \mathbb{N}$ of its first-stage operation, the processing time $q_j \in \mathbb{N}$ of its second-stage operation, and a weight $w_j \in \mathbb{N}$.
Question:	Is there a feasible schedule for $\mathcal{J}$ with $\sum w_jZ_j \geq W^*$?

In a traditional flow shop, each stage consists of a single machine and all jobs pass through the stages in the same fixed order. A flexible flow shop retains this fixed stage order but allows several parallel machines per stage, so that each job may be processed on any machine of a stage. This added flexibility helps balance workloads, reduce bottlenecks, and improve overall service quality; at the same time, it makes finding optimal schedules considerably harder. Indeed, as noted in the survey of Lee and Loong [LL19], approximately three-quarters of the research papers on the topic pursue heuristic approaches, a reflection

of the intrinsic difficulty of flexible flow shop problems (see also [RV10, TMT20]). In this chapter, we study the exact complexity of $FF(1,m)||\sum w_jZ_j$: we show that the general problem is hard even in its unweighted form, and identify restrictions and parameters under which efficient exact algorithms exist.

We next discuss some notation and terminology that will simplify our discussion throughout the remainder of the chapter. We will refer to the processing time p_j of the first operation of job j as the *preprocessing time of* j, and to q_j as the *processing time of* j, for any input job $j \in \mathcal{J}$. Recall that for a subset of jobs $\mathcal{J}' \subseteq \mathcal{J}$ we write $w(\mathcal{J}') = \sum_{j\in\mathcal{J}'} w_j$ for the total weight of all jobs in $\mathcal{J}'$, similarly we write $p(\mathcal{J}') = \sum_{j\in\mathcal{J}'} p_j$ for the total preprocessing time of all jobs in $\mathcal{J}'$. For a given feasible schedule, we use $\sigma = \{j \in \mathcal{J} | Z_j = 1\}$ to denote the set of jobs that complete Just-in-Time. As the remaining jobs $\mathcal{J} \setminus \sigma$ do not contribute to the $\sum w_jZ_j$ objective, we can always assume they are processed in an arbitrary order after all jobs in σ are completed. Equivalently, we may assume that only jobs in σ are *scheduled*, and all remaining jobs are rejected. Henceforth, we shall use this convention.

For a job $j \in \mathcal{J}$, let $s_j = d_j - q_j$ denote the *start time* of its second operation. Note that due to the JIT objective, the second operation of job j must be scheduled to start at time s_j if it is to be completed Just-in-Time. Thus, s_j can be viewed as a deadline for completing the first operation of any job $j \in \sigma$. It is well known that if a set of jobs can be completed no later than their due dates, then this can be done by scheduling the jobs in a non-decreasing order of due dates [Moo68]. Therefore, we have the following result:

Observation 1 *There exists an optimal schedule for any $FF(1,m)||\sum w_jZ_j$ instance where the first-stage operations of the jobs in σ are scheduled in an Early Start Time (EST) order of their second-stage operations; that is, in a non-decreasing order of the s_j values.*

Henceforth, we will assume that the input set of jobs $\mathcal{J}$ is sorted by the EST order, and so $s_1 \leq \cdots \leq s_n$. If this is not the case, we can always sort the jobs in $O(n \log n)$ time, which will be negligible in the running times of our algorithms. We say that a subset of jobs $\sigma \subseteq \mathcal{J}$ *can be preprocessed starting at time* P', for some (positive or negative) integer P', if for all jobs $j \in \sigma$ we have $P' + \sum_{i\in\sigma, i\leq j} p_i \leq s_j$. If σ can be preprocessed starting at time $P' \geq 0$, we say that σ *can be preprocessed in time*. Note that checking whether a given set σ can be preprocessed in time requires $O(n)$ time (excluding the sorting operation).

We say that two distinct jobs $i \neq j \in \mathcal{J}$ are *conflicting* if $[s_i, d_i) \cap [s_j, d_j) \neq \emptyset$, meaning that we cannot process the second operation of these jobs on the same second-stage machine. More generally, a subset of jobs $\sigma' \subseteq \mathcal{J}$ is called *independent* if there is no pair of conflicting jobs in σ'. For $m \geq 1$, an *m-schedule* for σ is a partition (allowing empty classes) of σ into $\sigma = \sigma_1 \cup \cdots \cup \sigma_m$, where σ_i is independent for each $i \in \{1, \ldots, m\}$. If such a partition exists, we say that σ *can be scheduled on m machines*, and the subset of jobs σ_i is said to be *scheduled* on (the second-stage) machine i. Using the algorithm of Carlisle and Lloyd [CL95], one can determine in $O(n \log n)$ time whether a given subset of jobs can be scheduled on m machines.

It follows from the above analysis that if a subset of jobs $\sigma \subseteq \mathcal{J}$ satisfies Conditions 1 and 2 below, then there exists a feasible schedule for $\mathcal{J}$ where σ is precisely the subset of

jobs that complete Just-in-Time. Moreover, the converse is also true. In the following, we will call such a set σ *feasible*.

- Condition 1: σ can be preprocessed in time, i.e., $\sum_{i\in\sigma, i\leq j} p_i \leq s_j$ for any $j \in \sigma$.
- Condition 2: σ can be scheduled on m machines, i.e., we can partition σ into m subsets $\sigma_1 \cup \cdots \cup \sigma_m$, where σ_i is independent for each $i \in \{1, \ldots, m\}$.

Due to Observation 1, we can check whether Condition 1 holds for a given set σ in $O(n \log n)$ time. Condition 2 can also be checked in $O(n \log n)$ time using the algorithm of Carlisle and Lloyd [CL95] for solving the $P||\sum Z_j$ problem. This can be done by including only the jobs in σ as input to the algorithm and verifying whether the optimal solution satisfies $\sum Z_j = |\sigma|$, that is, whether all input jobs can be scheduled Just-in-Time. Thus we have:

Observation 2 *Given $\sigma \subseteq \mathcal{J}$, one can determine in $O(n \log n)$ time whether there is a feasible schedule where exactly the jobs from σ are scheduled.*

The $FF(1, m)||\sum w_j Z_j$ problem is therefore reduced to finding a feasible subset $\sigma \subseteq \mathcal{J}$ of maximum weight.

4.1 Hardness

4.1.1 The Unweighted Case

In the following section, we prove the main technical result of this chapter, namely that the unweighted version of our two stage flexible flow shop problem is NP-hard.

Theorem 1.4 *$FF(1, m)||\sum Z_j$ is strongly* NP*-hard and strongly* W[2]*-hard with respect to the number of the second-stage machines m.*

To prove Theorem 1.4, we present a reduction from the Hitting Set problem. In Hitting Set we are given a family $\mathcal{F}$ of m subsets of a universe $U = \{1, \ldots, n\}$, and an integer k. We are then asked whether there exists a set $H \subseteq U$ such that $|H| = k$ and $|H \cap F| \geq 1$ for every $F \in \mathcal{F}$. The Hitting Set problem is well known to be NP-hard [Kar72]. It is also known to be W[2]-hard with respect to k [DF13], meaning that no algorithm with running time $f(k) \cdot n^{O(1)}$ is possible, for any computable function f, unless FPT = W[2]. In our reduction, we create an instance of $FF(1, m)||\sum Z_j$ where the number of machines equals the size of the solution for Hitting Set. Thus, our reduction also implies that $FF(1, m)||\sum Z_j$ is W[2]-hard with respect to m.

Given $(U, \mathcal{F}, k)$, an instance of Hitting Set, our construction creates an instance of $FF(1, k)||\sum Z_j$ with a set of jobs $\mathcal{J}$ such that a k-sized hitting set $H \subseteq U$ exists if and only if $|\sigma| = R \cdot m \cdot (2k - 1)$ jobs can be feasibly scheduled, where $R = k(n - 1) + 1$. (Note that in this section n, m, and k denote the universe size, number of sets, and the size of the hitting set, respectively, in the Hitting Set instance.)

The constructed set $\mathcal{J}$ of jobs contains *selection jobs* and *dummy jobs*. The time axis is subdivided into $k \cdot (n-1)+1$ *segments*, each subdivided into m *epochs*. In each segment, the selection jobs of the j-th epoch correspond to the elements of the j-th set in $\mathcal{F}$, denoted F_j, while the dummy jobs of the j-th epoch correspond to all elements of U (including the ones in F_j).

The key idea is that in each epoch j, one second-stage machine executes a selection job (thereby encoding an element that hits the set F_j), while the other $k-1$ machines execute two dummy jobs each (this property is achieved by carefully designing the preprocessing times). Therefore, if there exists a segment such that in all the epochs of this segment every second-stage machine i executes only selection jobs corresponding to one specific element $u_i \in U$, then the set $\{u_i \mid i \in \{1,\dots,k\}\}$ will form a hitting set.

To ensure that at least one such segment exists, the selection jobs are designed in such a way that whenever a second-stage machine executes a selection job that corresponds to an element $i \in U$, the selection jobs this machine can execute afterwards must correspond to elements $i' \in U$ where $i' \geq i$. In other words, the elements of U that correspond to the selection jobs executed on each machine cannot decrease over time. Thus, after sufficiently many segments, there is at least one segment where each machine does not contain selection jobs corresponding to two different elements of U.

Let u_j be an element of F_i. Scheduling the selection job of u_j on the k'-th machine in the i-th epoch corresponds to *selecting* u_j as the k'-th element of the hitting set. The preprocessing times are defined such that exactly one selection job and exactly $2(k-1)$ dummy jobs must be scheduled in each epoch to obtain a feasible instance with $|\sigma|$ jobs. The intervals of the jobs are defined such that the jobs (dummy and selection) that correspond to u_j at some epoch conflict with the corresponding jobs of $u_{j'}$ in the previous epoch with $j' > j$. Hence, the scheduled jobs of each epoch must be compatible with the scheduled jobs of the previous epoch. The large number of segments ensures that at some segment, all selection jobs correspond to at most k elements of U and therefore a hitting set must exist if a $|\sigma|$-sized schedule exists.

Construction 4 *Let $(U, \mathcal{F}, k)$ be an instance of* Hitting Set *with $U = \{1,\dots,n\}$ and with $\mathcal{F} = \{F_1,\dots,F_m\}$. Our reduction uses the following two integers throughout:*

- *the number $R = k \cdot (n-1)+1$ of segments, and*
- *a sufficiently large number Q, more precisely $Q = (k-1)\cdot(n+1)$.*

We construct an instance $\mathcal{J}$ of $FF(1,k)||\sum Z_j$ as follows. The time axis is divided into R segments*, each consisting of m* epochs*. For any $r \in \{0,\dots,R-1\}$ and $j \in \{1,\dots,m\}$, define the function $g(r,j) = r \cdot m + j$, that gives the lexicographic position of the pair (r,j). We denote the j-th epoch of the r-th segment by T_j^r. This epoch starts at time $G(r,j) = g(r,j)^2 \cdot Q$ and ends at the start of the next epoch (if such exists); either at time $G(r,j+1)$ or, if $j = m$, at time $G(r+1,1)$ where $G(R,1) = (R \cdot m + 1)^2 \cdot Q$. Note that the first epoch begins at time Q, rather than at time 0, and that the length of every epoch T_j^r is $[(g(r,j)+1)^2 - g(r,j)^2] \cdot Q = (2 \cdot g(r,j)+1)\cdot Q$.*

We will assign the jobs to the epochs based on their starting times. That is, we say that a job belongs *to epoch T_j^r if the starting time of its second-stage operation is contained in T_j^r. The due date of a job in epoch T_j^r might either be in T_j^r or in the next epoch. For $r \in \{0, \ldots, R-1\}$ and $j \in \{1, \ldots, m\}$, our reduction creates the following jobs that belong to T_j^r:*

- *For every $i \in F_j$, we add one* selection job $S_{i,j}^r$ *with*
  - *a preprocessing time of $p(S_{i,j}^r) = 0$,*
  - *a processing time of $q(S_{i,j}^r) = (2g(r,j)+1) \cdot Q$, and*
  - *a due date of $d(S_{i,j}^r) = G(r,j) + (2g(r,j)+1) \cdot Q + i$.*
- *For every $i \in \{1, \ldots, n\}$, we add a dummy job $A_{i,j}^r$ with*
  - *a preprocessing time of $p(A_{i,j}^r) = \frac{1}{k-1} \cdot g(r,j) \cdot Q = g(r,j) \cdot (n+1)$,*
  - *a processing time of $q(A_{i,j}^r) = g(r,j) \cdot Q$, and*
  - *a due date $d(A_{i,j}^r) = G(r,j) + g(r,j) \cdot Q + i$.*
- *For every $i \in \{1, \ldots, n\}$, we add an additional dummy job $B_{i,j}^r$ with*
  - *a preprocessing time of $p(B_{i,j}^r) = \frac{1}{k-1} \cdot g(r,j) \cdot Q = g(r,j) \cdot (n+1)$,*
  - *a processing time of $q(B_{i,j}^r) = (g(r,j)+1) \cdot Q$, and*
  - *a due date of $d(B_{i,j}^r) = G(r,j) + (2g(r,j)+1) \cdot Q + i$.*

Note that $s(B_{i,j}^r) = G(r,j) + g(r,j) \cdot Q + i = d(A_{i,j}^r)$ and for $r > 0$,

$$s(A_{i,j}^r) = G(r,j) + i = \begin{cases} G(r,j-1) + (2 \cdot g(r,j-1)+1) \cdot Q + i & \text{if } j \neq 1 \\ G(r-1,m) + (2 \cdot g(r-1,m)+1) \cdot Q + i & \text{if } j = 1 \end{cases}$$
$$= \begin{cases} d(B_{i,j-1}^r) & \text{if } j \neq 1 \\ d(B_{i,m}^{r-1}) & \text{if } j = 1 \end{cases}.$$

Thus, the dummy job $A_{i,j}^r$ ends when $B_{i,j}^r$ starts, and for $r > 0$, $A_{i,1}^r$ starts when $B_{i,m}^{r-1}$ ends, and $A_{i,j}^r$, for $j > 1$, starts when $B_{i,j-1}^r$ ends.

We use $\mathcal{J}$ to denote the set

$$\mathcal{J} = \{S_{i,j}^r \mid 0 \leq r \leq R-1, 1 \leq j \leq m, i \in F_j\} \cup \{A_{i,j}^r, B_{i,j}^r \mid 0 \leq r \leq R-1, 1 \leq i \leq n, 1 \leq j \leq m\}$$

of all jobs in our constructed $FF(1,k)||\sum Z_j$ instance. The desired cardinality of a feasible solution is set to be $R \cdot m \cdot (2k-1)$. See Figure 4.1 for a visualization of our construction.

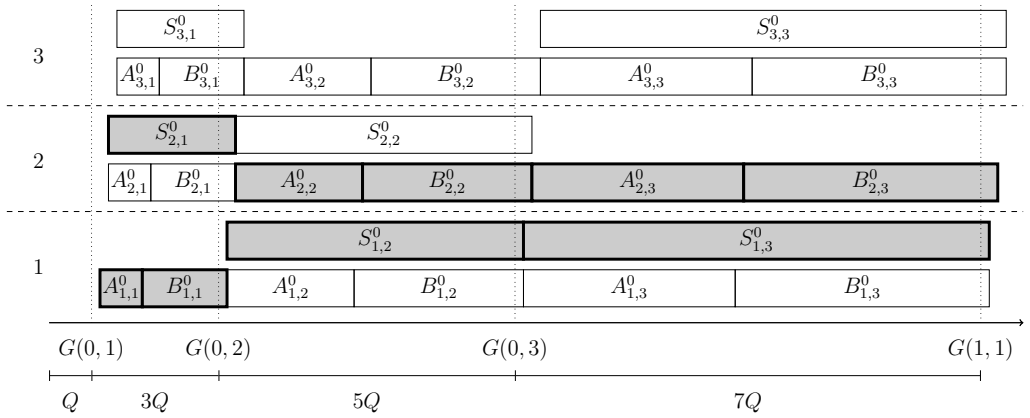


Figure 4.1: An example of the reduction for the HITTING SET instance $(U, \mathcal{F}, k)$ with $U = \{1,2,3\}$, $\mathcal{F} = \{F_1 = \{2,3\}, F_2 = \{1,2\}, F_3 = \{1,3\}\}$, and $k = 2$. For such an instance, the number of segments is $R = k \cdot (n-1)+1 = 5$, and $Q = (k-1) \cdot (n+1) = 4$. The figure depicts only the second-stage operations starting in the first segment. The vertical dotted lines separate the $m = 3$ epochs. The schedule created in the proof of Lemma 3 for the hitting set $H = \{1,2\}$ is marked in gray.

We continue by proving the correctness of the reduction, namely that $\mathcal{F}$ has a hitting set of size k if, and only if, there is a feasible solution $\sigma \subseteq \mathcal{J}$ of size $R \cdot m \cdot (2k-1)$. We begin with the forward direction:

Lemma 3 *If $H = \{i(1), \ldots, i(k)\} \subseteq \{1, \ldots, n\}$ is a hitting set of $\mathcal{F} = \{F_1, \ldots, F_m\}$, then there is a feasible solution $\sigma \subseteq \mathcal{J}$ with $|\sigma| = R \cdot m \cdot (2k-1)$.*

Proof. For each $j \in \{1, \ldots, m\}$, define $i_j \in H$ to be the minimum element in $F_j \cap H$; such an i_j always exists as $H = \{i(1), \ldots, i(k)\}$ is a hitting set. For each $r \in \{0, \ldots, R-1\}$ and each $j \in \{1, \ldots, m\}$, we schedule on the k'-th machine, for $k' \in \{1, \ldots, k\}$, the selection job $S^r_{i(k'),j}$ if $i(k') = i_j$, and the two dummy jobs $A^r_{i(k'),j}$ and $B^r_{i(k'),j}$, otherwise. (See Figure 4.1 for an illustration.) Let σ denote the set of all scheduled jobs.

It is easy to verify that σ can be scheduled on k machines: For machine k', the jobs in epoch T^r_j which are scheduled on machine k' are either the selection job $S^r_{i(k'),j}$ or the two dummy jobs $A^r_{i(k'),j}$ and $B^r_{i(k'),j}$. In both cases, these jobs together are scheduled starting from time $G(r,j) + i(k')$ until time

$$G(r,j) + (2g(r,j)+1) \cdot Q + i(k') = \begin{cases} G(r, j+1) + i(k') & \text{if } j < m \\ G(r+1, 1) + i(k') & \text{if } j = m\,. \end{cases}$$

Therefore, the jobs scheduled on machine k' are not conflicting.

Furthermore, in every epoch, one selection job and $2 \cdot (k-1)$ dummy jobs are scheduled, and so $|\sigma| = R \cdot m \cdot (2k-1)$. Thus, it remains to show that σ can be preprocessed in time. By Observation 1, we can assume that the jobs in σ are preprocessed in EST order. The selection jobs do not require any preprocessing time, and thus we only need to argue about the dummy jobs in σ.

Consider a dummy job $A^r_{i,j} \in \sigma$. The dummy jobs in $\sigma \setminus \{A^r_{i,j}\}$ with a smaller starting time than $A^r_{i,j}$ (that are therefore preprocessed before $A^r_{i,j}$) are:

- $k-1$ pairs of dummy jobs for each $r' \in \{0,\dots,r\}$ and $j' \in \{1,\dots,m\}$ such that $g(r',j') < g(r,j)$. Specifically, for a fixed r' and j', the $k-1$ pairs are $A^{r'}_{i',j'}$ and $B^{r'}_{i',j'}$ for every $i' \in H \setminus \{i_{j'}\}$.
- Up to $k-2$ different dummy jobs $A^r_{i',j}$ for $i' \in H \setminus \{i_j, i\}$.

Consequently, the preprocessing of $A^r_{i,j}$ is completed no later than time

$$\begin{aligned}
&p(A^r_{i,j}) + \sum_{i' \in H \setminus \{i_j,i\}} p(A^r_{i',j}) + \sum_{g(r',j')<g(r,j)} \sum_{i' \in H \setminus \{i_{j'}\}} p(A^{r'}_{i',j'}) + p(B^{r'}_{i',j'}) \\
&= \frac{1}{k-1} \cdot g(r,j) \cdot Q + \frac{k-2}{k-1} \cdot g(r,j) \cdot Q + 2 \cdot \sum_{\ell=1}^{g(r,j)-1} \ell \cdot Q \\
&= g(r,j) \cdot Q + 2 \cdot \frac{g(r,j) \cdot (g(r,j)-1)}{2} \cdot Q = g(r,j)^2 \cdot Q \\
&= G(r,j) \;<\; G(r,j) + i \;=\; s(A^r_{i,j}).
\end{aligned}$$

Let $B^r_{i,j} \in \sigma$. Analogously, the other dummy jobs in $\sigma \setminus \{B^r_{i,j}\}$ with a smaller starting time than $B^r_{i,j}$ are

- $k-1$ pairs of dummy jobs for each $r' \in \{0,\dots,r\}$ and $j' \in \{1,\dots,m\}$ with $g(r',j') < g(r,j)$,
- $k-1$ different dummy jobs $A^r_{i',j}$ for $i' \in \{i(1),\dots,i(k)\} \setminus \{i_j\}$, and
- up to $k-2$ different dummy jobs $B^r_{i',j}$ for $i' \in \{i(1),\dots,i(k)\} \setminus \{i_j, i\}$.

Consequently, the preprocessing of $B^r_{i,j}$ is completed by time

$$\begin{aligned}
&p(B^r_{i,j}) + \sum_{i' \in H \setminus \{i_j\}} p(A^r_{i',j}) + \sum_{i' \in H \setminus \{i_j,i\}} p(B^r_{i',j}) + \sum_{g(r',j')<g(r,j)} \sum_{i' \in H \setminus \{i_{j'}\}} p(A^{r'}_{i',j'}) + p(B^{r'}_{i',j'}) \\
&= p(B^r_{i,j}) + \sum_{i' \in H \setminus \{i_j,i\}} p(B^r_{i',j}) + g(r,j)^2 \cdot Q \;=\; G(r,j) + g(r,j) \cdot Q \\
&< G(r,j) + g(r,j) \cdot Q + i \;=\; s(B^r_{i,j}).
\end{aligned}$$

Thus, the set of jobs σ can indeed be preprocessed in time. □

We continue with the backward direction. In order to prove the correctness of the backward direction, we aim to show that a feasible solution σ for our constructed $FF(1,k)||\sum Z_j$ instance schedules exactly one selection job and $2 \cdot (k-1)$ dummy jobs in each epoch. As a first step towards this, we start by upper bounding the number of scheduled dummy jobs per epoch that may be included in σ.

Lemma 4 *Let $\sigma \subseteq \mathcal{J}$ be a feasible solution. Then the number of dummy jobs in σ is at most $2(k-1) \cdot R \cdot m$. If σ contains $2(k-1) \cdot R \cdot m$ dummy jobs, then it contains exactly $2(k-1)$ dummy jobs in each epoch.*

Proof. To prove the lemma, we first prove the following claim: For any $\ell \in \{1, \ldots, R \cdot m\}$, at most $2(k-1) \cdot \ell$ dummy jobs belonging to the first ℓ epochs are included in σ. Suppose that the claim is not true, and let ℓ be the minimum value in $\{1, \ldots, R \cdot m\}$ for which the claim is false. Then, there are at least $2(k-1) \cdot \ell + 1$ dummy jobs from the first ℓ epochs in σ. Let $\sigma_\ell \subseteq \sigma$ denote these dummy jobs, and let $r \in \{0, \ldots, R-1\}$ and $j \in \{1, \ldots, m\}$ be such that $\ell = g(r, j)$. Since, in our construction, the preprocessing time of dummy jobs increases with ℓ, we can lower-bound the total preprocessing time of the jobs in σ_ℓ by taking exactly $2(k-1)$ dummy jobs from each of the first $\ell - 1$ epochs, and $2(k-1)+1$ dummy jobs from the ℓ-th epoch. Thus, the preprocessing time of these dummy jobs is at least

$$\begin{aligned} p(\sigma_\ell) &\geq \left(\sum_{(r',j'):g(r',j')<\ell} 2(k-1) \cdot p(A^{r'}_{1,j'}) \right) + \left((2(k-1)+1) \cdot p(A^{r}_{1,j}) \right) \\ &= \left(\sum_{\ell'=1}^{\ell-1} 2\ell' \cdot Q \right) + \left((2(k-1)+1) \cdot \frac{1}{k-1} \cdot \ell \cdot Q \right) \\ &= \left(\ell^2 \cdot Q - \ell \cdot Q \right) + \left(2\ell \cdot Q + \frac{\ell}{k-1} Q \right) \\ &= \ell^2 \cdot Q + \ell \cdot Q + \ell \cdot (n+1) \\ &> G(r,j) + g(r,j) \cdot Q + n \;=\; s(B^r_{n,j}). \end{aligned} \tag{4.1}$$

Thus, the last job in σ_ℓ completes its preprocessing after the latest starting time of a dummy job belonging to T^r_j, a contradiction.

To complete the proof of the lemma, it remains to show that if $|\sigma| = 2(k-1) \cdot R \cdot m$, then σ contains exactly $2(k-1)$ dummy jobs from each epoch. By the claim shown in the beginning of the proof and since the preprocessing time of dummy jobs increases with ℓ, we can lower-bound the total preprocessing time of the jobs in σ by taking $2(k-1)$ dummy jobs from each epoch. Further, the claim implies that for each epoch T^r_j containing more than $2(k-1)$ dummy jobs, there has to be an earlier epoch $T^{r'}_{j'}$ with fewer than $2(k-1)$ dummy jobs. Thereby, each epoch with more than $2(k-1)$ dummy jobs increases the lower bound of the total preprocessing time by

$$p(A^r_{i,j}) - p(A^{r'}_{i',j'}) = \big(g(r,j) - g(r',j')\big) \cdot \frac{Q}{k-1} = \big(g(r,j) - g(r',j')\big) \cdot (n+1) \geq n+1\,.$$

Thus, analogous to the calculations in Equation (4.1), if there is at least one epoch that contains fewer or more than $2(k-1)$ dummy jobs, then the total preprocessing time of all

the dummy jobs σ is

$$\begin{aligned} p(\sigma) &\geq \left(\sum_{(r,j):g(r,j)\leq g(R-1,m)} 2(k-1)\cdot p(A^r_{1,j}) \right) + (n+1) = \left(\sum_{\ell=1}^{R\cdot m} 2\ell \cdot Q \right) + n + 1 \\ &= G(R-1,m) + g(R-1,m)\cdot Q + n + 1 > s(B^{R-1}_{n,m}). \end{aligned}$$ □

Therefore, the preprocessing time of σ is larger than the latest starting time of a dummy job, a contradiction.

Next, using Lemma 4 above, we show that any feasible solution of size $R \cdot m \cdot (2k-1)$ includes exactly $2(k-1)$ dummy jobs and one selection job per epoch.

Lemma 5 *Let $\sigma \subseteq \mathcal{J}$ be a feasible solution with $|\sigma| = R \cdot m \cdot (2k-1)$. Then σ contains exactly $2(k-1)$ dummy jobs and one selection job belonging to each epoch T^r_j.*

Proof. Suppose there is a feasible solution σ with $|\sigma| = R\cdot m\cdot(2k-1)$ that contains $R\cdot m+i$ many selection jobs for some $i > 0$. Note that in every epoch, any machine on which a selection job is scheduled cannot schedule another job, while any machine without a selection job can schedule up to two dummy jobs. Since there are $R \cdot m$ epochs in total, it follows that the total number of scheduled jobs is at most

$$R \cdot m + i + 2 \cdot (R \cdot m \cdot k - (R \cdot m + i)) = R \cdot m \cdot (2k-1) - i < |\sigma|\,,$$

a contradiction. Thus, σ includes at most $R \cdot m$ selection jobs, at most one from each epoch. Since $|\sigma| = R \cdot m \cdot (2k-1)$, it includes at least $2(k-1) \cdot R \cdot m$ many dummy jobs, and by Lemma 4 it then follows that σ includes a single selection job and exactly $2(k-1)$ dummy jobs per epoch. □

We can now proceed with proving the backward direction:

Lemma 6 *If there is a feasible solution $\sigma \subseteq \mathcal{J}$ with $|\sigma| = R \cdot m \cdot (2k-1)$, then there is a hitting set $H \subseteq U$ of size k.*

Proof. Let $\sigma \subseteq \mathcal{J}$ be a feasible solution with $|\sigma| = R \cdot m \cdot (2k-1)$. For $k' \in \{1,\ldots,k\}$ and $r \in \{0,\ldots,R-1\}$, let $i(r,k') \in \{1,\ldots,n\}$ denote the offset from $G(r,1)$ at which the first job belonging to segment r is scheduled on machine k'. In other words, there exists a job in σ that starts at time $G(r,1) + i(r,k')$ on machine k' (and consequently no job starts at time $G(r,1) + i'$, for $0 < i' < i(r,k')$ on machine k'). The existence of such an integer in $\{1,\ldots,n\}$ is guaranteed by our construction and Lemma 5. By our construction, the total processing time of the jobs belonging to any epoch T^r_j that are scheduled on machine k' is $(2g(r,j)+1)\cdot Q$, regardless of whether these are two dummy jobs or a single selection job. Therefore, the total processing time of jobs belonging to segment r that are scheduled

on machine k' is

$$\begin{aligned}\sum_{j=1}^{m}(2g(r,j)+1)\cdot Q &= \sum_{j=1}^{m}(2\cdot(rm+j)+1)\cdot Q\\ &= \Big(2rm^2+m(m+1)+m\Big)\cdot Q\\ &= \Big(m^2+2\cdot(rm+1)\cdot m\Big)\cdot Q\\ &= \Big((rm+1+m)^2-(rm+1)^2\Big)\cdot Q\\ &= G(r+1,1)-G(r,1).\end{aligned}$$

It follows that every machine k' that begins processing the jobs of σ belonging to segment r at time $G(r,1)+i(r,k')$, finishes processing the last of these jobs at least at time $G(r+1,1)+i(r,k')$.

For each machine k', let $C_{k'}$ denote the completion time of its last job in segment $R-1$, and define $i(R,k') = C_{k'} - G(R,1)$. The last job is either a selection job or a B-job in the final epoch, so $i(R,k') \in \{1,\dots,n\}$. By the preceding processing-time bound, $i(R-1,k') \le i(R,k')$.

Feasibility between consecutive segments, together with the terminal offset defined above, gives

$$i(0,k') \le i(1,k') \le \cdots \le i(R,k')$$

for every $k' \in \{1,\dots,k\}$. Since all these offsets belong to $\{1,\dots,n\}$, each machine has at most $n-1$ strict increases. Thus, among the $R = k(n-1)+1$ transitions indexed by $r \in \{0,\dots,R-1\}$, at most $k(n-1)$ have a strict increase on any machine. Consequently, there exists $r' \in \{0,\dots,R-1\}$ such that $i(r',k') = i(r'+1,k')$ for every machine k'.

For this segment, the total processing time on each machine equals the time from its first job's start to the next segment's first job's start, or to its final completion when $r' = R-1$. Hence, there is no idle time within the segment. By the construction, consecutive jobs on a machine can meet without idle time only when their offsets agree. Therefore, every selection job scheduled on machine k' in segment r' corresponds to element $i(r',k')$.

We claim that $H = \{i(r',1),\dots,i(r',k)\} \subseteq \{1,\dots,n\}$ is a hitting set of $\mathcal{F}$. Note that for each $F_j \in \mathcal{F}$, Lemma 5 implies that there is one selection job $S_{i,j}^{r'}$ scheduled in epoch $T_j^{r'}$. Let $k' \in \{1,\dots,k\}$ be the machine on which job $S_{i,j}^{r'}$ is scheduled. Then $i(r',k') = i(r'+1,k')$ implies that $i = i(r',k')$, and consequently F_j is hit by $i(r',k')$ (otherwise job $S_{i,j}^{r'}$ would not exist). □

Proof (Theorem 1.4). The reduction presented above clearly runs in polynomial time, and all occurring numbers are clearly polynomial in the size of the Hitting Set instance. The correctness of this reduction is implied by Lemmas 3 and 6. □

4.2 Positive Results

4.2.1 A Dynamic Program for Fixed Number of Machines

In the following section, we consider the case where the number of second-stage machines m is some fixed constant, *i.e.* $m \in O(1)$. We prove that in this case, $FF(1,m)||\sum w_j Z_j$ is solvable in pseudo-polynomial time via dynamic programming. This algorithm also provides an $O(n)$ speedup over the previous fastest algorithm for the $FF(1,1)||\sum w_j Z_j$ problem [SB12].

Recall that we assume that the input set of jobs $\mathcal{J}$ is ordered according to the EST order, so that $s_1 \leq \cdots \leq s_n$. To this end, let $X \subseteq \mathcal{J}$ with $|X| \leq m$, and write $X = \{x_1, \ldots, x_{|X|}\}$ with $x_1 < x_2 < \cdots < x_{|X|}$. We say that a feasible schedule is *compatible* with X if

- only jobs from $\{x_i, \ldots, n\}$ are scheduled on machine $i \in \{1, \ldots, |X|\}$, and
- no jobs are scheduled on machine $i \in \{|X|+1, \ldots, m\}$.

We say that a subset of jobs $\sigma \subseteq \mathcal{J}$ is *compatible* with X if σ can be scheduled on $|X|$ machines by a schedule that is compatible with X. Our dynamic programming table T will be indexed by subsets $X \subseteq \mathcal{J}$, $|X| \leq m$, and integers $W' \in \{-W, \ldots, W\}$. Throughout the algorithm, we will maintain the following invariant:

> $T[X, W'] =$ the maximum value P' such that there exists a feasible solution $\sigma \subseteq \mathcal{J}$ with $w(\sigma) = W'$ which is compatible with X, and can be preprocessed starting at time P'.

Note that we allow P' to have a negative value. If no feasible solution $\sigma \subseteq \mathcal{J}$ with $w(\sigma) = W'$ that is compatible with X exists, we define $T[X, W'] = -\infty$. Thus, assuming the above invariant holds for each entry, there exists a feasible solution $\sigma \subseteq \mathcal{J}$ of total weight $w(\sigma) \geq W^*$ if and only if $T[\{1, \ldots, m\}, W'] \geq 0$ for some $W' \geq W^*$. For the base case of our computation, we set $T[X, 0] = +\infty$ for all $X \subseteq \mathcal{J}$ with $|X| \leq m$, and $T[\emptyset, W'] = -\infty$ for all $W' \neq 0$. We also set $T[X, W'] = -\infty$ for all $X \neq \emptyset$ and $W' < 0$. Clearly, the invariant above holds for all these entries. We proceed to describe a recursive formula for the remaining entries of the table.

Let $X \subseteq \mathcal{J}$ with $X \neq \emptyset$, and let $W' \in \{0, \ldots, W\}$. Let j be the minimum job index in X, *i.e.* $j = \min\{j^* \in X\}$. We define two additional job indices $j_1 = \min\{j^* \in \mathcal{J} \setminus X \mid j < j^*\}$, and $j_2 = \min\{j^* \in \mathcal{J} \setminus X \mid d_j \leq s_{j^*}\}$. In other words, $j_1 \notin X$ is the minimum job index that is larger than j, while $j_2 \notin X$ is the minimum job index such that the start time of j_2 is at least the due date of job j. Note that j_1 and j_2 may not exist, but if they do, $j < j_1 \leq j_2$. Define the subset $X_1 \subseteq \mathcal{J}$ to be $X_1 = (X \setminus \{j\}) \cup \{j_1\}$ if j_1 exists, and $X_1 = X \setminus \{j\}$ otherwise. Similarly, define $X_2 = (X \setminus \{j\}) \cup \{j_2\}$ if j_2 exists, and $X_2 = X \setminus \{j\}$ otherwise. Then we compute $T[X, W']$ by

$$T[X, W'] = \max \begin{cases} T[X_1, W'], \\ \min\{T[X_2, W' - w_j], \ \ s_j\} - p_j. \end{cases} \tag{4.2}$$

Intuitively speaking, the first case in the recursion corresponds to the situation where job $j = \min\{j^* \in X\}$ is not added to the computed optimal feasible solution σ. The set X_1 and index $j_1 = \min\{j^* \in \mathcal{J} \setminus X \mid j < j^*\}$ are defined accordingly to adhere to our invariant. The second case corresponds to including job j. Let σ denote a set of remaining selected jobs, of total weight $W' - w_j$, compatible with X_2, that attains the latest feasible preprocessing start time $T[X_2, W' - w_j]$. Since j is preprocessed first, its preprocessing must finish no later than both s_j and $T[X_2, W' - w_j]$. Thus, the latest feasible preprocessing start time for $\sigma \cup \{j\}$ is $\min\{T[X_2, W' - w_j], s_j\} - p_j$. If no such remaining solution exists, this case is infeasible. This intuition is formalized in the lemma below.

Lemma 7 *The above recursion correctly computes $T[X, W']$ for all subsets $X \subseteq \mathcal{J}$, $|X| \leq m$, and all integers $W' \in \{-W, \ldots, W\}$.*

Proof. The correctness of the recursion above can be verified by induction. As the base cases maintain our invariant, we proceed to argue about the remaining entries. So let $X \subseteq \mathcal{J}$ with $X \neq \emptyset$, and let $W' \in \{1, \ldots, W\}$. Let $j, j_1, j_2 \in \mathcal{J}$ be the job indices associated with X, and let X_1 and X_2 be defined as above. Assuming the entries $T[X_1, W']$ and $T[X_2, W' - w_j]$ maintain the invariant above, we show that $T[X, W']$ also maintains this invariant. For this, we start by showing that the value of $T[X, W']$ is at least the right-hand side of the recursive formula.

- First, suppose that $T[X_1, W'] > -\infty$, and let $\sigma_1 \subseteq \mathcal{J}$ be a feasible solution of total weight $w(\sigma_1) = W'$ that is compatible with X_1 and can be preprocessed starting at time $T[X_1, W']$. Note that σ_1 is also compatible with X, since if j_1 does not exist we have $X_1 \subset X$, and otherwise $X_1 = (X \setminus \{j\}) \cup \{j_1\}$ and $j < j_1$. Thus, $T[X, W'] \geq T[X_1, W']$.

- Next, suppose that $T[X_2, W' - w_j] > -\infty$, and let $\sigma_2 \subseteq \mathcal{J}$ be a feasible solution of total weight $w(\sigma_2) = W' - w_j$ that is compatible with $X_2 = \{x_1^2, \ldots, x_{|X_2|}^2\}$ and can be preprocessed starting at time $T[X_2, W' - w_j]$.

  - If j_2 does not exist, then $X_2 = X \setminus \{j\}$, and σ_2 can be scheduled on $|X_2| = |X| - 1$ machines. Thus, $\sigma_2 \cup \{j\}$ can be scheduled on $|X|$ machines, and is compatible with $X = \{j, x_1^2, \ldots, x_{|X_2|}^2\}$, since we can schedule job j on its own machine. As σ_2 can be preprocessed starting at time $T[X_2, W' - w_j]$, and job j can be preprocessed starting at time $s_j - p_j$, the set of jobs $\sigma_2 \cup \{j\}$ can be preprocessed starting at time $\min\{T[X_2, W' - w_j],\ s_j\} - p_j$, with j being the first job to be preprocessed. Thus, as $w(\sigma_2 \cup \{j\}) = W'$, we have $T[X, W'] \geq \min\{T[X_2, W' - w_j],\ s_j\} - p_j$.
  - If j_2 exists, then let $i \in \{1, \ldots, m\}$ be such that $x_i = j_2 \in X_2$. Then job j is non-conflicting with any job on machine i, since only jobs in $\{j_2, \ldots, n\}$ are scheduled on this machine, and $d_j \leq s_{j_2}$. We can therefore schedule job j on machine i, obtaining a schedule for $\sigma_2 \cup \{j\}$ that is compatible with $X = (X_2 \cup \{j\}) \setminus \{j_2\}$. As argued above, the set of jobs $\sigma_2 \cup \{j\}$ can be preprocessed starting at time $\min\{T[X_2, W' - w_j],\ s_j\} - p_j$, and as $w(\sigma_2 \cup \{j\}) = W'$, we again get that $T[X, W'] \geq \min\{T[X_2, W' - w_j],\ s_j\} - p_j$.

Next, we show that $T[X, W']$ is at most the right-hand side of the formula above. This is clearly true if $T[X, W'] = -\infty$, so assume $T[X, W'] > -\infty$ and let $\sigma \subseteq \mathcal{J}$ be a subset of jobs with total weight $w(\sigma) = W'$ which is compatible with $X = \{x_1, \ldots, x_{|X|}\}$, and can be preprocessed starting at time $T[X, W']$; that is, it has maximum preprocessing starting time among all such subsets σ. Consider a schedule for σ that is compatible with X on $k \leq |X|$ machines, and let $z_1, \ldots, z_k \in \sigma$ be such that job z_i is the job with minimum start time scheduled on machine i, for $i \in \{1, \ldots, k\}$. By reordering the machines, we may assume that $z_1 < z_2 < \ldots < z_k$. There are two cases to consider:

- Suppose $j \notin \sigma$. Then $j + 1 \leq z_1$. We consider two possibilities:
  - If j_1 does not exist, then $j + 1, \ldots, n \in X$, and so $X_1 = \{j + 1, \ldots, n\}$. Since $j + 1 \leq z_1$ and since $j + 2 \leq z_2$, and so forth, we can see that σ is also compatible with X_1. Thus, $T[X, W'] \leq T[X_1, W']$ in this case.
  - If j_1 exists, then $j_1 = j + i$ for some $1 \leq i \leq |X|$, and

    $$X_1 = \{j + 1, j + 2, \ldots, j + i = j_1\} \cup \{x_{i+1}, \ldots, x_{|X|}\}.$$

    Since $z_1 \geq j+1$ and $z_1 < \cdots < z_i$, we have $j+\ell \leq z_\ell$ for each $\ell \in \{1, \ldots, i\}$. As σ is compatible with X, we have $x_{i+\ell} \leq z_{i+\ell}$ for each $\ell \in \{1, \ldots, k - i\}$. Thus, σ is also compatible with X_1, and we have $T[X, W'] \leq T[X_1, W']$ in this case as well.
- Suppose $j \in \sigma$. Then $j = z_1$ is the job with the minimum start time scheduled on the first second-stage machine. Let $z \in \sigma \setminus \{z_1, \ldots, z_k\}$ be the job scheduled immediately after job z_1 on the first second-stage machine. There are three possibilities to consider:
  - Suppose job z does not exist, and $z_1 = j$ is scheduled alone on the first machine. Then $\sigma \setminus \{j\}$ is compatible with $X \setminus \{j\} \subseteq X_2$, and so it is also compatible with X_2. As $w(\sigma \setminus \{j\}) = W' - w_j$, the set of jobs $\sigma \setminus \{j\}$ can be preprocessed starting at time $P' \leq T[X_2, W' - w_j]$. Since job j can be preprocessed starting at time $s_j - p_j$, it follows that σ can be preprocessed (starting with j) at time $\min\{P', \ s_j\} - p_j$. Thus, we have $T[X, W'] \leq \min\{T[X_2, W' - w_j], \ s_j\} - p_j$.
  - Suppose job z exists, and index j_2 exists as well. Then $j_2 \leq z$ by definition of j_2, and so $\sigma \setminus \{j\}$ is compatible with $X_2 = (X \setminus \{j\}) \cup \{j_2\}$. As $w(\sigma \setminus \{j\}) = W' - w_j$, the set of jobs $\sigma \setminus \{j\}$ can be preprocessed at time $P' \leq T[X_2, W' - w_j]$. Since $\{j\}$ can be preprocessed starting at time $s_j - p_j$, we have that σ can be preprocessed at time $\min\{P', \ s_j\} - p_j$. Thus, $T[X, W'] \leq \min\{T[X_2, W' - w_j], \ s_j\} - p_j$.
  - Suppose job z exists, but j_2 does not. Let $j_3 = \min\{j^* \in \mathcal{J} \mid d_j \leq s_{j^*}\}$. Since index j_2 does not exist, there is some $i \in \{2, \ldots, |X|\}$ with $x_i = j_3$ and

    $$X = \{x_1, \ldots, x_{i-1}\} \cup \{j_3, j_3 + 1, \ldots, n = x_{|X|}\}.$$

    We argue that one of the machines $i_0 \in \{i, \ldots, |X|\}$ must be empty. Suppose not. Then $k = |X|$, and we have $z_{i+\ell} = j_3 + \ell$ for all $\ell \in \{0, \ldots, |X| - i\}$.

Since $j_3 \leq z \leq n$, we have that $z = j_3 + \ell$ for some $\ell \in \{0, \ldots, |X| - i\}$. But then this means that $z \in \{z_1, \ldots, z_k\}$, a contradiction. Thus, as $j_3 \leq z$, all jobs in $\sigma \setminus \{j\}$ that are scheduled on the first second-stage machine (starting with z) can be scheduled on the empty machine $i_0 \in \{i, \ldots, |X|\}$, obtaining a schedule that is compatible with $X_2 = X \setminus \{j\}$. This again implies that $T[X, W'] \leq \min\{T[X_2, W' - w_j],\ s_j\} - p_j$.

The lemma thus follows. □

To determine the optimal solution for the $FF(1,m)||\sum w_j Z_j$ problem, we need to compute all values $T[X, W']$ for any set $X = \{x_1, \ldots, x_{|X|}\}$ with $x_1 < x_2 < \cdots < x_{|X|}$ and for all $W' \in \{0, \ldots, W\}$ using the recursion in Eq. (4.2). Therefore, we need to make sure that $T[X_1, W']$ and $T[X_2, W' - w_j]$ are computed prior to the computation of $T[X, W']$. To this end, we use the following domination rule between sets. We say that $X' = \{x'_1, \ldots, x'_{|X'|}\}$ is dominated by $X = \{x_1, \ldots, x_{|X|}\}$, denoted by $X' \preceq X$, if $|X| \geq |X'|$ and $x_i \leq x'_i$ for $i \in \{1, \ldots, |X'|\}$. Note that by definition $X_1 \preceq X$ and $X_2 \preceq X$. Therefore, we compute all values $T[X, W']$ according to the domination ordering, so that all values corresponding to a set X are computed only after all values corresponding to sets dominated by X have been computed. In the final stage, all values $T[\{1, \ldots, m\}, W']$ are computed (note that $\{1, \ldots, m\}$ dominates all other sets), and the optimal solution value corresponds to the maximum $W' \in \{0, \ldots, W\}$ such that $T[\{1, \ldots, m\}, W'] \geq 0$. The optimal solution can then be computed in $O(n)$ time using standard backtracking techniques.

Regarding the time complexity of our algorithm, note that given a subset X, we can easily compute the indices j, j_1, and j_2 in $O(m^2) \in O(1)$ time. Consequently, the subsets X_1 and X_2 can also be computed in $O(1)$ time, and so each table entry in T can be computed in $O(1)$ time. Since there are $(2W + 1) \cdot \binom{n}{m} \in O(W \cdot n^m)$ entries in total in T, we can compute the entire table T in $O(W \cdot n^m)$ time. This completes our proof of Theorem 1.5.

Theorem 1.5 *$FF(1,m)||\sum w_j Z_j$ is solvable in $O(W \cdot n^m)$ time.*

Observe that Theorem 1.5 directly implies an improved algorithm to the $O(n^3)$ algorithm for the two-stage unweighted (non-flexible) flow shop $FF(1,1)||\sum Z_j$ problem given in [Sha12]. Since $W = n$ in the unweighted case, we obtain:

Corollary 8 *$FF(1,1)||\sum Z_j$ is solvable in $O(n^2)$ time.*

The preceding dynamic program is pseudo-polynomial in the total weight W. A standard technique in dynamic programming is to create a *dual formulation* by swapping the roles of the objective value and a key resource parameter. In our case, the total processing time of the first-stage operations P provides another natural parameter for the problem. By swapping the target weight W' with the accumulated first-stage processing time, we can develop a symmetric algorithm.

This approach results in a new dynamic programming table, which we also denote by T, that stores the *maximum achievable weight* for a given compatible set X and a given

first-stage starting time P'. The table is indexed by a subset of at most m jobs and an integer $P' \in \{0, \ldots, P\}$. The algorithm will maintain the following invariant throughout:

> $T[X, P'] =$ the maximum W' such that there exists a feasible solution $\sigma \subseteq \mathcal{J}$ with $w(\sigma) = W'$ that is compatible with X and can be preprocessed starting at time P'.

For the base cases of the dynamic program, we set $T[\emptyset, P'] = 0$ for all P'. The remaining entries are computed by

$$T[X, P'] = \max \begin{cases} T[X_1, P'] \\ \begin{cases} -\infty & : P' + p_j > s_j \\ T[X_2, P' + p_j] + w_j & : P' + p_j \leq s_j \end{cases} \end{cases} \tag{4.3}$$

where j is the minimum job index in X. Correctness of the above recursion can be verified similarly to how it is done in the proof of Lemma 7. The computation of all $T[X, P']$ values for any set $X = \{x_1, \ldots, x_{|X|}\}$ with $x_1 < x_2 < \cdots < x_{|X|}$ and for all $P' \in \{0, \ldots, P\}$ is performed using the recursion in Eq. (4.3). The order of computation follows the domination rule between sets, ensuring that all values corresponding to a set X are computed only after all values corresponding to sets dominated by X have been calculated. After computing all $O(P \cdot n^m)$ possible $T[X, P']$ values, the value of the optimal solution is given by $T[\{1, \ldots, m\}, 0]$. The optimal solution can then be determined in $O(n)$ time using standard backtracking techniques.

Corollary 9 *$FF(1, m)||\sum w_j Z_j$ is solvable in $O(P \cdot n^m)$ time.*

4.2.2 A Dynamic Program for Small Chromatic Number

In this section, we consider the case where the chromatic number of the second-stage conflict graph χ is relatively small. For this case, we present a dynamic programming algorithm, which can solve the $FF(1, m)||\sum w_j Z_j$ problem in $O(W \cdot 2^{\chi} \cdot n)$ time, which is pseudo-polynomial when the value of χ is $O(\log n)$. To this end, recall that we assume the input set of jobs $\mathcal{J} = \{1, \ldots, n\}$ is ordered by nondecreasing EST, that is, $s_1 \leq \cdots \leq s_n$. Moreover, we may assume without loss of generality that all endpoints of the second-operation intervals are distinct. If this is not the case, we can scale up the due-dates and processing times by setting $d'_j = (n+1) \cdot d_j + j$, $q'_j = (n+1) \cdot q_j$ and $p'_j = (n+1) \cdot p_j$ for each job j. In this way, we obtain an equivalent instance in which no pair of jobs has the same starting time or due date.

Let $\mathcal{T}$ be the union of all job due dates and second operation start times, that is $\mathcal{T} = \{s_j \mid j \in \mathcal{J}\} \cup \{d_j \mid j \in \mathcal{J}\}$, and assume that the elements in $\mathcal{T}$ are ordered in ascending order. Given $t_i \in \mathcal{T}$, let X_i denote the job set $\{j \in \mathcal{J} \mid s_j \leq t_i < d_j\}$, the set of all jobs whose second operation interval contains t_i (recall that the time interval of the second operation of j is $[s_j, d_j)$, and therefore it contains s_j but *does not* contain d_j). Observe that

for $t_i = s_j$, we have $X_i = X_{i-1} \cup \{j\}$, and for $t_i = d_j$, we have $X_i = X_{i-1} \setminus \{j\}$. Moreover, note that $\chi = \max_i |X_i|$. These two properties will be used extensively throughout our dynamic program algorithm.

For $i \in \{0, 1, \ldots, 2n\}$, let $\mathcal{X}_i$ denote the job set $\bigcup_{\ell \leq i} X_\ell$, with $\mathcal{X}_0 = \emptyset$. Our algorithm computes schedules for $\mathcal{X}_i$ in increasing values of i. Here, we say that a feasible schedule $\sigma \subseteq \mathcal{X}_i$ is *compatible* with a subset $X \subseteq X_i$ if $X_i \cap \sigma = X$, *i.e.* each job of X is in σ, and each job in $X_i \setminus X$ is not in σ. We define a dynamic program that iterates over $\mathcal{T}$ in an ascending order and computes $|\mathcal{T}| + 1 = 2n + 1$ tables $T_0, \ldots, T_{2n}$. Each table T_i is indexed by a subset $X \subseteq X_i$ with $|X| \leq m$ and an integer $W' \in \{0, \ldots, W\}$. Throughout the algorithm, the following invariant is maintained:

> $T_i[X, W'] =$ the minimum P' such that there exists a feasible solution $\sigma \subseteq \mathcal{X}_i$ with $w(\sigma) = W'$ which is compatible with $X \subseteq X_i$, and $p(\sigma) = P'$.

The base case of our recursion is T_0 where $\mathcal{X}_0 = \emptyset$, hence we set $T_0[\emptyset, 0] = 0$ and $T_0[\emptyset, W'] = \infty$ for $W' \neq 0$. So assume $i \in \{1, \ldots, 2n\}$, and let $X \subseteq X_i$ and let $W' \in \{0, \ldots, W\}$. Again, the recursive step of our dynamic program considers two mutually exclusive cases for each job $j \in \mathcal{J}$, either it is included in a feasible solution σ with minimum $p(\sigma)$ that is compatible with X or it is not included. To compute $T_i[X, W']$, we consider two cases:

- If $t_i = s_j$ for some job $j \in \mathcal{J}$, then $T_i[X, W'] =$

$$\begin{cases} T_{i-1}[X, W'] & : j \notin X \\ T_{i-1}[X \setminus \{j\}, W' - w_j] + p_j & : j \in X \text{ and } T_{i-1}[X \setminus \{j\}, W' - w_j] + p_j \leq s_j \\ \infty & : \text{otherwise.} \end{cases} \tag{4.4}$$

- If $t_i = d_j$ for some job $j \in \mathcal{J}$, then $T_i[X, W'] =$

$$\begin{cases} T_{i-1}[X, W'] & : |X| = m \\ \min \begin{cases} T_{i-1}[X \cup \{j\}, W'] \\ T_{i-1}[X, W'] \end{cases} & : |X| < m \end{cases} \tag{4.5}$$

Intuitively speaking, the first case above corresponds to the situation where job j is a new job to be considered in our job set $\mathcal{X}_i$. We then consider whether $j \in X$ or not, which determines whether or not j is included in the computed feasible solution σ. If j is to be included in σ, we must check whether $\sigma \cup \{j\}$ can be preprocessed on time, otherwise we obtain an infeasible solution. In the second case, job j will still be in future job sets $\mathcal{X}_{i+1}, \mathcal{X}_{i+2}, \ldots, \mathcal{X}_{2n}$, but will not be in any subset $X_{i+1}, X_{i+2} \ldots X_{2n}$. In this case, we do not make any new decision about whether or not to include job j; instead, we observe that any solution compatible with $X \cup \{j\}$ for $\mathcal{X}_{i-1}$ remains compatible with X for $\mathcal{X}_i$. The lemma below formalizes this intuition.

Lemma 10 *For any $i \in \{0, \ldots, 2n\}$, the above recursion correctly computes $T_i[X, W']$ for all subsets $\mathcal{X}_i \subseteq \mathcal{J}$, $X \subseteq X_i$, and integers $W' \in \{0, \ldots, W\}$.*

Proof. The correctness of the recursion can be verified by induction on i. The invariant clearly holds for the base case $i = 0$. Thus, assume $i > 0$, and let $X \subseteq X_i$ and $W' \in \{0, \ldots, W\}$. We prove that $T_i[X, W']$ equals the minimum preprocessing time for any feasible solution $\sigma \subseteq \mathcal{X}_i$ that is compatible with X and has $w(\sigma) = W'$. Our proof is split into two cases, depending on whether $t_i = s_j$ or $t_i = d_j$ for some job $j \in \mathcal{J}$.

Suppose that $t_i = s_j$ for some $j \in \mathcal{J}$. Note that in this case, we have $X_i = X_{i-1} \cup \{j\}$ and $\mathcal{X}_i = \mathcal{X}_{i-1} \cup \{j\}$. We argue that our recursive formula is correct according to each of the three cases considered in the formula:

- $\underline{j \notin X}$. Then $X \subseteq X_{i-1}$, and so for any feasible solution $\sigma \subseteq \mathcal{X}_i$ that is compatible with X we have $\sigma \subseteq \mathcal{X}_{i-1}$, and vice versa. Thus, by induction we get that $T_i[X, W'] = T_{i-1}[X, W']$.

- $\underline{j \in X \text{ and } T_{i-1}[X \setminus \{j\}, W' - w_j] + p_j \leq s_j}$. Then $T_{i-1}[X \setminus \{j\}, W' - w_j] < \infty$, and so there exists a feasible solution for $\mathcal{X}_{i-1}$ which is compatible with $X \setminus \{j\}$ with total weight $W' - w_j$. Let $\sigma_{i-1} \subseteq \mathcal{X}_{i-1}$ be such a solution with total preprocessing time $p(\sigma_{i-1}) = T_{i-1}[X \setminus \{j\}, W' - w_j]$. Then, as the only jobs in $\mathcal{X}_i$ which are conflicting with j are the jobs in X, and as $\sigma_{i-1} \cap X = X \setminus \{j\}$ and $|X \setminus \{j\}| < m$, the set of jobs $\sigma_{i-1} \cup \{j\}$ can be scheduled on m machines. Since $p(\sigma_{i-1}) + p_j \leq s_j$, and j is preprocessed last in $\sigma_{i-1} \cup \{j\}$, the set $\sigma_{i-1} \cup \{j\}$ can be preprocessed in time. Thus, $\sigma_{i-1} \cup \{j\} \subseteq \mathcal{X}_i$ is a feasible solution which is compatible with X, and so $T_i[X, W'] \leq T_{i-1}[X \setminus \{j\}, W' - w_j] + p_j$. Conversely, for any feasible solution $\sigma_i \subseteq \mathcal{X}_i$ that includes j, is compatible with X, and has total weight W', it holds that the set of jobs $\sigma_i \setminus \{j\}$ is a feasible solution for $\mathcal{X}_{i-1}$ which is compatible with $X \setminus \{j\}$ and has total weight $W' - w_j$. Thus, $T_i[X, W'] \geq T_{i-1}[X \setminus \{j\}, W' - w_j] + p_j$, and so $T_i[X, W'] = T_{i-1}[X \setminus \{j\}, W' - w_j] + p_j$.

- $\underline{j \in X \text{ and } T_{i-1}[X \setminus \{j\}, W' - w_j] + p_j > s_j}$. We argue that in this case, there is no W' weighted feasible solution that is compatible with X. Suppose there exists such a solution σ. Then $\sigma \setminus \{j\} \subseteq \mathcal{X}_{i-1}$ is a feasible solution with weight $W' - w_j$ which is compatible with $X \setminus \{j\}$. Hence, $T_{i-1}[X \setminus \{j\}, W' - w_j] \leq p(\sigma \setminus \{j\})$. Yet, since σ is preprocessed in time, and s_j is the largest second-stage start time among the jobs in $\{1, \ldots, j\}$, it must be that $p(\sigma \setminus \{j\}) + p_j \leq s_j$, contradicting the fact that $T_{i-1}[X \setminus \{j\}, W' - w_j] + p_j > s_j$. Thus, there is no W' weighted feasible solution that is compatible with X, and indeed $T_i[X, W'] = \infty$.

Suppose that $t_i = d_j$ for some job $j \in \mathcal{J}$. Note that in this case, we have $j \in X_{i-1}$ and $X_i = X_{i-1} \setminus \{j\}$. Thus, $\mathcal{X}_i = \mathcal{X}_{i-1}$ and $j \notin X$. It follows that any feasible solution for $\mathcal{X}_{i-1}$ which is compatible with X is also a feasible solution for $\mathcal{X}_i$ that is compatible with X, and vice-versa. This implies that $T_i[X, W'] = T_{i-1}[X, W']$ whenever $|X| = m$. If,

however, we have $|X| < m$, then a feasible solution for $\mathcal{X}_{i-1}$ which is compatible with $X \cup \{j\}$ is also a feasible solution for $\mathcal{X}_i$ which is compatible with X, and vice-versa. Thus, $T_i[X, W']$ equals the minimum of $T_{i-1}[X \cup \{j\}, W']$ and $T_{i-1}[X, W']$ in this case. The lemma thus follows. □

To solve the problem, we compute all values $T_i[X, W']$ in table T_i using the previously computed table T_{i-1} for $i \in \{1, \ldots, 2n\}$. The computation is performed using the recursive relation in Eq. (4.4) if $t_i = s_j$ for some job $j \in \mathcal{J}$ and using the recursive relation in Eq. (4.5) if $t_i = d_j$ for some job $j \in \mathcal{J}$. As mentioned before, the base case of the recursion is table T_0 with $T_0[\emptyset, 0] = 0$ and $T_0[\emptyset, W'] = \infty$ for $W' \neq 0$. Given the correctness of Lemma 10, the optimal solution value can then be obtained by finding the maximum W' such that there exists an $X \subseteq X_{2n}$ where $T_{2n}[X, W'] \neq \infty$. The optimal solution is then computed in $O(n)$ time using simple backtracking techniques.

Since there are $2n$ points in $\mathcal{T}$, the above dynamic program requires the computation of $2n$ tables. Moreover, since for each $t_i \in \mathcal{T}$ we have $X \subseteq X_i$ and $|X_i| \leq \chi$, there are $W \cdot 2^\chi$ entries in each table. As each entry can be computed in $O(1)$ time, the running time of the entire algorithm is $O(W \cdot 2^\chi \cdot n)$, and the first bullet in Theorem 1.6 follows.

4.2.3 A Dynamic Program for Constant Processing Times

In this section, we present an $O(W \cdot m^{q_{\max}} \cdot n)$ time dynamic programming algorithm for $FF(1, m)||\sum w_j Z_j$, where $q_{\max} = \max_j q_j$ is the maximum processing time of any of the second-stage operations, proving the second bullet in Theorem 1.6. Recall that we assume the input set $\mathcal{J}$ of jobs is ordered according to the EST order, giving us $s_1 \leq \cdots \leq s_n$. We compute $n + 1$ dynamic programming tables $T_0, \ldots, T_n$, where table T_j corresponds to the sub-instance defined by the set of jobs $\mathcal{J}_j = \{1, \ldots, j\}$ (and T_0 corresponds to $\mathcal{J}_0 = \emptyset$). Each table T_j will be indexed by a vector $\vec{x} = (x_1, \ldots, x_{q_{\max}}) \in \{0, \ldots, m\}^{q_{\max}}$ with $\sum_{i=1}^{q_{\max}} x_i \leq m$, and an integer $W' \in \{-W, \ldots, W\}$.

Let $j \in \mathcal{J}$ and $\vec{x} = (x_1, \ldots, x_{q_{\max}}) \in \{0, \ldots, m\}^{q_{\max}}$. We say that a feasible solution $\sigma \subseteq \mathcal{J}_j = \{1, \ldots, j\}$ is *compatible* with $\vec{x}$ *at* s_j if σ admits a schedule where there are exactly x_i second-stage machines whose last job completes at time $s_j + i$, for every $i \in \{1, \ldots, q_{\max}\}$. Throughout the algorithm, we will maintain the following invariant:

> $T_j[\vec{x}, W'] =$ the minimum P' such that there exists a feasible solution $\sigma \subseteq \mathcal{J}_j$ with $w(\sigma) = W'$ and $p(\sigma) = P'$ that is compatible with $\vec{x}$ at s_j.

In this way, a vector $\vec{x}$ as above gives us sufficient information about a solution for the job set $\mathcal{J}_j = \{1, \ldots, j\}$ relative to time s_j, as there can be no machine whose last job completes at time greater than $s_j + q_{\max}$, given that s_j is the maximum starting time in $\mathcal{J}_j$ and $q_{\max}$ is the maximum processing time of any second-stage operation.

For the base case, set $T_0[\vec{0}, 0] = 0$ and $T_0[\vec{x}, W'] = \infty$ for every $(\vec{x}, W') \neq (\vec{0}, 0)$. For $j > 0$, set $T_j[\vec{0}, 0] = 0$. Set $T_j[\vec{x}, W'] = \infty$ whenever $W' < 0$, or $W' = 0$ and $\vec{x} \neq \vec{0}$, or

$\sum_{i=1}^{q_{\max}} x_i > m$. Before presenting a recursive formula for the remaining entries, we need to establish several definitions. For a job $j \in \mathcal{J}$, let $\delta_j = s_j - s_{j-1}$ (where $s_0 = 0$). Given a vector $\vec{x} = (x_1, \ldots, x_{q_{\max}}) \in \{0, \ldots, m\}^{q_{\max}}$ and integer δ, we use $\vec{x}[\delta]$ to denote the set of vectors

$$\vec{x}[\delta] = \left\{ \vec{y} = (y_1, \ldots, y_{q_{\max}}) \in \{0, \ldots, m\}^{q_{\max}} \;\middle|\; y_i = x_{i-\delta} \text{ for all } i \in \{\delta + 1, \ldots, q_{\max}\} \right\}$$

if $x_i = 0$ for every $i \in \{1, \ldots, q_{\max}\}$ with $i > q_{\max} - \delta$, and $\vec{x}[\delta] = \emptyset$ otherwise.

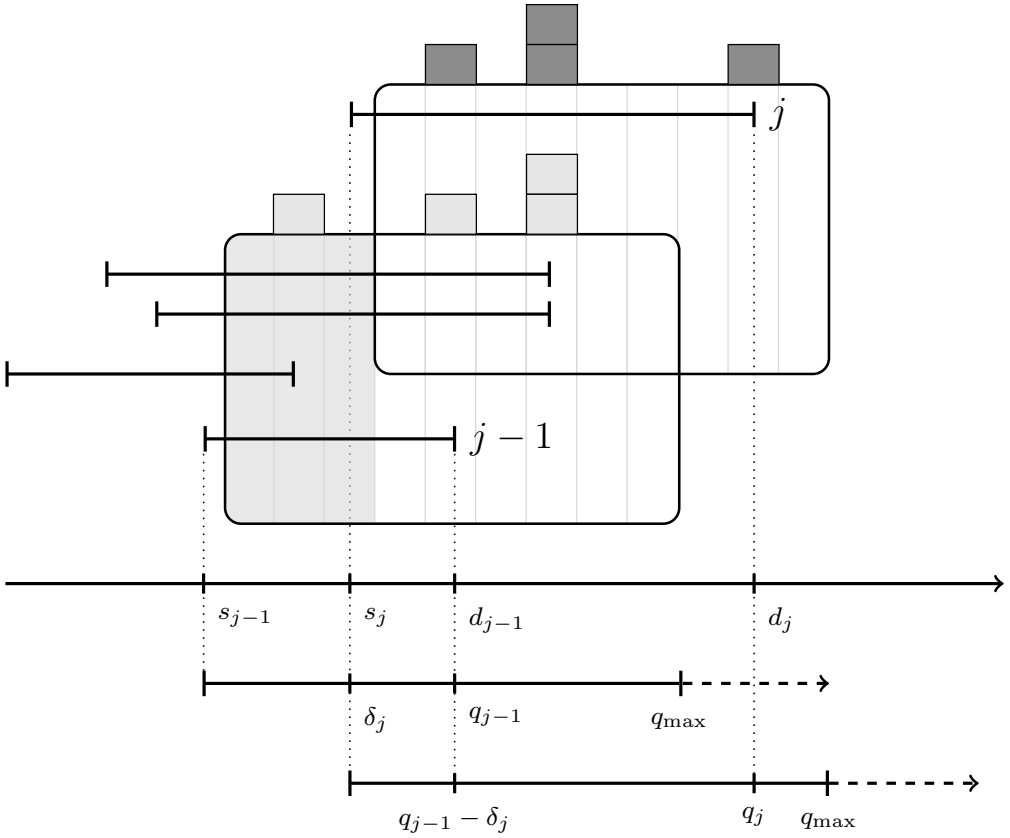


Figure 4.2: The dynamic program visualized. A set σ of jobs is shown in three reference frames: the top axis is the main line beginning at 0, the middle axis the reference of $j-1$ beginning at s_{j-1} and the bottom one is the reference of j beginning at s_j. The boxes above each sliding window represent the number of machines that complete processing their last job at that time. In the example, assuming that all jobs can be preprocessed in time, σ is compatible with $\vec{x}$ at s_j; the solution $\sigma \setminus \{j\}$ is compatible with $\vec{y}$ at s_{j-1}; with $q_{\max} = 9$, $\delta_j = 3$, $\vec{x} = (0, 1, 0, 2, 0, 0, 0, 1, 0)$ and $\vec{y} = (0, 1, 0, 0, 1, 0, 2, 0, 0)$. Observe that $\vec{y} \in \vec{x}^{(q_j)}[\delta_j]$, which in our example is $\vec{x}^{(8)}[3]$.

Thus, $\vec{x}[\delta]$ is the set of vectors obtained by shifting the coordinates of $\vec{x}$ to the right by δ, where the coordinates that are shifted out of range must be zero (no second-stage machine of a solution for $\mathcal{J}_{j-1}$ completes its last job after $s_{j-1} + q_{\max}$), and the newly exposed coordinates are irrelevant. In particular, if $\delta \geq q_{\max}$ we have $\vec{x}[\delta] = \{0, \ldots, m\}^{q_{\max}}$ when $\vec{x} = \vec{0}$, and $\vec{x}[\delta] = \emptyset$ otherwise. Finally, for a vector $\vec{x} \in \{0, \ldots, m\}^{q_{\max}}$ and an

integer $q \in \{1, \ldots, q_{\max}\}$, let $\vec{x}^{(q)}$ denote the vector (if it exists) in $\{0, \ldots, m\}^{q_{\max}}$ with value $x_i^{(q)} = x_i$ in any coordinate $i \neq q$, and value $x_q^{(q)} = x_q - 1$ in coordinate q. The recursive formula for the remaining entries of our dynamic program is then given by:

$$T_j[\vec{x}, W'] = \min \begin{cases} \min_{\vec{y} \in \vec{x}[\delta_j]} \{T_{j-1}[\vec{y}, W']\} \\ \min_{\vec{y} \in \vec{x}^{(q_j)}[\delta_j]} \begin{cases} T_{j-1}[\vec{y}, W' - w_j] + p_j & : T_{j-1}[\vec{y}, W' - w_j] + p_j \leq s_j \\ \infty & : \text{otherwise.} \end{cases} \end{cases} \tag{4.6}$$

See Figure 4.2 for an illustration of the dynamic program. As with our previous recursions, the above formula is split into two cases depending on whether or not job j is added to a feasible solution $\sigma \subseteq \mathcal{J}_{j-1}$. In the first case, job j is not included, the solution σ corresponds to one of the vectors in $\vec{x}[\delta_j]$, since δ_j is the difference between s_j and s_{j-1}. In the second case, j is added to σ, so the solution $\sigma \cup \{j\}$ has one extra job associated with it in the q_jth coordinate of $\vec{x}$. Thus, the solution σ corresponds to one of the vectors in $\vec{x}^{(q_j)}[\delta_j]$. Correctness of our recursive formula is proven in the lemma below.

Lemma 11 *The above recursion correctly computes $T_j[\vec{x}, W']$ for all vectors $\vec{x} \in \{0, \ldots, m\}^{q_{\max}}$ and all integers $W' \in \{0, \ldots, W\}$.*

Proof. The correctness of the recursion can be verified by induction. The invariant clearly holds for the base cases, and so suppose that $j > 0$ and that the invariant holds for all entries of T_{j-1}. We argue that the invariant also holds for all entries of T_j. Let $W' \in \{1, \ldots, W\}$, and let $\vec{x}$ be an arbitrary vector in $\{0, \ldots, m\}^{q_{\max}}$, and consider entry $T_j[\vec{x}, W']$.

We first argue that $T_j[\vec{x}, W']$ is at most the right-hand side of the above formula. There are two cases to consider here according to the formula.

- Let $\vec{y} \in \vec{x}[\delta_j]$ be a vector minimizing $T_{j-1}[\vec{y}, W']$, and suppose that $T_{j-1}[\vec{y}, W'] < \infty$ (otherwise, clearly $T_j[\vec{x}, W'] \leq T_{j-1}[\vec{y}, W']$). Then any feasible solution $\sigma \subseteq \mathcal{J}_{j-1}$ with $w(\sigma) = W'$ and $p(\sigma) = T_{j-1}[\vec{y}, W']$ that is compatible with $\vec{y}$ at s_{j-1}, is also compatible with $\vec{x}$ at s_j, since $y_i = x_{i-\delta_j}$ for all $i \in \{\delta_j + 1, \ldots, q_{\max}\}$ and $x_i = 0$ for all $i > q_{\max} - \delta_j$ (by definition of $\vec{x}[\delta_j]$), while no second-stage machine of this schedule completes its last job after $s_{j-1} + q_{\max}$. Thus, $T_j[\vec{x}, W'] \leq T_{j-1}[\vec{y}, W']$.

- Let $\vec{y} \in \vec{x}^{(q_j)}[\delta_j]$ be a vector minimizing $T_{j-1}[\vec{y}, W' - w_j]$. Then $x_{q_j} > 0$, otherwise $\vec{x}^{(q_j)}$ does not exist and $\vec{x}^{(q_j)}[\delta_j] = \emptyset$. Suppose that $T_{j-1}[\vec{y}, W' - w_j] + p_j \leq s_j$ (otherwise, clearly $T_j[\vec{x}, W']$ is at most the second item of the right-hand side of the formula). Then $T_{j-1}[\vec{y}, W' - w_j] < \infty$, and so there exists a feasible solution $\sigma \subseteq \mathcal{J}_{j-1}$ with $w(\sigma) = W' - w_j$ and $p(\sigma) = T_{j-1}[\vec{y}, W' - w_j] \leq s_j - p_j$ that is compatible with $\vec{y}$ at s_{j-1}. Note that $\sigma \cup \{j\}$ can be preprocessed in time, since σ can be preprocessed in time, and since $p(\sigma) + p_j \leq s_j$. Moreover, $\sigma \cup \{j\}$ is compatible with $\vec{x}$ at s_j since $y_i = x_{i-\delta_j}$ for all $i \in \{\delta_j + 1, \ldots, q_{\max}\} \setminus \{q_j + \delta_j\}$, $y_{q_j+\delta_j} = x_{q_j} - 1$ if $y_{q_j+\delta_j}$ exists, and $x_i^{(q_j)} = 0$ for all $i > q_{\max} - \delta_j$ (by definition of $\vec{x}^{(q_j)}[\delta_j]$), while job j completes at time $s_j + q_j$ and no job of σ completes after $s_{j-1} + q_{\max}$. Thus, $\sigma \cup \{j\} \subseteq \mathcal{J}_j$ is a W' weighted feasible

solution with $p(\sigma \cup \{j\}) = T_{j-1}[\vec{y}, W' - w_j] + p_j$ that is compatible with $\vec{x}$ at s_j, and so $T_j[\vec{x}, W'] \leq T_{j-1}[\vec{y}, W' - w_j] + p_j$.

Next, we argue that $T_j[\vec{x}, W']$ is at least the right-hand side of the above formula. We can assume that $T_j[\vec{x}, W'] < \infty$, as otherwise the claim is trivial, and so let $\sigma \subseteq \mathcal{J}_j$ be a feasible solution with $w(\sigma) = W'$ and $p(\sigma) = T_j[\vec{x}, W']$ that is compatible with $\vec{x}$ at s_j. There are two cases to consider here, depending on whether $j \in \sigma$ or not.

- Suppose $j \notin \sigma$. Then $\sigma \subseteq \mathcal{J}_{j-1}$. Consider the schedule for σ corresponding to $\vec{x}$. Then this schedule has exactly $x_{i-\delta_j}$ second-stage machines whose last job completes at time $s_j + i - \delta_j$ for each $i \in \{\delta_j + 1, \ldots, q_{\max}\}$, meaning it has exactly $y_i = x_{i-\delta_j}$ second-stage machines whose last job completes at time $s_{j-1} + i$ for each $i \in \{1, \ldots, q_{\max} - \delta_j\}$. Moreover, as no job of $\sigma \subseteq \mathcal{J}_{j-1}$ completes after $s_{j-1} + q_{\max}$, we have $x_i = 0$ for all $i > q_{\max} - \delta_j$. Thus, the schedule σ is also compatible with some $\vec{y} \in \vec{x}[\delta_j]$ at s_{j-1}, and so it follows that $T_j[\vec{x}, W'] \geq \min_{\vec{y} \in \vec{x}[\delta_j]} T_{j-1}[\vec{y}, W']$.

- Suppose $j \in \sigma$, and let $\sigma' = \sigma \setminus \{j\}$. Then $\sigma' \subseteq \mathcal{J}_{j-1}$. Consider the schedule for σ corresponding to $\vec{x}$ without job j. Then this schedule has exactly $x_{i-\delta_j}$ second-stage machines whose last job completes at time $s_j + i - \delta_j$ for each $i \in \{\delta_j + 1, \ldots, q_{\max}\} \setminus \{q_j + \delta_j\}$, and exactly $x_{q_j} - 1 \geq 0$ second-stage machines whose last job completes at time $s_j + q_j$. This means it has exactly $y_i = x^{(q_j)}_{i-\delta_j}$ second-stage machines whose last job completes at time $s_{j-1} + i$ for each $i \in \{\delta_j + 1, \ldots, q_{\max}\}$. Moreover, as no job of $\sigma' \subseteq \mathcal{J}_{j-1}$ completes after $s_{j-1} + q_{\max}$, we have $x^{(q_j)}_i = 0$ for all $i > q_{\max} - \delta_j$. Thus, σ' is compatible with some $\vec{y} \in \vec{x}^{(q_j)}[\delta_j]$ at s_{j-1}. Since $w(\sigma') = W' - w_j$ and $p(\sigma') = p(\sigma) - p_j$, it follows that $T_{j-1}[\vec{y}, W' - w_j] \leq p(\sigma) - p_j \leq s_j - p_j$, where the last inequality holds because σ can be preprocessed in time and job j is preprocessed last. Hence, $T_j[\vec{x}, W'] = p(\sigma) \geq T_{j-1}[\vec{y}, W' - w_j] + p_j$, which is at least the second item of the right-hand side of the formula.

The lemma thus follows. □

To solve the problem, we compute all values $T_j[\vec{x}, W']$ for any $W' \in \{0, \ldots, W\}$ and $\vec{x} = (x_1, \ldots, x_{q_{\max}}) \in \{0, \ldots, m\}^{q_{\max}}$ using the recursion in Eq. (4.6), processing indices in increasing order from $j = 1$ to $j = n$. That is, we complete the computation of all values in table T_{j-1} before proceeding to compute table T_j. Initialize the tables as specified above: for the base case, set $T_0[\vec{0}, 0] = 0$ and $T_0[\vec{x}, W'] = \infty$ for every $(\vec{x}, W') \neq (\vec{0}, 0)$. For $j > 0$, set $T_j[\vec{0}, 0] = 0$. Set $T_j[\vec{x}, W'] = \infty$ whenever $W' < 0$, or $W' = 0$ and $\vec{x} \neq \vec{0}$, or $\sum_{i=1}^{q_{\max}} x_i > m$. Given the correctness of Lemma 11, an optimal solution can be obtained by finding the maximum value W' for which there exists a vector $\vec{x} = (x_1, \ldots, x_{q_{\max}}) \in \{0, \ldots, m\}^{q_{\max}}$ such that $T_n[\vec{x}, W'] \neq \infty$. The corresponding optimal solution can then be recovered in linear time using a simple backtracking technique.

Regarding the time complexity of our algorithm, consider table T_j for some $j \in \mathcal{J}$, and fix some $W' \in \{-W, \ldots, W\}$. Note that to compute entry $T_j[\vec{x}, W']$, for some vector $\vec{x} \in \{0, \ldots, m\}^{q_{\max}}$, we find the minimum over a set of size $|\vec{x}[\delta_j]| + |\vec{x}^{(q_j)}[\delta_j]| \in O(m^{\delta'_j})$,

where $\delta'_j = \min\{\delta_j, q_{\max}\}$, which takes $O(m^{\delta'_j})$ time. However, this minimum is identical for every vector $\vec{x'}$ whose last $q_{\max} - \delta'_j$ entries are the same as $\vec{x}$. Consequently, we only need to perform this computation for $O(m^{q_{\max}-\delta'_j})$ distinct vectors $\vec{x}$, yielding a total of $O(m^{q_{\max}})$ running time for computing all entries in T_j corresponding to W'. Since there are $O(W)$ such values W', and n tables T_j, the overall time complexity is $O(W \cdot m^{q_{\max}} \cdot n)$, as stated in the second item of theorem 1.6.

4.2.4 Algorithms for Uniform Preprocessing Times

In the following section, we consider the special case of $FF(1,m)||\sum w_j Z_j$ where all jobs require the same preprocessing time p, *i.e.*, the $FF(1,m)|p_j = p|\sum w_j Z_j$ problem. Note that when all preprocessing times are uniform, the running time of the (second) dynamic program presented in Section 4.2.1 becomes $O(n^{m+1})$, as there are only $O(n)$ possible values for P' in the dynamic program table. Thus, as another corollary of Theorem 1.5, we get the following result for $FF(1,m)|p_j = p|\sum w_j Z_j$:

Corollary 12 *$FF(1,m)|p_j = p|\sum w_j Z_j$ is solvable in $O(n^{m+1})$ time.*

Below, we show that the unweighted version of $FF(1,m)|p_j = p|\sum w_j Z_j$ is polynomial-time solvable, even for unbounded m, as well as the weighted version restricted to *proper* instances (instances where there are no two jobs i and j with $[s_i, d_i) \subseteq [s_j, d_j)$). In particular, we provide a complete proof of Theorem 1.7.

Unweighted Instances

We begin by presenting a polynomial time algorithm for $FF(1,m)|p_j = p|\sum Z_j$. As always, we assume that the jobs in $\mathcal{J}$ are sorted according to the EST rule ($s_1 \leq \ldots \leq s_n$). Our algorithm iteratively constructs a feasible solution σ_j, for $j \in \{0,\ldots,n\}$, with $\sigma_j \subseteq \mathcal{J}_j = \{1,\ldots,j\}$ for each j. Initially, it starts with $\sigma_0 = \emptyset$, and at iteration $j \in \mathcal{J}$ it decides whether or not to add job j to the previously computed feasible solution σ_{j-1}. At the end, the algorithm outputs σ_n as a solution to the entire $FF(1,m)|p_j = p|\sum Z_j$ instance.

Let $j \in \mathcal{J}$. To compute σ_j, we first check if one of the following two cases occurs:

- $s_j < (|\sigma_{j-1}| + 1) \cdot p$, or
- there are $j_1,\ldots,j_m \in \sigma_{j-1}$ with $s_{j_i} \leq s_j < d_{j_i}$ for each $i \in \{1,\ldots,m\}$.

Note that if neither case occurs, the set $\sigma_{j-1} \cup \{j\}$ is a feasible solution, since the jobs are ordered according to the EST rule. Thus, if neither case occurs, we set $\sigma_j = \sigma_{j-1} \cup \{j\}$. Otherwise, we remove a single job from $\sigma_{j-1} \cup \{j\}$ to make it a feasible solution. The removed job is the one with the largest due date in $\sigma_{j-1} \cup \{j\}$. The resulting subset σ_j is feasible since σ_{j-1} is feasible, and since $s_{j_i} \leq s_j < d_{j_i}$ for the removed job $j_i \in \sigma_{j-1} \cup \{j\}$.

To prove the correctness of our algorithm, we introduce the notion of domination between feasible solutions. Let $\sigma = \{j_1,\ldots,j_k\}$ be a feasible solution for $\mathcal{J}_j = \{1,\ldots,j\}$,

indexed so that $d_{j_1} \leq \cdots \leq d_{j_k}$. Then σ *dominates* all feasible solutions for $\mathcal{J}_j$ of smaller cardinality, and all feasible solutions $\sigma' = \{j'_1, \ldots, j'_k\}$ for $\mathcal{J}_j$, indexed so that $d_{j'_1} \leq \cdots \leq d_{j'_k}$, with $d_{j_i} \leq d_{j'_i}$ for all $i \in \{1, \ldots, k\}$. In other words, σ' dominates σ'' if $|\sigma'| > |\sigma''|$, or $|\sigma'| = |\sigma''|$ and the ith largest due date in σ' is not greater than the ith largest due date in σ'', for each $i \in \{1, \ldots, |\sigma'|\}$.

Lemma 13 *For each $j \in \{0, \ldots, n\}$, the set of jobs σ_j dominates all feasible solutions of the instance $\mathcal{J}_j = \{1, \ldots, j\}$ (where $\mathcal{J}_0 = \emptyset$).*

Proof. The proof is by induction on j. The lemma is trivial for $j = 0$, so assume $j \geq 1$. As argued above, the set of jobs σ_j is a feasible solution for $\mathcal{J}_j$, so we only need to argue that it dominates all feasible solutions for $\mathcal{J}_j$. There are three cases to distinguish here:

1. $\sigma_j = \sigma_{j-1} \cup \{j\}$: In this case, the lemma follows from the induction hypothesis, since job j must be in any feasible solution of maximum cardinality for $\mathcal{J}_j$, and σ_{j-1} dominates all feasible solutions for $\mathcal{J}_{j-1}$.

2. $s_j < (|\sigma_{j-1}| + 1) \cdot p$: In this case, the largest cardinality solution of $\mathcal{J}_j$ must be of size $|\sigma_{j-1}|$, and thus σ_j is of maximum cardinality. Since σ_{j-1} dominates all feasible solutions of $\mathcal{J}_j \setminus \{j\}$, and the job removed from $\sigma_{j-1} \cup \{j\}$ is the job with the largest due date, the solution σ_j dominates all feasible solutions of $\mathcal{J}_j$.

3. There are $j_1, \ldots, j_m \in \sigma_{j-1}$ with $s_{j_i} \leq s_j < d_{j_i}$ for each $i \in \{1, \ldots, m\}$: Since σ_{j-1} dominates all feasible solutions for $\mathcal{J}_{j-1}$, in any feasible solution σ' for $\mathcal{J}_{j-1}$ with $|\sigma'| = |\sigma_{j-1}|$ there are $j_1, \ldots, j_m \in \sigma'$ as above. It follows that the largest cardinality solution for $\mathcal{J}_j$ must be of size $|\sigma_{j-1}|$, and thus σ_j is of maximum cardinality. Since the job removed from $\sigma_{j-1} \cup \{j\}$ is the job with the largest due date, the domination property is maintained for σ_j.

In all cases, the set σ_j dominates all feasible solutions of $\mathcal{J}_j$, and the lemma follows. □

Let us now discuss the time complexity of our algorithm. Naively, we can implement each step $j \in \mathcal{J}$ in $O(n)$ time, yielding an overall time complexity of $O(n^2)$. However, this can be improved to $O(n \log n)$ as follows. We maintain a counter a of the number of jobs in the current solution, and a balanced binary search tree H, ordered by due date (ties broken by job index), that contains the jobs of the current solution whose due date has not yet been reached. Specifically, we iterate over the set of all starting times and due dates $\mathcal{T} = \{s_j \mid j \in \mathcal{J}\} \cup \{d_j \mid j \in \mathcal{J}\}$ in ascending order, where a due date precedes a starting time of the same value. For each time point $t \in \mathcal{T}$, we perform one of two operations. If $t = d_j$ for some job j, we remove j from H if it is there (job j remains counted in a). If $t = s_j$, we insert j into H and increment a; now $a = |\sigma_{j-1}| + 1$, and H contains exactly job j and the jobs $j' \in \sigma_{j-1}$ with $s_{j'} \leq s_j < d_{j'}$. Hence, one of the two cases above occurs if and only if $s_j < a \cdot p$ or H contains more than m jobs. In this case, we remove from H the job with the largest due date and decrement a. Since every job in σ_{j-1} that is not in H

has a due date of at most s_j, this is indeed the job with the largest due date in $\sigma_{j-1} \cup \{j\}$, as required by the algorithm. Each job is inserted into and removed from H at most once, each such operation takes $O(\log n)$ time, and we process $O(n)$ time points in total, so the algorithm runs in $O(n \log n)$ time, as stated in the first bullet of Theorem 1.7.

Proper Instances

We next present a polynomial-time algorithm for the $FF(1,m)|p_j = p, \text{proper}|\sum w_j Z_j$ problem, the variant where all jobs have uniform preprocessing time p, and $[s_i, d_i) \nsubseteq [s_j, d_j)$ for any pair of distinct jobs $i \neq j \in \mathcal{J}$. We refer to instances $\mathcal{J}$ satisfying this latter condition as *proper instances*. As usual, we assume that the jobs in $\mathcal{J}$ are ordered according to the EST order, so that $s_1 \leq \ldots \leq s_n$. Note that the fact that $\mathcal{J}$ is proper implies that this order is the same as the Early Due Date (EDD) order; that is, we also have that $d_1 \leq \ldots \leq d_n$.

Let $\mathcal{J}$ be an instance of $FF(1,m)|p_j = p, \text{proper}|\sum w_j Z_j$. We present an integer linear program (ILP) formulation that computes the total weight of an optimal feasible solution $\sigma \subseteq \mathcal{J}$. We then show that the resulting constraint matrix of this ILP formulation satisfies the consecutive ones property, which is defined as follows:

Definition 1 *A matrix has the* consecutive ones property *if each of its rows is of the form*

$$(\underbrace{0,\ldots,0}_{a \text{ times}},\underbrace{1,\ldots,1}_{b \text{ times}},\underbrace{0,\ldots,0}_{c \text{ times}})$$

for some a, b and c.

It is well-known that any matrix A that has the consecutive ones property is *totally unimodular* (TUM), *i.e.*, every square submatrix of A has a determinant of -1, 0 or 1 [FG65]. Moreover, it is well known that a linear program whose constraint matrix is TUM and whose right-hand side is integral has an integral optimal solution whenever it has an optimal solution [Sch98], and that such a solution can be found in (weakly) polynomial time [Vai89].

We proceed to formulate the ILP. For each job $j \in \mathcal{J}$, we define a binary variable, where we interpret $x_j = 1$ as the case where job j is added to a feasible solution σ. The objective of our ILP is then to maximize the total weight of jobs added to this solution:

$$\sum_{j=1}^{n} w_j x_j.$$

Apart from the integrality constraints $x_j \in \{0,1\}$ for all $j \in \mathcal{J}$, we include two additional sets of constraints. The first ensures that if $x_j = 1$, its completion time on the preprocessing machine must be prior to s_j. In this way, this set of constraints ensures that our solution can be preprocessed in time. Thus, for each $j \in \mathcal{J}$ we add the following constraint:

$$\sum_{i=1}^{j} x_i \leq \lfloor s_j/p \rfloor. \tag{4.7}$$

The second set of constraints ensures that the selected solution can be scheduled on m machines. This is equivalent to requiring that for each $j \in \mathcal{J}$, the number of jobs $i \in \mathcal{J}$ with $s_i \leq s_j < d_i$ selected into a solution is at most m. Thus, we add the following constraint for each $j \in \mathcal{J}$:

$$\sum_{s_i \leq s_j < d_i} x_i \leq m. \tag{4.8}$$

It is easy to see that any feasible solution $\mathbf{x}$ to our ILP corresponds to a feasible solution σ to the given $FF(1,m)|p_j = p, \text{proper}|\sum w_j Z_j$ instance, and vice versa. Thus, the optimal solution to this ILP corresponds to the optimal solution $\sigma \subseteq \mathcal{J}$. In the next lemma, we prove that the constraint matrix of our ILP has the consecutive ones property, thereby allowing us to optimally solve the ILP by solving its linear programming relaxation.

Lemma 14 *The $2n \times n$ constraint matrix defined by the set of equations 4.7 and 4.8 has the consecutive ones property.*

Proof. First note that for each $j \in \mathcal{J}$, the jth constraint in (4.7) has j consecutive ones followed by $n - j$ zeros. Next, consider a constraint $j \in \mathcal{J}$ in the set of constraints given by (4.8). Note that since $\mathcal{J}$ is proper, for any two jobs $i_1 < i_2 \in \mathcal{J}$ with $s_{i_1} \leq s_j < d_{i_1}$ and $s_{i_2} \leq s_j < d_{i_2}$ we have $s_i \leq s_j < d_i$ for all $i \in \{i_1, \ldots, i_2\}$. Thus, if i_1 is the minimum job index satisfying $s_{i_1} \leq s_j < d_{i_1}$, and i_2 is the maximum such index, then the constraint corresponding to $j \in \mathcal{J}$ in (4.8) has $i_1 - 1$ consecutive zeros, followed by $i_2 - i_1 + 1$ ones, followed by $n - i_2$ zeros. □

We can solve the linear programming relaxation of the ILP using the algorithm of Vaidya [Vai89], which requires $O((c+n)^{1.5} nL)$ arithmetic operations for a linear program with c constraints and n variables, where L is the number of bits required to encode the program. Since $\sum_{i=1}^{j} x_i \leq j$ holds trivially for every $j \in \mathcal{J}$, we may replace the right-hand side of (4.7) by $\min\{j, \lfloor s_j/p \rfloor\} \leq n$ without changing the set of feasible solutions. Our relaxation then has $c = 2n$ constraints, a binary constraint matrix, right-hand sides of value at most n (we may assume $m \leq n$, as otherwise the constraints in (4.8) are redundant), and objective coefficients of value at most W. Hence, $L \in O(n^2 + n \log W)$, and the relaxation can be solved in $O(n^{2.5}(n^2 + n \log W)) = O(n^{3.5}(n + \log W))$ time, which is the running time promised in the second bullet of Theorem 1.7.

4.2.5 Approximation Schemes

In this section, we show how to convert our dynamic programming algorithms of Sections 4.2.1 and 4.2.3 to algorithms that produce $(1-\varepsilon)$-approximate solutions in polynomial time. In particular, using the classical scaling and rounding technique of Ibarra and Kim [IK75], we provide a complete proof for Theorem 1.8.

Theorem 1.8 *$FF(1,m)||\sum w_j Z_j$ admits an FPTAS when $m \in O(1)$, or $q_{\max} \in O(1)$, or when $\chi \in O(\log n)$.*

Let $\mathcal{J}$ be a given instance of $FF(1,m)||\sum w_jZ_j$, and let $\varepsilon > 0$. Throughout this section, we assume that $p_j \leq s_j$ for all jobs $j \in \mathcal{J}$ (as jobs not fulfilling this requirement can be discarded), and so the optimal solution for $\mathcal{J}$ has total weight at least $w_{\max} = \max_j w_j$. To construct our approximation scheme, we define a *scaling factor* $k = \varepsilon \cdot w_{\max}/n$. We then use the scaling factor k to construct an alternative weight function for the set of jobs $\mathcal{J}$. Accordingly, we define $\overline{w}_j = \lceil w_j/k \rceil$ for each $j \in \mathcal{J}$. Note that $k \cdot (\overline{w}_j - 1) \leq w_j \leq k \cdot \overline{w}_j$ for each $j \in \mathcal{J}$. We use $\overline{w}$ to denote the modified weight function, and we let $\mathcal{J}^*$ be the job set obtained from $\mathcal{J}$ by replacing the weight function w with $\overline{w}$. Let σ^* denote the optimal feasible solution of $\mathcal{J}^*$. We use σ^* as our suggested approximate solution for $\mathcal{J}$.

Let σ denote the optimal feasible solution for the original job set $\mathcal{J}$, and let $\overline{w}(\sigma^*)$ and $\overline{w}(\sigma)$ be the total weight of jobs in σ^* (resp. σ) according to the weight function $\overline{w}$. It follows that

$$\overline{w}(\sigma^*) \geq \overline{w}(\sigma) = \sum_{j \in \sigma} \lceil w_j/k \rceil \geq \sum_{j \in \sigma} w_j/k,$$

and so

$$k \cdot \overline{w}(\sigma^*) \geq \sum_{j \in \sigma} w_j = w(\sigma).$$

Thus, we get

$$\begin{aligned} w(\sigma^*) &= \sum_{j \in \sigma^*} w_j \geq \sum_{j \in \sigma^*} k \cdot (\overline{w}_j - 1) = \sum_{j \in \sigma^*} k \cdot \overline{w}_j - \sum_{j \in \sigma^*} k \\ &= k \cdot \overline{w}(\sigma^*) - k \cdot |\sigma^*| \geq k \cdot \overline{w}(\sigma^*) - kn \geq w(\sigma) - kn \\ &= w(\sigma) - \varepsilon \cdot w_{\max} \geq w(\sigma) - \varepsilon \cdot w(\sigma) = (1 - \varepsilon) \cdot w(\sigma). \end{aligned}$$

Note that the last above inequality holds because $w_{\max}$ is a lower bound on the optimal solution value of $\mathcal{J}$. It follows that σ^* is a $(1 - \varepsilon)$-approximation solution for $\mathcal{J}$.

Note that σ^* can be obtained by applying any optimization algorithm for the problem by replacing the original weights with the modified ones. This gives an improved running time if the running time of the algorithm depends on the total weight $W = w(\mathcal{J})$ of the instance, as is the case for the dynamic programming algorithms presented in Sections 4.2.1 and 4.2.3. Let $\overline{W} = \overline{w}(\mathcal{J})$ be the total weight of the jobs according to $\overline{w}$. Then the fact that

$$O(\overline{W}) = O(\sum \overline{w}_j) = O(\sum w_j/k) = O\left(\frac{n \cdot \sum w_j}{\varepsilon \cdot w_{\max}}\right) = O\left(\frac{n^2}{\varepsilon}\right),$$

together with the results in Theorems 1.5 and 1.6, completes the proof of Theorem 1.8.

4.3 Discussion

We conducted a systematic study of the two-stage flexible flow shop scheduling problem $FF(1,m)||\sum w_jZ_j$. We showed that the unweighted version is strongly NP-hard when the number of second-stage machines is part of the input, whereas for constant m even the

weighted problem becomes pseudo-polynomial-time solvable and admits an FPTAS. We further extended this tractability result to the settings where $q_{\max} \in O(1)$ or $\chi \in O(\log n)$. We also examined several special cases, such as the case where $p_j = p$ for all jobs, and presented either pseudo-polynomial or polynomial-time algorithms for all these cases.

While our results address a broad range of settings for the $FF(1,m)||\sum w_j Z_j$ problem, several important questions remain open. Most notably, the existence of a constant-factor approximation algorithm for the problem, either in its weighted form $FF(1,m)||\sum w_j Z_j$ or even in the unweighted variant $FF(1,m)||\sum Z_j$, is still unresolved in the general case, where no restrictions are placed on parameters such as m, $q_{\max}$, or χ. Identifying whether such approximability results are possible, or whether the problem admits strong inapproximability bounds, remains a compelling direction for further study. Note that our strong NP-hardness result already excludes an FPTAS for the general problem unless P=NP; the question is thus whether a PTAS or a constant-factor approximation exists, or whether stronger inapproximability bounds can be shown.

In addition, our work primarily focused on two structural parameters, $q_{\max}$ (the maximum length of any second-stage operation) and χ (the chromatic number of the second-stage conflict graph). Both dynamic programs in Section 4.2.2 and Section 4.2.3 admit a dual formulation, symmetric to the one in Corollary 9, in which the roles of W and P are exchanged. This shows that the problem is FPT with respect to χ whenever either the weights or the processing times are polynomially bounded (equivalently, encoded in unary). Yet, this is not the case for the parameter $q_{\max}$. In such a case, our results only yield pseudo-polynomial-time algorithms, and only when $q_{\max}$ is constant, via the $O(W \cdot m^{q_{\max}} \cdot n)$ and $O(P \cdot m^{q_{\max}} \cdot n)$ bounds. Whether a polynomial bound on W or P is enough to enable an FPT algorithm parameterized by $q_{\max}$, as it does for χ, remains open.

Open Question 3 *Is $FF(1,m)||\sum w_j Z_j$ fixed-parameter tractable with respect to $q_{\max}$ when either W or P are polynomial in the input size?*

It would also be natural to explore other parameters that may influence the problem's complexity. For instance, one could consider $p_{\max} = \max_j p_j$, the maximum job preprocessing time, or α, the maximum number of pairwise non-conflicting jobs (i.e., the *independence number* of the conflict graph). These and other parameters may reveal additional tractable cases or offer new algorithmic strategies.

Finally, a natural next step is to investigate the complementary two-stage problem $FF(m,1)||\sum w_j Z_j$, in which the first stage consists of m parallel machines and the second stage has a single machine. This setting models systems where jobs undergo parallel preprocessing before a centralized JIT operation. It is not difficult to show that even the unweighted variant $FF(m,1)||\sum Z_j$ is strongly NP-hard via a reduction from the classical Bin Packing problem, in which we ask whether n items fit into m bins of capacity B. The idea is to give the i-th job a unit second-stage operation with due date $B+i$. The second-stage intervals are then pairwise disjoint, so any subset of jobs can be processed on the single second-stage machine without conflicts, and all preprocessing must essentially

be completed by time B; the m first-stage machines thus act precisely as m bins of capacity B. Since Bin Packing arises as a special case, the general problem is computationally intractable [Joh+74, GJ78].

Nevertheless, as with the $FF(1, m)$ setting, there may exist special cases that admit efficient algorithms. For example, it would be natural to examine whether the problem becomes easier when job processing times are uniform or when structural parameters such as $q_{\max}$, χ, or α are small. Extending the techniques and structural insights developed for $FF(1, m)||\sum w_j Z_j$ to this setting is a promising direction for future work.

Chapter 5

Fair Repetitive Interval Scheduling

"Social and economic inequalities are to be arranged so that they are to the greatest benefit of the least advantaged."

— John Rawls, *A Theory of Justice*

In this chapter, we study interval scheduling in a setting where a fixed set of clients is served repeatedly over time and fairness plays a central role. A schedule must not only be feasible on each day, but also guarantee each client sufficiently many Just-in-Time completions over the entire horizon. Such fairness requirements arise naturally in service systems where long-term client satisfaction is as important as short-term efficiency, for example in healthcare, maintenance planning, or shared computational resources. The objective is to maximize the minimum quality of service a client receives over the entire period, measured by the number of JIT jobs.

$1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$:

Input: A set of n clients, where each client $j \in \{1, \ldots, n\}$ has a job with due date $d_{i,j} \in \mathbb{N}$ and processing time $p_{i,j} \in \mathbb{N}$ for each day $i \in \{1, \ldots, q\}$, and a target value $k \in \mathbb{N}$.

Question: Are there q schedules, $\sigma_1, \ldots, \sigma_q$, such that σ_i is feasible on each day $i \in \{1, \ldots, q\}$, and $\sum_i Z_{i,j} \geq k$ for each $j \in \{1, \ldots, n\}$?

Fairness as an objective is a relatively new and unexplored concept in scheduling problems. In recent years, customer satisfaction has gained prominence as a goal of high importance [Hom00, LWC11]. This shift has naturally led to an ascent in the prioritization of fairness, compared to alternative scheduling objectives [SPC17, Che+11, LLP97, Dou+07]. Despite the general pursuit of fair resource allocation, no clear and universally accepted notion of fairness exists [Gup+26]. Fairness is context-dependent and varies with customer preferences. Any set of preferences may necessitate a unique model for fair resource allocation.

For this reason, there is a significant lack of theoretical models that support fair decision-making in service systems, particularly in the context of fair scheduling decisions.

Traditionally, deterministic scheduling models focus on a single scheduling horizon (*e.g.*, a single day). Such models typically aim to find an optimal daily schedule from a global perspective, while ignoring fairness issues altogether. When jobs are associated with clients, an optimal daily schedule may result in a large variance in the Quality of Service (QoS) that each client receives. For example, a daily schedule may minimize the total tardiness of all jobs submitted by the clients, at the expense of a single client whose job is significantly tardier than the others. Hence, in such scenarios, solely focusing on the global perspective may lead to poor customer satisfaction and future abandonment of customers.

To promote fairness, one may consider a daily schedule that ensures the highest possible QoS for the *least satisfied* client. Accordingly, if tardiness is used as a measure of QoS, a fair solution minimizes the maximum tardiness. In a repetitive setting, however, optimizing each day individually may still produce an *unfair* outcome: the same client may be the worst-off on every single day. Fairness must therefore be measured across the entire scheduling horizon, and providing a framework for such decisions is precisely the goal of fair repetitive scheduling.

The daily conflict relation between the jobs of the clients can be usefully viewed through the prism of graph theory. This relation can be modeled as a *daily conflict graph* which will be used throughout the chapter. We also define the *overall conflict graph* which will be used to prove Theorem 1.12:

Definition 2 *Given an instance $\mathcal{I}$ of* $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ *with n clients and q days, the* day i conflict graph *associated with $\mathcal{I}$ is the graph $G_i = (\{1, \ldots, n\}, E_i)$ where each vertex $j \in \{1, \ldots, n\}$ is associated with a client j of $\mathcal{I}$. Two vertices $j_1, j_2 \in \{1, \ldots, n\}$ are adjacent in G_i if their corresponding clients j_1 and j_2 have a pair of conflicting jobs on day i. The* overall conflict graph *associated with $\mathcal{I}$ is the graph $G = (\{1, \ldots, n\}, E_1 \cup \cdots \cup E_q)$.*

Note that for each $i \in \{1, \ldots, q\}$, the day i conflict graph is an *interval graph*, an intersection graph of intervals on the real line [Haj57, Gol80].

One prominent parameter that we will consider is the *treewidth* [RS86] of a graph. This parameter is defined through *tree decompositions* as follows:

Definition 3 *A tree-decomposition of a graph $G = (V, E)$ is a tree $\mathcal{T} = (\mathcal{X}, F)$ with a node set $\mathcal{X} \subseteq \{X \mid X \subseteq V\}$ and $F \subseteq \mathcal{X} \times \mathcal{X}$ that upholds the following:*

- *$V = \bigcup_{X \in \mathcal{X}} X$.*
- *For every edge $\{u, v\} \in E$ there exists $X \in \mathcal{X}$ such that $u, v \in X$.*
- *The set of nodes $\mathcal{X}_v = \{X \mid X \in \mathcal{X} \wedge v \in X\}$ containing any vertex $v \in V$ is a connected subgraph in $\mathcal{T}$.*

The width of a tree-decomposition $\mathcal{T}$ is $\max\{|X| \mid X \in \mathcal{X}\} - 1$. The treewidth $\omega(G)$ of G is the minimum width over all tree decompositions of G.

It is known that computing the treewidth of a graph is NP-hard, yet it is FPT with respect to the parameter treewidth [Bod93]. Hence, when describing an FPT algorithm with respect to the treewidth parameter, one can assume that a tree decomposition with minimum width is given alongside the input.

Although treewidth is defined in terms of the existence of a tree decomposition of small width, algorithmic applications typically rely on working with an explicit decomposition. To facilitate dynamic programming, it is standard to transform an arbitrary tree decomposition into a *nice tree decomposition*, introduced by Kloks [Klo94], which has a restricted set of node types and a canonical structure. Such a transformation preserves bounded treewidth and enables concise and uniform algorithmic descriptions.

Definition 4 *A nice tree decomposition is a tree decomposition $\mathcal{T} = (\mathcal{X}, F)$ where the following conditions additionally hold:*

1. *Every node $X \in \mathcal{X}$ has at most two children.*
2. *If node X has two children X' and X'' then $X = X' = X''$, and X is called a join node.*
3. *If node X has a single child X' then either of the following two conditions holds:*
   - *i. $X = X' \setminus \{j\}$ for some client $j \in X'$. In this case, we call X a forget node.*
   - *ii. $X = X' \cup \{j\}$ for some client $j \in \{1, \ldots, n\} \setminus X'$. In this case, we say that X is an introduce node.*

It is known that a nice tree decomposition of width $\omega = \omega(G)$ of a graph G can be computed from a given tree decomposition of equal width in linear time [AN02, Klo94].

5.1 Hardness

5.1.1 The Fairness Parameter and the Number of Days

In this section, we prove Theorem 1.9, which maps out the effect of parameters k and q (the fairness parameter and the number of days) on the computational hardness of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$. We begin with an NP-hardness proof for the case that $k = 1$ and $q = 3$ in Theorem 5.1, which we later extend to show NP-hardness for every pair $(q, k) \in \mathbb{N}^2$ such that $0 < k < q-1$ and $q \geq 3$. We complement these hardness results with Theorem 5.7 where we specify a reduction from $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ where $k = q - 1$ to 2-SAT, essentially showing a polynomial time algorithm for the remaining case.

We begin by showing that $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ is NP-hard for $k = 1$ and $q = 3$, even if each job of each client has the same processing time p.

Theorem 5.1 *The $1 \mid \text{rep}, p_{i,j} = p \mid \min_j \sum_i Z_{i,j}$ problem is NP-hard for $k = 1$ and $q = 3$.*

Proof. We present a polynomial-time many-one reduction from [2,3]–BOUNDED 3–SAT [Tov84] to $1 \mid \text{rep}, p_{i,j} = p \mid \min_j \sum_i Z_{i,j}$ with $k = 1$ and $q = 3$. In [2,3]–BOUNDED 3–SAT we need to decide whether a Boolean formula ϕ in conjunctive normal form is satisfiable, given the promise that every clause in ϕ contains two or three variables and every variable appears in at most three clauses. We can also assume without loss of generality that no variable x appears all three times either negated or non-negated, that is, because we can create an equivalent [2,3]–BOUNDED 3–SAT formula ϕ' by setting x to satisfy all three clauses and removing its clauses from ϕ.

Construction 5 *Given a [2,3]–BOUNDED 3–SAT formula ϕ, we construct an instance $\mathcal{I}$ of $1 \mid \text{rep}, p_{i,j} = p \mid \min_j \sum_i Z_{i,j}$ as follows:*

- *For every variable x, we create two clients: x^T and x^F.*
- *For every clause that contains two literals c_A, we create two clients: c^{A_1} and c^{A_2}.*
- *For every clause that contains three literals c_B, we create three clients: c^{B_1}, c^{B_2}, and c^{B_3}.*
- *We create three "dummy" clients: a_1, a_2, and a_3.*

Let α denote the number of variables in ϕ, let β_A denote the number of two-literal clauses (type A), and let β_B denote the number of three-literal clauses (type B). Accordingly, the number of clauses in ϕ is $\beta_A + \beta_B$. The first and second days contain variable and clause gadgets. The variable gadgets ensure that every variable is consistently set in all of its clauses. The clause gadgets ensure that each clause is satisfied by one of its literals.

As all jobs have the same processing time, we define every job's execution interval solely by its due date. We set all processing times to 2. On the first day (see Figure 5.1), we have the following.

- *The due date of the jobs of a_1, a_2, and a_3 is 2.*
- *The due date of the jobs of x_ℓ^T and x_ℓ^F is $2\ell + 3$.*
- *The due date of the jobs of $c_\ell^{A_1}$ and $c_\ell^{A_2}$ is $2\alpha + 2\ell + 5$.*
- *The due date of the jobs of $c_\ell^{B_1}$, $c_\ell^{B_2}$, and $c_\ell^{B_3}$ is $2\alpha + 2\beta_A + 2\ell + 7$.*

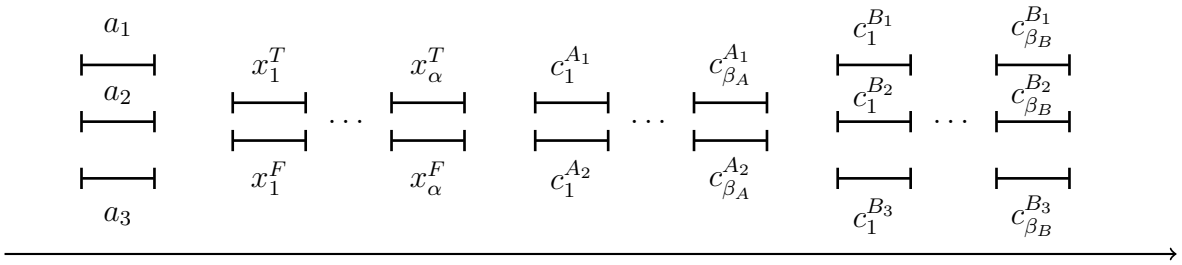


Figure 5.1: Day 1.

On the second day (see Figure 5.2), we have the following.

- *The due date of the jobs of* a_1*,* a_2*, and* a_3 *is* 2*.*
- *The due date of the jobs of* x_ℓ^T *and* x_ℓ^F *is* 2*.*
- *The due date of the jobs of* $c_\ell^{A_1}$ *and* $c_\ell^{A_2}$ *is 2.*
- *The due date of the jobs of* $c_\ell^{B_1}$*,* $c_\ell^{B_2}$*, and* $c_\ell^{B_3}$ *is* $3 + 3\ell$*.*

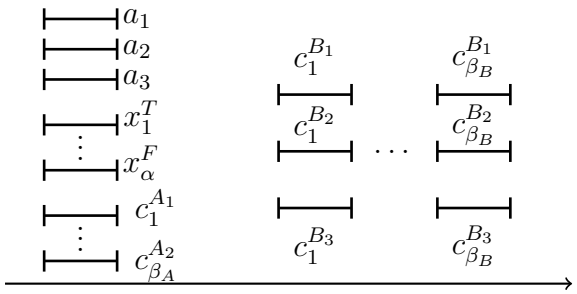


Figure 5.2: Day 2.

On the third day (see Figure 5.3), we have the following.

- *The due date of the jobs of* a_1*,* a_2*, and* a_3 *is* 2*.*
- *The due date of the job of* x_ℓ^T *is* $10\ell - 4$*. The due date of the job of* x_ℓ^F *is* $10\ell + 1$*.*
- *Let* c_1*,* c_2*, and* c_3 *be the clauses in which* x *appears. As we can assume that* x *does not homogeneously appear in all three clauses,* x *must appear negated in at most two clauses. Symmetrically, we can assume that* x *appears non-negated in at most two clauses. Let* $c_1(x_\ell^F)$ *and* $c_2(x_\ell^F)$ *be the literals in which* x *appears negated, the due date of* $c_1(x_\ell^F)$ *is* 10ℓ*. If* $c_2(x_\ell^F)$ *exists (*i.e.*, if* x_ℓ *appears negated in more than one clause), then its due date is* $10\ell + 2$*. In the same fashion the due date of* $c_1(x_\ell^T)$ *is* $10\ell - 5$ *and if* $c_2(x_\ell^T)$ *exists, then its due date is* $10\ell - 3$*.*

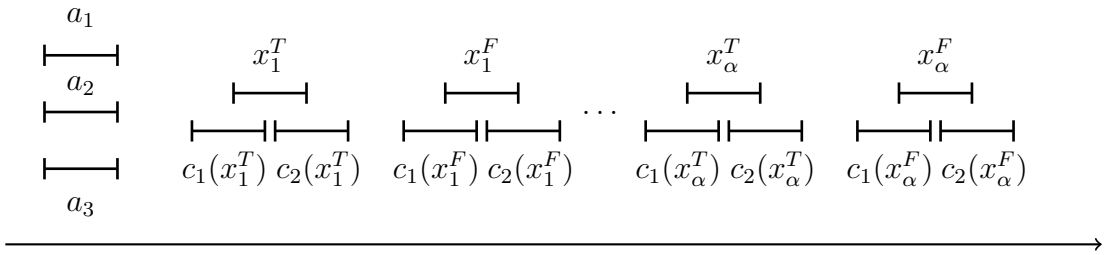


Figure 5.3: Day 3.

Finally, we set $k = 1$*. This finishes the construction of* $\mathcal{I}$*, which can clearly be computed in polynomial time. The scheduling period of* $\mathcal{I}$ *is* 3 *days long.*

($\Rightarrow$) We show that if ϕ is satisfiable, then $\mathcal{I}$ has a feasible 1-fair schedule. Assume that the assignment satisfies all clauses of ϕ, then the following schedule is 1-fair: The job of client a_1 is to be scheduled on day one, and the jobs of clients a_2 and a_3 are to be scheduled on days two and three, respectively. For every x_ℓ that is assigned `true`, schedule the job of x_ℓ^T on day one. In the same fashion, schedule on day one the job of x_ℓ^F for every x_ℓ that is assigned `false`. For each variable x_ℓ that is set to `true` schedule the job of client x_ℓ^F on day three, otherwise schedule on day three the job of x_ℓ^T. Let the ℓ-th type A clause be satisfied with the assignment of literal c^{A_1} then on day one schedule the job of c^{A_2} and on day three the job of c^{A_1}, otherwise schedule the job of c^{A_1} on day one and the job of c^{A_2} on day three. For any type B clause, without loss of generality, assume that c^{B_1} satisfies the clause, then schedule its job on day three. The other clients' jobs c^{B_2} and c^{B_3} are to be scheduled on days one and two, respectively. This schedule is feasible because none of the scheduled jobs conflict, and it is 1-fair because every client gets at least one job scheduled.

($\Leftarrow$) We show that if $\mathcal{I}$ has a 1-fair schedule, then ϕ is satisfiable. Let σ be a feasible and 1-fair schedule and let $x(\sigma)$ be an assignment of ϕ that is computed from σ in the following fashion: For every ℓ we set x_ℓ to `true` if x_ℓ^T is scheduled on the first day, otherwise we set it to `false`. Assume that ϕ is not satisfiable by $x(\sigma)$, then there must exist a clause c_ℓ in ϕ that evaluates to `false`. Assume that the type A clause c_ℓ^A is not satisfied, then both literals $c_\ell^{A_1}$ and $c_\ell^{A_2}$ evaluate to `false`. Let the literal of $c_\ell^{A_1}$ be satisfied by setting x_i to either `true` or `false`, we denote by x_i^Y the client x_i^T if and only if $c_\ell^{A_1}$ is satisfied by setting x_i to `true` (otherwise x_i^Y denotes x_i^F). It is clear that since σ is feasible and 1-fair, one of a_1, a_2, and a_3 must be scheduled on each day. It follows that because $c_\ell^{A_1}$ and $c_\ell^{A_2}$ conflict on day one and conflict on day two with the jobs of clients a_1, a_2, and a_3, one of them must be scheduled on day three. Assume, without loss of generality, that $c_\ell^{A_1}$ is scheduled on day three, then the client x_i^Y must be scheduled on day one because it conflicts with $c_\ell^{A_1}$ on day three. It then follows that x_i^Y satisfies c_ℓ because in the assignment that corresponds to σ, a variable is set to `true` if it is scheduled on day one.

In the same fashion, assume that the type B clause c_ℓ^B is not satisfied, then all three literals $c_\ell^{B_1}$, $c_\ell^{B_2}$ and $c_\ell^{B_3}$ evaluate to `false`. Assume, without loss of generality, that $c_\ell^{B_1}$ is scheduled in σ on day three, then clearly $c_\ell^{B_2}$ and $c_\ell^{B_3}$ are scheduled on days one and two respectively. Let $c_\ell^{B_1}$ be satisfied by assigning x_i to `true`, then by the construction x_i^T conflicts with $c_\ell^{B_1}$ on day three. Since σ is conflictless, it must be that x_i^T is scheduled on day one, yet this contradicts that c_ℓ^B is not satisfied by $x(\sigma)$. Similarly, when $c_\ell^{B_1}$ is satisfied by assigning x_i to `false`, x_i^F must be scheduled on day one, which contradicts our assumption that c_ℓ^B is not satisfied by $x(\sigma)$. □

We next use the NP-hardness proof for the case of $(q, k) = (3, 1)$ of Theorem 5.1 as a base case to inductively show NP-hardness for every $(q, k) \in \mathbb{N}^2$ where $0 < k < q - 1$ and $q \geq 3$. We show this via two induction steps from (q, k), one to $(q + 1, k + 1)$ and the other to $(q + 1, k)$.

Observe that an NP-hardness result for any pair (q, k) implies an NP-hardness for the pair $(q + 1, k + 1)$, as we can add a single day with no conflicts to any instance $\mathcal{I}$

of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ with q days. Clearly $\mathcal{I}$ is k-fair if and only if it is $(k+1)$-fair when the extra day without conflicts is added.

Moreover, observe that NP-hardness for any pair $(q, 1)$ implies NP-hardness for $(q+1, 1)$, as can be seen by adding an additional client and an additional day to an instance $\mathcal{I}$ such that the new client conflicts with every other client on each of the first q days, and on the new day the jobs of *all* clients are mutually conflicting. If $\mathcal{I}$ is 1-fair, then we can clearly schedule a job of the new client on day $q+1$ to obtain a 1-fair schedule for the new instance. Conversely, if the new instance is 1-fair, then we can assume by a swapping argument that the new client has its job scheduled on day $q+1$, implying that $\mathcal{I}$ is 1-fair as well. This can be done with day-independent processing times, i.e., each client has the same processing time throughout all days.

Using these two steps, we can reduce the case of $(3, 1)$ to every instance $(3+i+j, 1+i)$ by using i times the first reduction after applying j times the second reduction. Thus we obtain the following corollary:

Corollary 2 *The* $1 \mid \text{rep}, p_{i,j} = 2 \mid \min_j \sum_i Z_{i,j}$ *problem is* NP*-hard when* $0 < k < q - 1$ *and* $q \geq 3$.

Observe that Corollary 2 gives us the first bullet of Theorem 1.10.

5.1.2 The Treewidth of the Overall Conflict Graph

We begin by showing that $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ is NP-hard even when the treewidth ω of the overall conflict graph is constant. For this, we first show a hardness result for a generalization of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$, and then we reduce this generalized version to $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ such that the treewidth increases only by a small constant. In the generalized version $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$ of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$, we are given a number k_j for each client j, and are asked to find a schedule with $\sum_i Z_{i,j} \geq k_j$ for every j. Thus, each client has its own fairness parameter, instead of the instance having a single global parameter.

Lemma 3 *The* $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$ *problem is* NP*-hard even when the overall conflict graph has treewidth at most 4.*

Proof. We reduce from the NP-hard Multicolored Independent Set problem [Pie03, Fel+09]. In the Multicolored Independent Set problem, we get as input an ℓ-partite graph $G = (V_1 \cup \ldots \cup V_\ell, E)$, where each vertex class in the partition is referred to as a *color class* (and vertices $v \in V_i$ are of *color* i). The question is whether there exists a multicolored independent set I of size ℓ in G, *i.e.*, a set I of nonadjacent vertices in G with $|I \cap V_i| = 1$ for each $i \in \{1, \ldots, \ell\}$. Let $G = (V_1 \cup \ldots \cup V_\ell, E)$ be an instance of Multicolored Independent Set. We can assume without loss of generality that G is r-regular (*i.e.*, every vertex has r neighbors), that $|V_1| = \cdots = |V_\ell| = n$, that $|E|$ is even, and that $n \geq 4$. Let $V_i = \{v_1^i, \ldots, v_n^i\}$ for each $i \in \{1, \ldots, \ell\}$ and let $E = \{e_1, \ldots, e_{|E|}\}$. To simplify notation,

we assume that G is directed, where each edge connecting $v \in V_i$ to $u \in V_j$ with $i < j$ is directed from V_i to V_j.

Construction 6 *We next describe the construction of the* $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$ *instance. The instance will contain* $\ell \cdot (n+1) + |E|$ *days: For every color* $i \in \{1, \ldots, \ell\}$*, we create* n *vertex days and a single validation day. These will allow us to encode which vertex from* V_i *is selected to be in the independent set. For every edge* $e \in E$ *we create a single edge day, which will help ensure that the selection done in vertex days indeed encodes a multicolored independent set in* G*. The clients of the* $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$ *instance are constructed as follows:*

- *For vertex* $v \in V_1 \cup \cdots \cup V_\ell$ *we create a vertex client* c_v *with a fairness parameter of* $k_v = 1$*.*
- *For each color* $i \in \{1, \ldots, \ell\}$ *we create a selection client* c_i *with fairness parameter* $k_i = 1$*.*
- *For each edge* $(v, u) \in E$ *we create two edge clients* $c_{v,u}$ *and* $c_{u,v}$ *with* $k_{v,u} = k_{u,v} = 1$*.*
- *We create two additional interaction clients* c^+ *and* c^- *with* $k^+ = k^- = |E|/2$*.*
- *We create a single dummy client* c_0 *with* $k_0 = \ell \cdot (n+1) + |E|$*.*

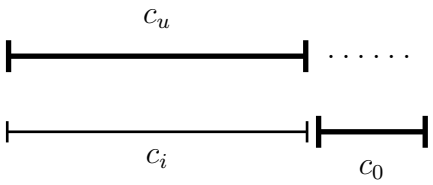
The general idea of the construction is that for every color, the vertex days and validation day form a vertex selection gadget which compels the selection of a single vertex client for the color (Figure 5.4). The edge days (Figure 5.5) form an incidence-checking gadget which certifies that all clients are satisfied only if the selected vertices form a multicolored independent set in G*. On each day, only a few clients take part in the gadget, and the job of the dummy client* c_0 *is used to block the jobs of all other clients. To this end, the job of* c_0 *starts right after the last job of the gadget, and every client that does not take part in the gadget has a unit-length job of its own inside the job of* c_0*. Formally, let* N *be the number of clients other than* c_0*, and fix an arbitrary bijection* π *between these clients and* $\{1, \ldots, N\}$*; the job of* c_0 *has processing time* N*, and the job of such a client* c *occupies the* $\pi(c)$*'th time unit of it. More specifically, for every vertex* $v \in V_1 \cup \cdots \cup V_\ell$ *we create* v*'s vertex day as follows:*

- *Client* c_v*'s job has due date* r *and processing time* r*.*
- *Client* c_i*'s job has due date* r *and processing time* r*.*
- *Client* c_0*'s job has due date* $r + N$ *and processing time* N*.*
- *Every other client* c *has a job with due date* $r + \pi(c)$ *and processing time* 1*.*

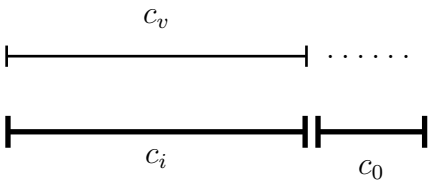
For every color $i \in \{1, \ldots, \ell\}$ *we create* i*'s validation day as follows:*

- *For every* $v = v_p^i \in V_i$ *we set the due date of* c_v*'s job to* rp *and its processing time to* r*.*

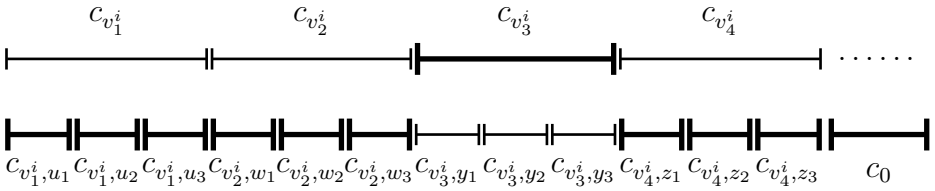
- *For every $v = v_p^i \in V_i$ we define an arbitrary order over v's neighbors. Let u be the q'th neighbor of v, then $c_{v,u}$ has due date $r(p-1)+q$ and processing time 1.*
- *Client c_0's job has due date $rn+N$ and processing time N.*
- *Every other client c has a job with due date $rn+\pi(c)$ and processing time 1.*



(a) The vertex-selection day for $u \in V_i$, where $u \notin I$.



(b) The vertex-selection day for $v \in V_i$, where $v \in I$.



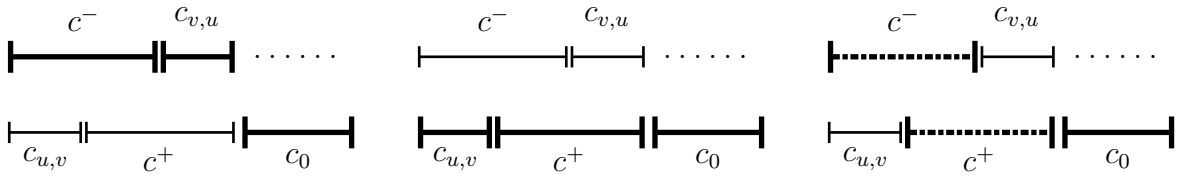
(c) The i-th validation day where $v_3^i \in I$.

Figure 5.4: An example of schedules on the vertex selection days and the validation days given a multicolored independent set I. We mark in **bold** the jobs which are scheduled; the jobs not appearing in the gadget are pairwise disjoint and are all contained in the job of client c_0.

For every edge $(v,u) \in E$ we create (v,u)'s edge day:

- *Client c^-'s job has due date 2 and processing time 2.*
- *Client c^+'s job has due date 3 and processing time 2.*
- *Client $c_{v,u}$'s job has due date 3 and processing time 1.*
- *Client $c_{u,v}$'s job has due date 1 and processing time 1.*
- *Client c_0's job has due date $3+N$ and processing time N.*
- *Every other client c has a job with due date $3+\pi(c)$ and processing time 1.*

Observe that the jobs of the dummy client c_0 must be scheduled on all $\ell(n+1)+|E|$ days to satisfy its fairness requirement k_0. These jobs of c_0 are used in the gadget creation to *"block"* the execution of jobs that are irrelevant for the gadget. For this reason, every vertex client c_v can only be scheduled on v's *vertex day* or on the *validation day* that corresponds to v's color. For every color $i \in \{1,\ldots,\ell\}$, the selection client c_i can only be scheduled on



(a) $v \in I, u \notin I$.

(b) $v \notin I, u \in I$.

(c) $u, v \notin I$: schedule either c^+ or c^- such that c^+ and c^- are satisfied.

Figure 5.5: The three possible cases on an *edge day* (with $v \in V_i$ and $u \in V_j$ with $i < j$) given a multicolored independent set I. We mark the jobs which are included in a schedule constructed from I in **bold**, and the jobs which do not appear in the figure are pairwise disjoint and are all contained in the job of client c_0 (but are not scheduled). There are $\ell \cdot r$ days of types (a) and (b), as $|E|$ is even we can freely use (c) days to ensure both c^+ and c^- are satisfied.

one of i's *vertex days*, as it is blocked by the dummy client in the rest of the days. This means that ℓ vertex clients must be scheduled on the ℓ *validation days*, as the ℓ selection clients block them on their vertex days. See Figure 5.4 for illustration of these two gadgets.

The aforementioned *vertex-selection* gadget forces the selection of ℓ vertices to satisfy the fairness requirements of the selection clients and vertex clients. Every edge client $c_{v,u}$ can be scheduled on the validation day of v's color as long as v is not selected. Thus, this leaves $r \cdot \ell$ edge clients that need to be scheduled on the edge days. The fairness requirements of c^+ and c^- force the scheduling of one of their jobs on every single edge day. Essentially, the edge days and the validation days form an *incidence-checking* gadget. Assuming that the jobs of the selected clients form an independent set I, there exists no edge (v, u) with $u, v \in I$ whose edge clients must be served on the same day, therefore c^+ and c^- may be satisfied. See Figure 5.5 for illustration.

After describing the reduction, we continue by showing that the overall conflict graph of the constructed instance indeed has bounded treewidth and that it is correct. We can construct a tree-decomposition $(\mathcal{X}, F)$ of width four as described in Figure 5.6.

($\Rightarrow$) We next show that if G contains a multicolored independent set $I \subseteq V_1 \cup \cdots \cup V_\ell$ of size ℓ then our constructed $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$ instance has a solution in which each client's fairness parameter is met. We construct such a solution as follows: On each day, the dummy client c_0 is scheduled. For each $i \in \{1, \ldots, \ell\}$, the clients from the vertex-selection gadget are scheduled as follows: For each vertex $v = v_p^i \in V_i \setminus I$, we schedule the job of c_v on the p'th vertex day of the i'th color. For $v = v_p^i \in V_i \cap I$, we schedule the selection client c_i on the p'th vertex day of the i'th color and the job of c_v on i'th validation day. Additionally, on the i'th validation day, all clients $c_{v,u}$ are scheduled for each $v \in V_i \setminus I$. Finally, for each edge $e_p = (v, u)$ or $e = (u, v)$ incident to $v \in V_i \cap I$, we schedule client $c_{v,u}$ on the p'th edge day. Note that by this construction, all these jobs can be scheduled on their appropriate

days without conflicts. It remains to schedule the interaction between clients c^+ and c^-. For each edge day of an edge (v, u) with $v \in I$ or $u \in I$, we schedule the job of c^- if $v \in I$, and the job of c^+ if $u \in I$ (see Figure 5.5). Note that this does not lead to a conflict, as I is an independent set, only one of $c_{v,u}$ and $c_{u,v}$ is scheduled on this day, and on (v, u)'s edge day the job of $c_{v,u}$ conflicts only with the job of c^+, while the job of $c_{u,v}$ conflicts only with the job of c^-. On the remaining edge days, no client has been scheduled so far (except for the dummy client), and we schedule either c^+ or c^- in a way that, in the end, c^+ and c^- are scheduled equally often. This is possible since exactly $r\ell$ edge days correspond to edges incident to I, and $n \geq 4$ implies $r\ell \leq |E|/2$ (recall that $|E| = rn\ell/2$), so neither interaction client has been scheduled more than $|E|/2$ times so far. Altogether, the interaction clients c^+ and c^- are scheduled on $|E|/2$ days each, the dummy client is scheduled on every day, and all other clients are scheduled on a single day.

($\Leftarrow$) Assume that there is a schedule to the constructed $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$ instance in which the fairness parameter of each client is met. Then, the jobs of the dummy client are scheduled each day, and so, each selection client c_i has to have his job scheduled on some vertex day. For each color $i \in \{1, \ldots, \ell\}$, let $p_i \in \{1, \ldots, n\}$ be an index such that the job of selection client c_i is scheduled on the p_i'th vertex day of color i. Fix a color i and consider the vertex $v = v^i_{p_i} \in V_i$. By construction, client c_v needs to be scheduled on the i'th validation day as otherwise the job of the dummy client cannot be scheduled. This implies that a job of the edge client $c_{v,u}$ is scheduled on (v, u)'s edge day for each neighbor u of v in G. The jobs of the interaction clients c^+ and c^- can only be scheduled on the edge days, and at most one interaction client job per day (as c^+ and c^- conflict on each day). As $k^+ + k^- = |E|$, it follows that on each edge day, exactly one of c^+ and c^- is scheduled. Consequently, there is no edge (v, u) such that on (v, u)'s edge day the jobs of both $c_{v,u}$

Nodes $\mathcal{X}$:

- $X_{c_0} = \{c_0, c^+, c^-\}$
- $\forall i \in \{1, \ldots, \ell\} \ : \ X_{S_i} = \{c_0, c^+, c^-, c_i\}$
- $\forall v \in V \ : \ X_v = \{c_0, c^+, c^-, c_v, c_i\}$
- $\forall u \in N(v) \ : \ X_{v,u} = \{c_0, c^+, c^-, c_v, c_{v,u}\}$

Edges $F = F_1 \cup F_2 \cup F_3$:

- $F_1 = \{(X_{c_0}, X_{S_i}) \mid i \in \{1, \ldots, \ell\}\}$
- $F_2 = \{(X_{S_i}, X_v) \mid v \in V_i\}$
- $F_3 = \{(X_v, X_{v,u}) \mid v \in V \wedge u \in N(v)\}$

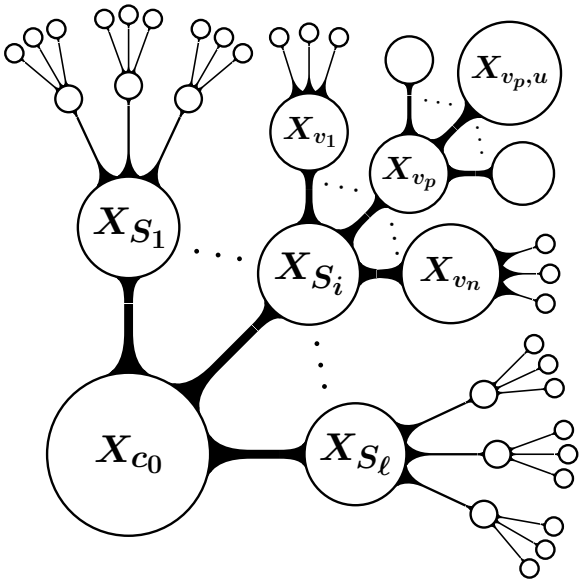
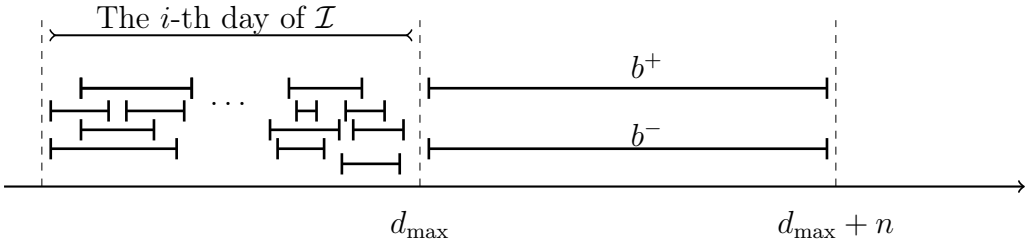


Figure 5.6: A tree decomposition $(\mathcal{X}, F)$ of the overall conflict graph constructed from the instance of $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$. Observe that the width of this decomposition is $|X_v| - 1 = 4$.

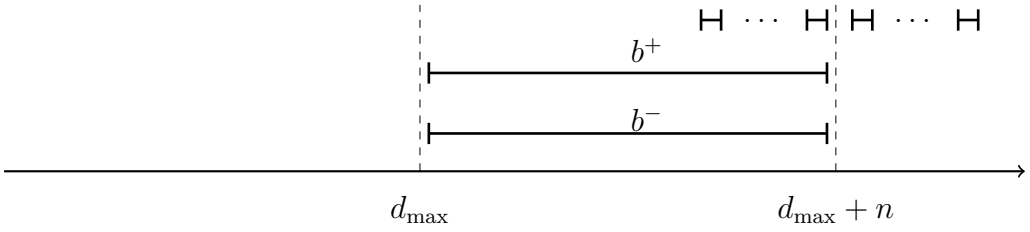
and $c_{u,v}$ are scheduled (as c^+ conflicts with $c_{v,u}$ and c^- conflicts with $c_{u,v}$). It follows that no two of the vertices $v^1_{p_1}, \ldots, v^\ell_{p_\ell}$ are adjacent in G, and so $\{v^1_{p_1}, \ldots, v^\ell_{p_\ell}\}$ is a multicolored independent set in G. □

Lemma 4 *There is a polynomial time reduction from* $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$ *to* $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ *which increases the treewidth by at most 2.*

Proof. Let $\mathcal{I}$ be an instance of $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$ with q days and n clients. We construct an instance $\mathcal{I}'$ (illustrated in Figure 5.7) of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ as follows: Starting with $\mathcal{I}$, we append another q days, add two clients b^- and b^+, and set $k := q$. The jobs of b^- and b^+ have the same processing time and due date, so they cannot be scheduled on the same day. On the first "original" q days, none of the clients from $\mathcal{I}$ intersects with clients b^- and b^+. On the last "additional" q days, no two different clients from $\mathcal{I}$ conflict. For each client c from $\mathcal{I}$, there are exactly k_c days where client c intersects with b^- and b^+. More formally, let $d_{\max}$ be the maximum due date in $\mathcal{I}$. On each day of the $2q$ days in $\mathcal{I}'$, clients b^- and b^+ have processing time n and due date $d_{\max} + n$. On days 1 to q, each client from $\mathcal{I}$ has the same processing time and due date as in $\mathcal{I}$. Let $c_1, \ldots, c_n$ be the clients from $\mathcal{I}$ such that $k_{c_i} \leq k_{c_j}$ for $i < j$ (ordering the clients in this way is not needed for correctness, but makes the illustration in Figure 5.7 nicer). Client c_i has processing time 1 and due date $d_{\max} + i$ on days $q + 1, \ldots, q + k_{c_i}$ and processing time 1 and due date $d_{\max} + n + i$ on days $q + k_{c_i} + 1, \ldots, 2q$.



(a) When $1 \leq i \leq q$, the jobs of the original clients have the same intervals as in the i-th day of $\mathcal{I}$.



(b) When $q < i \leq 2q$ let the jobs of original clients either intersect the jobs of both b^- and b^+ (for $\{\ c \mid i - q \leq k_c\ \}$) *or* intersect no jobs at all (for $\{\ c \mid i - q > k_c\ \}$).

Figure 5.7: An intersection model of the i-th day's jobs.

As we only add two clients (and no new conflicts between clients from $\mathcal{I}$), the treewidth of the overall conflict graph increases by at most two. What remains is to show the correctness

of our construction. Given a solution to $\mathcal{I}'$, we construct a solution to $\mathcal{I}$ by deleting clients b^- and b^+ and days $q+1,\ldots,2q$. Note that on each day, either b^- or b^+ needs to be scheduled as they conflict on each day. Thus, each client c from $\mathcal{I}$ cannot be scheduled on days $q+1,\ldots,q+k_c$, implying that c is scheduled at least k_c times on days $1,\ldots,q$. Thus, we have a solution to $\mathcal{I}$. Conversely, given a solution to $\mathcal{I}$, we construct a solution to $\mathcal{I}'$ as follows: Client b^- is scheduled on the first q days, while b^+ is scheduled on the last q days (days $q+1,\ldots,2q$). On days $1,\ldots,q$, the clients from $\mathcal{I}$ are scheduled as in the solution to $\mathcal{I}$. Furthermore, every client c from $\mathcal{I}$ is scheduled on days $q+k_c+1,\ldots,2q$. Thus, client c is scheduled on $k_c+q-k_c=q$ days, so we get a solution to $\mathcal{I}'$. □

Theorem 5.5 *The* $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ *problem is* NP*-hard for* $\omega \leq 6$.

Proof. In Lemma 3 we described a reduction from the NP-hard Multicolored Independent Set to an instance of $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$ with treewidth at most 4. Lemma 4 describes a reduction from $1 \mid k_j, \text{rep} \mid \min_j \sum_i Z_{i,j}$ to $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ by which the treewidth of the overall conflict graph increases by at most 2. The theorem thus follows. □

We next turn our attention to day-independent due dates, *i.e.*, the case where $d_{i,j} = d_j$. As it turns out, this variant when $k = 1$ is rather similar to the $R \mid\mid \sum_j Z_j$ problem. In $R \mid\mid \sum_j Z_j$ we are given a set of n jobs, where job j has a due date d_j. We are given as well a set of m machines with unrelated speeds, *i.e.*, $p_{i,j}$ is the processing time of job j on machine i. We are then asked whether we can execute *all* jobs in a JIT mode; that is, whether we can schedule all jobs on the set of m machines such that all jobs on each machine are non-conflicting. The $R \mid\mid \sum_j Z_j$ is strongly NP-hard [SV05] and was proven in Theorem 1.1 to be W[1]-hard with respect to the number of machines [Her+24]. Finally, we show that the NP-hardness of $R \mid\mid \sum_j Z_j$ implies NP-hardness for the problem $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$.

5.1.3 The Day-Independent Due Dates Case

Theorem 5.6 *The* $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ *problem is* NP*-hard.*

Proof. We give a reduction from the NP-hard and W[1]-hard problem $R \mid\mid \sum_j Z_j$ to the problem $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ with $k = 1$ [SV05, Her+25]. Given an instance of $R \mid\mid \sum_j Z_j$ we create an instance of $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ as follows: For every job j we create a client c_j, and for every machine i create a day i. The due dates are day-independent. Accordingly, d_j is the common due date of all jobs belonging to client c_j. The processing time of client c_j's job on day i is $p_{i,j}$ (that is, the processing time of job j on machine i). We then ask if there exists a 1-fair schedule for the constructed instance of $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$. We show below that the answer to this question is yes if and only if all jobs of $R \mid\mid \sum_j Z_j$ can be scheduled in a JIT mode.

($\Leftarrow$) Given a 1-fair schedule for $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ we obtain a schedule σ for $R \mid\mid \sum_j Z_j$ by scheduling on machine i all jobs that are scheduled on day i in the instance of $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$. Clearly, all jobs are scheduled in σ as each of the n clients

was served at least once in the 1-fair schedule. There are no conflicts on any machine as there were no conflicts on any of the days in the 1-fair schedule.

($\Rightarrow$) Given a schedule σ for $R \mid\mid \sum_j Z_j$ we obtain a 1-fair schedule for the original instance of $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ by scheduling on day i all jobs that are assigned to machine i of $R \mid\mid \sum_j Z_j$. Clearly, we have created a conflictless and 1-fair schedule as all jobs were conflictlessly scheduled in σ, hence every client is satisfied on exactly one day. □

5.2 Positive Results

5.2.1 One Easy Case for the Fairness Parameter

Returning to the dichotomy of NP-hardness of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ with respect to the parameters k and q, it is clear that the cases where $k = 0$ and $k \geq q$ are straightforward. When $k = 0$, every feasible schedule is k-fair, making it trivially a YES instance. When $k = q$, only conflict-free instances are classified as YES instances, as every job of every client must be scheduled (instances where $k > q$ are trivially NO instances). These observations leave the remaining case of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ when $k = q - 1$, for which we give a polynomial time algorithm using a reduction to 2-SAT.

Theorem 5.7 *The* $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ *problem is solvable in* $O(qn^2 + nq^2)$ *time in case* $k = q - 1$.

Proof. To show the above theorem, we reduce $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ with $k = q-1$ to 2-SAT. With each combination of client j and day i we associate a variable $x_{i,j}$. We then define two sets of clauses. The first set (denoted as the set of *conflicting clauses*) ensures that no two conflicting jobs are scheduled, while the second set (denoted as the set of *validation clauses*) ensures that for each client, not more than one job is discarded. Accordingly, for every pair of clients j_1 and j_2 that are conflicting on day i, we add the conflict clause $(\neg x_{i,j_1} \vee \neg x_{i,j_2})$ that is unsatisfied if both variables are set to `true`. Moreover, for every pair of a client's jobs on two different days, i_1 and i_2 $(1 \leq i_1 < i_2 \leq q)$ we add a validation clause $(x_{i_1,j} \vee x_{i_2,j})$ which cannot be satisfied if both variables are set to `false`.

The size of this construction is polynomial in the size of the input with $O(nq)$ variables. We create $O(nq^2)$ clauses to ensure that at least $q - 1$ jobs of every client are scheduled with $\binom{q}{2}$ many such clauses per client. For every day we keep additional $O(n^2)$ clauses, as many as the edges in the conflict graph, which ensures that no two conflicting jobs are simultaneously scheduled. Hence, there are $O(qn^2 + nq^2)$ clauses in the constructed 2-SAT formula which can be solved in linear time with respect to its size [APT79].

($\Leftarrow$) If there exists a k-fair schedule for $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$, then the 2-SAT is satisfiable. To obtain a satisfying assignment from a feasible schedule, simply set $x_{i,j}$ to `true` if and only if the job of client j is scheduled on day i. In a feasible schedule, no two scheduled jobs are in conflict. It follows that in no conflicting clause both variables are set to true, which means that all conflicting clauses are satisfied by a feasible schedule. If the

schedule is k-fair (for $k = q - 1$), then no client has two or more rejected jobs. All validation clauses are therefore satisfied as every validation clause has at least one variable set to `true`.

($\Rightarrow$) If the 2-SAT formula is satisfiable, then there exists a k-fair schedule. To obtain such a schedule, we schedule client j's job on day i if and only if $x_{i,j}$ is `true` in the assignment. We know that none of these jobs are in conflict, as the conflict clauses are satisfied, and hence the schedule is feasible. We know as well that for every client, at most one job is rejected, otherwise a validation clause will not be satisfied. □

Combining Corollary 2 with Theorem 5.7 completes the proof of Theorem 1.9.

5.2.2 Day-Independent Due Dates and Processing Times

In this section, we will examine the $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ problem in the case where either processing times are day-independent, or where the due dates are day-independent, *i.e.*, problems $1 \mid \text{rep}, p_{i,j} = p_j \mid \min_j \sum_i Z_{i,j}$ and $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$. We begin by completing the proof of Theorem 1.10 by showing that $1 \mid \text{rep}, p_{i,j} = 1 \mid \min_j \sum_i Z_{i,j}$, the special case of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ where all processing times are of unit length, is polynomial-time solvable. This is done via a reduction to the Bipartite Maximum Matching problem.

Theorem 5.8 *The* $1 \mid \text{rep}, p_{i,j} = 1 \mid \min_j \sum_i Z_{i,j}$ *problem is solvable in* $O(n^{1.5} \cdot q^{2.5})$ *time.*

Proof. We show the above theorem with a reduction from $1 \mid \text{rep}, p_{i,j} = 1 \mid \min_j \sum_i Z_{i,j}$ to the Bipartite Maximum Matching problem. Informally, we create a bipartite graph with the set of *job vertices* on one side and the set of *due-date vertices* on the other. To the latter we add a set of *rejection vertices*, $q - k$ per client. A matching in this graph where all job vertices are matched corresponds to a schedule in which every job is either completed at the due date to which it was matched or rejected. The schedule is fair if *all* job vertices are matched since the number of rejection vertices of every client is bounded by $q - k$ (see Figure 5.8 for illustration).

Given an instance of $1 \mid \text{rep}, p_{i,j} = 1 \mid \min_j \sum_i Z_{i,j}$, we create the following instance of the Bipartite Maximum Matching problem:

- For each $j \in \{1, \ldots, n\}$ and each $i \in \{1, \ldots, q\}$, we create a *job* vertex $v_{i,j}$. The set of vertices $V = \{v_{i,j} \mid 1 \leq i \leq q, 1 \leq j \leq n\}$ represents all input jobs of all clients over all days.

- For each $i \in \{1, \ldots, q\}$ and each $d \in \{d_{i,j} \mid 1 \leq j \leq n\}$, we create a *due date* vertex $u_{i,d}$ if no such vertex already exists. The set $U = \{u_{i,d_{i,j}} \mid 1 \leq j \leq n, 1 \leq i \leq q\}$ represents all distinct possible due dates of all input jobs that get scheduled.

- For each $j \in \{1, \ldots, n\}$ and each $\ell \in \{1, \ldots, q - k\}$, we create a *rejection* vertex $r_{\ell,j}$. Matching a job vertex to a vertex of $R = \{r_{\ell,j} \mid 1 \leq j \leq n, 1 \leq \ell \leq q - k\}$ is analogous to rejecting the corresponding job.

The edges of G are constructed as follows. For each $j \in \{1, \ldots, n\}$ and each $i \in \{1, \ldots, q\}$ we connect $v_{i,j}$ to:

- vertices $r_{1,j}, \ldots, r_{q-k,j}$, and
- vertex $u_{i,d_{i,j}}$.

For the correctness, we prove that G has a matching of size nq if and only if there exists a schedule σ where every client is served on at least k days.

($\Rightarrow$) We show how to obtain an nq size matching M given a k-fair schedule σ. Let σ be a schedule where every client is served on at least k days. Consider the job of client j on day i, for some $j \in \{1, \ldots, n\}$ and $i \in \{1, \ldots, q\}$. If the job is scheduled in σ_i and its completion time is $d = d_{i,j}$, then there is an edge $\{v_{i,j}, u_{i,d}\}$ in G. We add this edge to the matching M. Otherwise, the job of client j is not scheduled on day i. Let ℓ denote the number of days prior to i in which the job of client j is not scheduled. Then $\ell < q - k$, as otherwise σ is not a k-fair schedule. We add the edge $\{v_{i,j}, r_{\ell+1,j}\}$ to the matching M. Observe that no pair of edges in M has a common vertex, and hence M is a matching of size nq in G.

($\Leftarrow$) Assume that G contains a matching M of size nq. First note that the fact that G is bipartite with one part being V, and $|V| = qn$, implies that every vertex in the set of job vertices V has to be matched. Let $v_{i,j} \in V$. Then this vertex is either matched to a vertex in U or a vertex in R. We create the schedule σ as follows.

- Suppose that $v_{i,j}$ is matched to some vertex $r_{\ell_0,j_0} \in R$. Note that $j_0 = j$ and $\ell_0 \leq q - k$ by the construction of G. Then we do not schedule the job of client j on day i.
- Suppose that $v_{i,j}$ is matched to some $u_{i_0,d} \in U$. Observe that $i = i_0$ and $d = d_{i,j}$ by the construction of G. We set σ so that the job of client j is scheduled on day i. Note that the fact that $u_{i_0,d}$ cannot be matched to any other vertex in V guarantees that no two jobs can be simultaneously scheduled.

The schedule σ is feasible with no conflicting jobs; otherwise, two job vertices would be matched to the same due date vertex. Assume towards a contradiction that σ is not a k-fair schedule, then there is some client with less than k jobs scheduled over the period of q days. However, this leads to a contradiction as every vertex in V must be matched, either to a vertex in U or R. If client j has less than k scheduled jobs, then less than k vertices of type $v_{i,j}$ are matched to vertices in U for $i \in \{1, \ldots, q\}$, and the rest of these vertices can only be matched to at most $q - k$ vertices of type $r_{i,j}$ in R.

We observe that G has $O(nq)$ vertices and $O(q^2 n)$ edges, and it can be constructed in $O(q^2 n)$ time. Using the algorithm of Hopcroft and Karp [HK73] for the Bipartite Maximum Matching problem, we can solve $1 \mid \text{rep}, p_{i,j} = 1 \mid \min_j \sum_i Z_{i,j}$ in $O(E\sqrt{V})$ where E and V are the number of edges and vertices in the graph. The theorem thus follows. $\square$

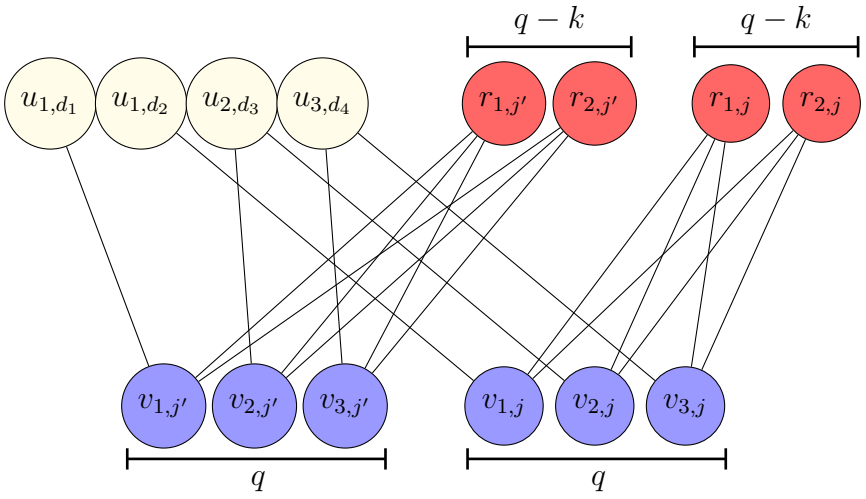


Figure 5.8: The construction of a bipartite graph from an instance of $1 \mid \text{rep}, p_{i,j} = 1 \mid \min_j \sum_i Z_{i,j}$ with $q = 3$ and $k = 1$, where only the relevant vertices for two clients j and j' are depicted. Clients j and j' have different due dates on day 1, and the same due date on days 2 and 3. Both jobs of client j and j' have two rejection vertices with which they can also be potentially matched.

Note that Theorem 5.8 together with Corollary 2 provides the proof of Theorem 1.10.

The problem $R \mid\mid \sum_j Z_j$ admits a polynomial time algorithm for a constant number of machines [SV05]. Note that this result carries over to $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ where $k = 1$, as can be shown by inverting the construction of Theorem 5.6. Our next result generalizes this algorithm for any value of k and proves that $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ is polynomial-time solvable when the number of days q is constant. Recall that $k \leq q$ in all non-trivial instances; hence, we can assume that if q is constant, then k is constant as well.

Theorem 5.9 *The* $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ *problem is solvable in* $O(q^{k+1}n^{q+1})$ *time.*

Proof. Given an instance of $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ we order the clients $\{1, \ldots, n\}$ in $O(n \log n)$ time in non-decreasing order of their due dates. Thus, assume henceforth that we have $d_1 \leq d_2 \leq \cdots \leq d_n$. Let $j^* \in \{1, \ldots, n\}$, and consider a partial schedule $\sigma = (\sigma_1, \ldots, \sigma_q)$ defined on the set of clients $\mathcal{J}_{j*} = \{1, \ldots, j^*\}$ (that is, $\sigma_i \subseteq \mathcal{J}_{j*}$ for each $i \in \{1, \ldots, q\}$). We say that σ is feasible if each σ_i contains no conflicting jobs. Further, we let Σ_{j^*} denote the set of all feasible schedules on $\mathcal{J}_{j*}$ with $\sum_i Z_{i,j} = k$ for every $j \in \mathcal{J}_{j*}$ (that is, *exactly* k of the jobs of each client from $\mathcal{J}_{j*}$ are scheduled). An instance of $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ is clearly a YES instance if and only if $\Sigma_n \neq \emptyset$.

We represent a schedule $\sigma \in \Sigma_{j^*}$ with the state $[j^*, j_1, \ldots, j_q]$ in which j_i represents the index of the last scheduled client on day i ($i \in \{1, \ldots, q\}$). Consider Σ_1. It is obvious that every schedule σ for the client set $\{1\}$ that schedules k jobs of client 1 is feasible and thus contained in Σ_1. Thus, we have $|\Sigma_1| = \binom{q}{k}$, and Σ_1 can be computed in $O(q^k)$ time. We next show how to compute the set Σ_{j^*} from Σ_{j^*-1}.

To compute Σ_{j^*} from Σ_{j^*-1} we iterate over the $\binom{q}{k}$ combinations for scheduling client j^*'s

jobs. Let S be one of the aforementioned combinations with $S \subseteq \{1, \ldots, q\}$ and $|S| = k$. For every schedule $[j^* - 1, j_1, \ldots, j_q] \in \Sigma_{j^*-1}$ we check whether $p_{i,j^*} \leq d_{j^*} - d_{j_i}$ for every $i \in S$, *i.e.*, if the jobs of client j^* can be feasibly scheduled in the partial schedule $[j^* - 1, j_1, \ldots, j_q]$. If yes, we add $[j^*, j_1^{'}, \ldots, j_q^{'}]$ to Σ_{j^*}, where $j_i^{'} = j^*$ if $i \in S$ and $j_i^{'} = j_i$ otherwise.

The correctness of the above algorithm is clear. To analyze its time complexity, note that for any $j^* \in \{1, \ldots, n\}$ there are $O(n^q)$ possible schedules of the form $[j^*, j_1, \ldots, j_q]$, and so the size of each Σ_{j^*} is $O(n^q)$. We compute all schedules in Σ_{j^*} by iterating through all $O(q^k)$ combinations S, and for each such S and every $[j^* - 1, j_1, \ldots, j_q] \in \Sigma_{j^*-1}$ we require $O(k)$ time, which is naturally bounded by $O(q)$. Thus, in total, we compute Σ_{j^*} in $O(q^{k+1}n^q)$ time. As there are n such sets to compute, we get the stated running time of the theorem. □

As a final result for the section, we consider the case where both the processing times and due dates are day-independent, *i.e.*, the $1 \mid \text{rep}, d_{i,j} = d_j, p_{i,j} = p_j \mid \min_j \sum_i Z_{i,j}$ problem. This corresponds to the case that each client has the same time interval for each day, and so the conflict graph is identical for each of the days. Recall that a daily conflict graph is an interval graph. One can compute the chromatic number $\chi(G)$ of any interval graph G with n vertices in $O(n \log n)$ time given its interval representation [Gav72]. In scheduling terms, the chromatic number is the minimum number of machines required to schedule all jobs in a JIT mode. We use this fact to show that $1 \mid \text{rep}, d_{i,j} = d_j, p_{i,j} = p_j \mid \min_j \sum_i Z_{i,j}$ can be solved in $O(n \log n)$ time.

Theorem 5.10 *The* $1 \mid \text{rep}, d_{i,j} = d_j, p_{i,j} = p_j \mid \min_j \sum_i Z_{i,j}$ *problem is solvable in* $O(n \log n)$ *time.*

Proof. Let $G = (V, E)$ be the conflict graph of a given $1 \mid \text{rep}, d_{i,j} = d_j, p_{i,j} = p_j \mid \min_j \sum_i Z_{i,j}$ instance. As mentioned above, we can compute the chromatic number $\chi = \chi(G)$ of G in $O(n \log n)$ time. We show that there exists a feasible k-fair schedule for our given instance if and only if $k \cdot \chi \leq q$.

If $k \cdot \chi \leq q$, then there exist χ independent sets in G whose union is V. We can therefore schedule each of the χ independent sets on a different day. Since there are at least $k \cdot \chi$ days, each of the χ independent sets can be scheduled k times without conflicts. If $k \cdot \chi > q$, then there cannot exist a k-fair schedule due to the fact that interval graphs are perfect graphs, meaning that their chromatic number equals the size of their largest clique [Gol80]. In our case, a clique is a subset of jobs that are mutually in conflict with each other. Clearly, each of the members of the largest clique must be scheduled on a separate day and must be satisfied at least k times. Seeing that we can only schedule one client in the clique per day, if $k \cdot \chi > q$, there are simply not enough days to schedule all clients of the clique. □

Theorem 1.11 now follows directly from Theorem 5.6, Theorem 5.9, and Theorem 5.10.

5.2.3 An Algorithm for Small Number of Days and Small Treewidth

Recall the definition of treewidth Definition 3 and overall conflict graph Definition 2.

Let $\mathcal{I}$ be an instance of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ and let $\mathcal{T}$ be a given nice tree decomposition of the overall conflict graph $G = (\{1, \ldots, n\}, E_1 \cup \cdots \cup E_q)$ of $\mathcal{I}$. To present our algorithm, we will need additional terminology. Let $V = \{1, \ldots, n\}$ denote the set of n clients of $\mathcal{I}$. Recall that a (not necessarily feasible) schedule σ for V is a tuple $\sigma = (\sigma_1, \ldots, \sigma_q)$ of q subsets of V; that is, $\sigma_i \subseteq V$ for each day $i \in \{1, \ldots, q\}$. For a subset of clients $X \subseteq V$, we let $\sigma \cap X$ denote the schedule $(\sigma_1 \cap X, \ldots, \sigma_q \cap X)$.

We will be particularly interested in schedules that are defined only on subsets of clients. Let $X \subseteq V$. We say that a schedule $\sigma = (\sigma_1, \ldots, \sigma_q)$ *is defined on* X if $\sigma_i \subseteq X$ for each day $i \in \{1, \ldots, q\}$. We say that σ is *feasible and fair on* X if σ is feasible, defined on X, and we have $\sum_i Z_{i,j} \geq k$ for each client $j \in X$. That is, a feasible and fair schedule on X is a feasible schedule, which is defined only on the clients of X, and in which each client of X has at least k of its jobs scheduled among all q days. We let $\Sigma(X)$ denote the set of all feasible and fair schedules on X. The following easy lemma will be crucial for our algorithm:

Lemma 11 *Let $X \subseteq V$ be a subset of at most $\omega + 1$ clients. Then $|\Sigma(X)| \leq 2^{(\omega+1)q}$, and we can compute $\Sigma(X)$ in $2^{O(\omega q)}$ time (here and throughout, we assume $\omega \geq 1$; if $\omega = 0$ the conflict graph has no edges and the problem is trivial).*

Let $X \in \mathcal{X}$ be a node of the given nice tree decomposition $\mathcal{T}$. We let $\mathcal{T}_X$ denote the subtree of $\mathcal{T}$ rooted at X, *i.e.*, the subtree of $\mathcal{T}$ that contains X and all of its descendants in $\mathcal{T}$. We use $V(\mathcal{T}_X)$ to denote the set of all clients that are in some node X' of $\mathcal{T}_X$. In other words, $V(\mathcal{T}_X)$ is the set of all clients in X, and all clients in any descendant of X in $\mathcal{T}$. We compute a dynamic programming table T on the nodes of $\mathcal{T}$, in bottom-up fashion, to determine whether the set $\Sigma(V)$ is non-empty. For any node X of $\mathcal{T}$, and any schedule $\sigma \in \Sigma(X)$, we have a corresponding entry $T[X, \sigma]$ in T where the following invariant will hold:

$$T[X, \sigma] = 1 \iff \sigma = \sigma^* \cap X \text{ for some } \sigma^* \in \Sigma(V(\mathcal{T}_X))$$

Equivalently, $T[X, \sigma] = 1$ if and only if σ can be extended to some feasible and fair schedule σ^* for $V(\mathcal{T}_X)$. In this way, there exists a solution to the entire $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ instance if and only if $T[X, \sigma] = 1$ for the root node X of $\mathcal{T}$ and some σ, as $V(\mathcal{T}_X) = V$ in this case. We proceed to show how to compute all four types of nodes of $\mathcal{T}$, starting with leaf nodes, which are the base case of the dynamic program.

Leaf nodes. For leaf node X of $\mathcal{T}$ we have $V(\mathcal{T}_X) = X$ by definition. Thus, we have that $T[X, \sigma] = 1$ for all $\sigma \in \Sigma(X)$. We can therefore compute all entries in T corresponding to X in $2^{O(\omega q)}$ time using Lemma 11.

Forget nodes. Let X be a forget node in $\mathcal{T}$, and let $X' \in \mathcal{X}$ be the child of X in $\mathcal{T}$. Then we have $X = X' \setminus \{j\}$ for some client $j \in \{1, \ldots, n\}$. Given a schedule $\sigma \in \Sigma(X)$, we

compute the entry $T[X, \sigma]$ by

$$T[X,\sigma] = \begin{cases} 1 & T[X',\sigma'] = 1 \text{ for some } \sigma' \text{ with } \sigma = \sigma' \cap X; \\ 0 & \text{otherwise.} \end{cases}$$

Note that this computation can be done in $2^{O(\omega q)}$ time, and its correctness follows from the fact that by definition we have $V(\mathcal{T}_X) = V(\mathcal{T}_{X'})$. Thus, $T[X, \sigma] = 1$ if and only if there exists $\sigma' \in \Sigma(X')$ such that $\sigma' \cap X = \sigma$ and $T[X', \sigma'] = 1$.

Introduce nodes. Let X be an introduce node in $\mathcal{T}$, and let $X' \in \mathcal{X}$ be the child of X in $\mathcal{T}$. Then $X = X' \cup \{j\}$ for some client $j \in \{1, \ldots, n\}$. For a schedule $\sigma \in \Sigma(X)$, we compute the entry $T[X, \sigma]$ by

$$T[X,\sigma] = \begin{cases} 1 & T[X',\sigma'] = 1 \text{ where } \sigma' = \sigma \cap X'; \\ 0 & \text{otherwise.} \end{cases}$$

Observe that as $X = X' \cup \{j\}$ we have $V(\mathcal{T}_X) = V(\mathcal{T}_{X'}) \cup \{j\}$. Moreover, by requirements of a tree decomposition, any client $j_0 \in V(\mathcal{T}_{X'}) \setminus X'$ is not adjacent to client j in the overall conflict graph. This means that any such client j_0 does not have any conflicting jobs with the jobs of client j on any of the q days. Now let $\sigma' = \sigma \cap X'$ and suppose that $T[X', \sigma'] = 1$, and let $\sigma^* = (\sigma_1^*, \ldots, \sigma_q^*) \in \Sigma(V(\mathcal{T}_{X'}))$ be such that $\sigma' = \sigma^* \cap X'$. It follows that $\sigma^{**} = (\sigma_1^* \cup \sigma_1, \ldots, \sigma_q^* \cup \sigma_q)$ is feasible, and so $\sigma^{**} \in \Sigma(V(\mathcal{T}_X))$. Thus, as $\sigma = \sigma^{**} \cap X$, we have $T[X, \sigma] = 1$. Conversely, for any $\sigma \in \Sigma(X)$ for which there exists some $\sigma^{**} \in \Sigma(V(\mathcal{T}_X))$ with $\sigma = \sigma^{**} \cap X$, we have by definition that $\sigma^* = \sigma^{**} \cap V(\mathcal{T}_{X'})$ is in $\Sigma(V(\mathcal{T}_{X'}))$, and that $\sigma^* \cap X' = \sigma \cap X' \in \Sigma(X')$. Therefore, we have $T[X', \sigma \cap X'] = 1$.

Join nodes. Let X be a join node in $\mathcal{T}$, and let $X', X'' \in \mathcal{X}$ be the children of X in $\mathcal{T}$. Then $X = X' = X''$. For a schedule $\sigma \in \Sigma(X)$, we compute the entry $T[X, \sigma]$ by

$$T[X,\sigma] = \begin{cases} 1 & T[X',\sigma] = 1 \text{ and } T[X'',\sigma] = 1; \\ 0 & \text{otherwise.} \end{cases}$$

Note that we have $V(\mathcal{T}_X) = V(\mathcal{T}_{X'}) \cup V(\mathcal{T}_{X''})$. Thus, for any schedule $\sigma^* \in \Sigma(V(\mathcal{T}_X))$ we have $\sigma^* \cap V(\mathcal{T}_{X'}) \in \Sigma(V(\mathcal{T}_{X'}))$ and $\sigma^* \cap V(\mathcal{T}_{X''}) \in \Sigma(V(\mathcal{T}_{X''}))$. Moreover, by requirements of a tree decomposition, any client $j_1 \in V(\mathcal{T}_{X'}) \setminus X$ is not adjacent to any client $j_2 \in V(\mathcal{T}_{X''}) \setminus X$. Thus, for any pair of schedules $\sigma^* = (\sigma_1^*, \ldots, \sigma_q^*) \in \Sigma(V(\mathcal{T}_{X'}))$ and $\sigma^{**} = (\sigma_1^{**}, \ldots, \sigma_q^{**}) \in \Sigma(V(\mathcal{T}_{X''}))$ with $\sigma^* \cap X = \sigma^{**} \cap X = \sigma$, we have that $(\sigma_1^* \cup \sigma_1^{**}, \ldots, \sigma_q^* \cup \sigma_q^{**}) \in \Sigma(V(\mathcal{T}_X))$. It follows that $T[X, \sigma] = 1$ if and only if $T[X', \sigma] = T[X'', \sigma] = 1$ for all schedules $\sigma \in \Sigma(X)$.

In conclusion, it is easy to see from the above that we can compute the value of each entry $T[X, \sigma]$ in our dynamic programming table in $2^{O(\omega q)}$ time. As there are $2^{O(\omega q)} \cdot n$ such entries, it follows that the total running time of our algorithm can be bounded by $2^{O(\omega q)} \cdot n$.

Theorem 5.12 $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ *is solvable in* $2^{O(\omega q)} \cdot n$ *time.*

5.2.4 An Algorithm for Small Number of Clients

We next present an FPT algorithm for $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ with respect to the number of clients parameter n. Our algorithm exploits the fact that when q is large enough (specifically $q \in 2^{\omega(n \log n)}$), some daily conflict graphs are guaranteed to occur multiple times. We provide an ILP formulation where the number of variables is bounded by a function of n, which directly implies an FPT algorithm with respect to n due to Lenstra's Theorem [Len83].

Theorem 5.13 (Lenstra's ILP Algorithm [Len83]) *There exists an algorithm for solving any ILP on* n *variables and* m *constraints in* $n^{O(n)} m^{O(1)}$ *time.*

Theorem 5.14 *The* $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ *problem is solvable in* $2^{2^{O(n \log n)}} \cdot q^{O(1)}$ *time.*

Proof. We prove this by providing an ILP formulation of the problem with $2^{O(n \log n)}$ variables and $2^{O(n \log n)}$ constraints.

Recall that every daily conflict graph G_i is an interval graph with n vertices. The number of distinct interval graphs with n vertices is bounded by $2^{O(n \log n)}$ [BJ25, YP17]. For every such graph type G^t and every subset of vertices $I \subseteq \{1, \ldots, n\}$ that is an independent set in G^t we create an integer variable $x_I^{G^t}$, whose value indicates how many times we schedule the set I on days of type G^t.

We proceed to introduce the three sets of constraints, which will guarantee us a feasible and k-fair solution. First, we introduce a set (5.1) of $2^{O(n \log n)}$ constraints ensuring that all variables are non-negative.

$$\forall G^t, I \mid x_I^{G^t} \geq 0. \tag{5.1}$$

Next, for every G^t we add a constraint to ensure that the total number of independent sets we schedule on days of type G^t is exactly the number of times that G^t appears in the q days.

$$\forall G^t \mid \sum_I x_I^{G^t} = |\{i \in \{1, \ldots, q\} \mid G_i = G^t\}|. \tag{5.2}$$

Lastly, we introduce another n constraints, one for each client to ensure that its containing subsets are scheduled at least k times.

$$\forall j \in \{1, \ldots, n\} : \sum_{I : j \in I} \sum_{G^t} x_I^{G^t} \geq k. \tag{5.3}$$

This completes the description of the ILP. Constructing it requires computing the q daily conflict graphs and writing down the $2^{O(n \log n)}$ variables and constraints, which takes $2^{O(n \log n)} \cdot q^{O(1)}$ time overall. Solving the program via Theorem 5.13 then takes $2^{2^{O(n \log n)}}$ time, and the running time stated in the theorem follows. It remains to prove that the ILP is feasible if and only if the $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ instance admits a feasible and k-fair schedule.

($\Rightarrow$) Let σ be a feasible and k-fair schedule. We compute from σ an assignment to the variables of the ILP such that no constraint is violated. First, set all variables to 0, then iterate on each of the daily schedules in σ. When the set I is scheduled on day i and the conflict graph on that day is G^t, increment the variable $x_I^{G^t}$ by one. Clearly (5.1) is met as σ cannot schedule some job subset a negative number of times. The schedule σ is feasible, hence (5.2) must be upheld, as the scheduled jobs on every day must be independent. Since σ is k-fair, we know that (5.3) is met as every client j is scheduled in at least k days, which means that the sum of $x_I^{G^t}$ must be at least k for every G^t and every I that contains j.

($\Leftarrow$) Any assignment to the variables that satisfies all constraints can be mapped to a feasible and k-fair schedule σ. It is only required that each client has k of its jobs scheduled, regardless of which of the q days this happens. We can therefore define an arbitrary order π over the subsets of V and iterate over the days. Let the i-th day have the conflict graph G^t, and let $x_I^{G^t}$ be the first non-zero variable over the arbitrary order π. The schedule σ will then schedule the jobs associated with I on the i-th day. We then proceed to decrease $x_I^{G^t}$ by one, and repeatedly continue to day $i+1$.

Suppose that σ is infeasible, then there must exist two clients whose jobs intersect on a certain day, let the day type be G^t. This is a contradiction to the fact that the assignment upholds (5.2) as the number of independent sets occurrences on days of type G^t must equal the number of scheduled client-subsets on days of type G^t, and as no client subset can be scheduled a negative number of times it follows that only independent sets are scheduled on σ, and therefore it must be feasible. Suppose that σ is not k-fair, then some client j has less than k jobs scheduled, but this is also a contradiction as we have that (5.3) is met by the assignment. It follows that σ is k-fair. □

5.3 Discussion

In this chapter, we studied a new problem $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ that models repetitive interval scheduling on a single machine when fairness amongst the different clients is the main objective. We provided a thorough complexity analysis for the problem, and identified for which values of k and q, the problem is NP-hard. We also considered the special cases of day-independent processing times or due dates, as well as several different parameterizations for the problem through the lens of parameterized complexity.

Generalizations

In $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ all clients have the same fairness parameter, they all submit a single job on each day, and we have a single machine to process all these jobs on every day. Our results easily carry over to some natural generalizations of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ that we discuss below:

- *Client-dependent fairness parameters:* The case where each client j has its own fairness

parameter k_j is discussed in Section 5.1.2, where we show a reduction from this variant to $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ in Lemma 4.

- *Clients do not require service on some days:* Consider the variant of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ where any client j may not have a job to be processed on some of the days $i \in \{1, \ldots, q\}$. We can reduce this generalized variant to $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ by adding $\lceil \frac{q}{k} \rceil$ auxiliary clients to the set of clients. We also extend the number of days by $k(\lceil \frac{q}{k} \rceil - \frac{q}{k})$ days. We set the jobs of the auxiliary clients so that they are all mutually conflicting on all days. We then need to add jobs to each of the n original clients so that every client has a job on every day. On each of the original q days, if some client j does not have a job on some day i, we create a job for this client that conflicts only with the jobs of auxiliary clients. On the additional $k(\lceil \frac{q}{k} \rceil - \frac{q}{k})$ days, we set the jobs of the original n clients to conflict with the jobs of all auxiliary clients. See Figure 5.9 for an illustration. It is easy to verify that an instance of the generalized variant has a solution of value k if and only if the constructed instance of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ has a solution of the same value.

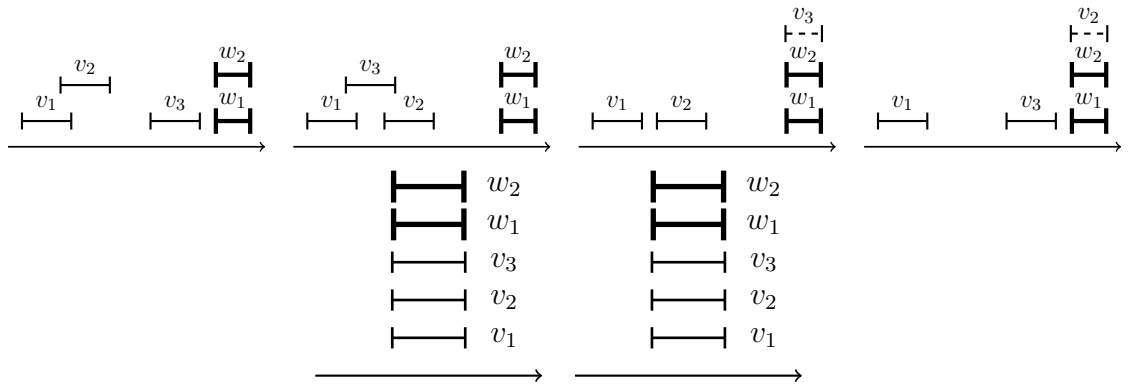


Figure 5.9: The construction of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ from the general case by which $q = 4$ and $k = 3$. The reduction adds $3(\lceil \frac{4}{3} \rceil - \frac{4}{3}) = 2$ additional days and $\lceil \frac{4}{3} \rceil = 2$ auxiliary clients w_1 and w_2. The top figure depicts the original days, while the bottom figure depicts the additional days. The jobs in **bold** are jobs belonging to the auxiliary clients, and the dashed jobs are new jobs that are added for the original clients and on the original days.

- *Agreeable due dates:* An instance of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ is said to have agreeable due dates if $d_{i,j} \leq d_{i,j+1}$ for all $i \in \{1, \ldots, q\}$ and $j \in \{1, \ldots, n-1\}$. Note that the case of day-independent due dates is a special case of agreeable due dates. Agreeable conditions are widely used in the scheduling literature to derive polynomial time algorithms for NP-hard problems [Bam+12, Gil+21, MY13, LNC07, SG22]. We can create from every *agreeable due dates* instance an equivalent instance by which the due dates are *day-independent*. To do so, for each day i and job j, let ℓ be the smallest index in $\{1, \ldots, j-1\}$ of a job that is in conflict with job j on day i, and let $\ell = j$ if there is no such job. We set the new due date to $d'_{i,j} = j$ and the new processing time to $p'_{i,j} = j - \ell + 1$, so that the new interval of job j on day i is $[\ell - 1, j)$.

Since the due dates are agreeable, the jobs among $\{1, \ldots, j-1\}$ that are in conflict with job j on day i are exactly $\ell, \ldots, j-1$, and these are precisely the jobs among $\{1, \ldots, j-1\}$ whose new intervals intersect $[\ell - 1, j)$. Hence, we obtain an equivalent instance of $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ in this way.

- *Multiple Machines:* Most of our algorithms also extend naturally to the case where there are m identical machines on each day to process the clients' jobs: The running time of the algorithm in Theorem 5.14 increases to $2^{2^{O(mn\log(mn))}} \cdot q^{O(1)}$ by replacing variables $x_I^{G^t}$ with variables of the form $x_{I_1,\ldots,I_m}^{G^t}$. We get the running time $(m+1)^{O(\omega q)} \cdot n$ by extending the algorithm in Theorem 5.12 so that it considers the schedules on m machines on each day for each node of the tree decomposition. For the case of day-independent processing times and due dates, we can easily reduce to $1 \mid \text{rep}, d_{i,j} = d_j, p_{i,j} = p_j \mid \min_j \sum_i Z_{i,j}$ with mq days, and the algorithms of Theorem 5.8 and Theorem 5.9 can be extended to algorithms with running times of $O(\sqrt{m}n^{1.5}q^{2.5} + (nqm)^{1.5})$ and $O((qm)^{k+1}n^{qm+1})$, respectively. A notable exception is Theorem 5.7 and the case of $k = q - 1$, which does not naturally extend to the multiple machine case.

Open problems

Despite our thorough investigation into the complexity of $1 \mid \text{rep} \mid \min_j \sum_i Z_{i,j}$ there are several interesting open questions left by our work. Below we give only a partial list:

Open Question 4 *Can* $1 \mid \text{rep}, p_{i,j} = 1 \mid \min_j \sum_i Z_{i,j}$ *be solved in* $O(qn)$ *time?*

Open Question 5 *Does* $1 \mid \text{rep}, d_{i,j} = d_j \mid \min_j \sum_i Z_{i,j}$ *admit an algorithm running in* $n^{o(q)}$ *time?*

Note that Theorem 5.1 excludes an FPT algorithm for q.

Open Question 6 *Can the m-machine case be solved in polynomial-time for* $k = q - 1$*?*

Naturally, apart from the Just-in-Time objective $\sum_i Z_{i,j}$, there are several other natural objectives to explore in the context of fair repetitive scheduling, and this work should serve as an encouragement to explore this interesting new area.

Conclusion

"Having no problems is the biggest problem of all."

— Taiichi Ohno, *Toyota Motor Corporation*

At its core, the Just-in-Time philosophy is not merely about delivery time, but about identifying and removing waste from the production process [Pie14, Rad+24, Kat17]. Rather than using the available resources to maximize production volume, it aims to deliver the *required* volume using minimal resources. Toyota built its entire production system around this principle. As Toyota Production System's chief architect Taiichi Ohno famously observed, waste remains invisible until someone names it. A process often runs inefficiently for years, not because it is unfixable, but because the location of the inefficiency has yet to be identified.

Algorithmic efficiency can as well be viewed through this same lens. The term "efficient" is normally used as shorthand for polynomial time, but efficiency, taken literally, is a comparative notion. An algorithm is more efficient than another if it achieves the same result using fewer resources. An exponential-time algorithm then leaves a real question open, whether its exponential dependence could be avoided by a less wasteful algorithm, or whether some exponential dependence is inherent to the problem itself. An NP-hardness classification removes this uncertainty. It tells us that, unless P = NP, some superpolynomial dependence is inherent to the problem, rather than a deficiency of the algorithm. In this sense, NP-hardness identifies and eliminates one form of invisible computational waste — the search for polynomial-time algorithms where none are expected to exist.

Yet NP-hardness does not end the search for better algorithms. It tells us that, for some problems, exponential-time dependence is likely unavoidable, but it does not determine "exponential in what". To refine NP-hardness results, parameterized complexity considers the structural properties of computational problems, in addition to their input size. Every unclassified parameterization leaves open the possibility of hidden computational waste. Its classification may reveal that the exponential dependence can be confined to a smaller structural property, leaving the remaining input size to be handled efficiently. Computational hardness has a boundary, and parameterized complexity assists us in charting it.

In scheduling, such parameterized inefficiency went largely unexplored for decades. Scheduling theory predates NP-completeness, and early scheduling researchers even developed a notion of hardness independently, in parallel with Cook, Karp, and Levin [LR72, LSV23]. Scheduling problems were also among the first to be proved NP-complete, with $1||\sum w_j U_j$ appearing as Karp's 19th problem under the name Job Sequencing

[Kar72]. By the end of the 1970s, scheduling constituted an entire category in Garey and Johnson's catalogue [GJ79]. The parameterized study of scheduling developed much later, through early work by Bodlaender and Fellows [BF95] and Fellows and McCartin [FM03]. Marx's challenge at the 2011 Dagstuhl seminar [Mar11] helped stimulate further work on classical scheduling problems [MW14], including Karp's own Job Sequencing [Her+18]. Mnich and van Bevern subsequently collected fifteen open problems [MB18], giving this research effort a shared agenda. This thesis forms part of the response to that call.

Within scheduling, interval scheduling has received particularly little parameterized attention. Out of the fifteen open problems Mnich and van Bevern collected, only one — Open Problem 8 — belongs to interval scheduling [MB18]. We resolve this question negatively in Chapter 3. We also extend the parameterized study of interval scheduling to two new models, one generalizing existing Just-in-Time flow shop models, and the other applying the repetitive scheduling framework to the Just-in-Time objective.

Open Problem 8 asked whether interval scheduling on unrelated machines admits an FPT algorithm parameterized by the number of machines m. We answer this negatively in Chapter 3 by studying the special case of eligible machines, where each job may run only on a given subset of the machines. Arkin and Silverberg [AS87] were the first to study this special case, proving that it is NP-hard and giving an algorithm that runs in $n^{O(m)}$ time. In almost forty years, this running time has not been asymptotically improved. We showed that $P|M_j|\sum w_j Z_j$ is W[1]-hard when parameterized by m, ruling out fixed-parameter tractability unless FPT = W[1]. Moreover, assuming the Exponential Time Hypothesis, our reduction rules out algorithms running in $n^{o(\sqrt{m})}$ time, leaving a gap between $O(\sqrt{m})$ and $O(m)$ in the exponent. Bounding the interval lengths alone does not help either, as the problem remains NP-hard even when $p_{\max}$ is a constant. Combining the two parameters, however, does help, as the problem is fixed-parameter tractable when parameterized by $m + p_{\max}$. These results redirect the search for better algorithms toward restrictions beyond the number of machines, such as the sizes and overlap of the eligibility sets, or nesting restrictions studied in other scheduling models [CA04].

The second problem generalizes interval scheduling to a flow shop, where each job must pass through a preprocessing stage before its interval begins (Chapter 4). Choi and Yoon [CY07] showed that the Just-in-Time objective in a flow shop is strongly NP-hard with three stages, even in the unweighted case, which rules out the number of stages as a parameter, and NP-hard only in the ordinary sense with two stages, for which Shabtay and Bensoussan [SB12] later gave a pseudo-polynomial time algorithm. Our generalization lets the second stage be served by m parallel machines, making the number of machines the natural parameter. The problem is solvable in pseudo-polynomial time for every constant m, with m in the exponent of the running time. It is strongly NP-hard when m is part of the input and W[2]-hard when parameterized by m, so m cannot be moved out of the exponent unless FPT = W[2]. Bounding the processing times helps once more: the problem is solvable in pseudo-polynomial time whenever the maximum second-stage processing time $q_{\max}$ is a constant, regardless of m, and likewise when the chromatic number of the second-stage conflict graph is small. The hardest instances are therefore those where m is large and $q_{\max}$

is unbounded.

The third problem brings the Just-in-Time objective into the fair repetitive scheduling framework, where the same clients are served day after day and each must receive sufficiently many Just-in-Time jobs over the entire horizon (Chapter 5). This problem is hard even under severe restrictions: it is NP-hard on a single machine with only three days, a fairness parameter of one, and all processing times equal, and it remains NP-hard when the conflict graph has constant treewidth, a parameter that renders a great many hard problems tractable. With respect to the fairness parameter itself, there is essentially a single tractable case, the one where every client must be served on all days but one. The problem is, however, fixed-parameter tractable when parameterized by the treewidth combined with the number of days. Since the hardness for constant treewidth relies on many days, the complementary regime is a long horizon with few clients, and here the problem is fixed-parameter tractable when parameterized by the number of clients alone, regardless of the number of days. Finally, restricting the input rather than a parameter, the problem is polynomial-time solvable for unit processing times and, when due dates are day-independent, for a constant number of days or for day-independent processing times.

Each tractable region identified in these three chapters does double duty, yielding an algorithm and confining the hardness to what is left, and the boundary it traces does not stop at fixed intervals. Interval scheduling is the special case of non-preemptive scheduling with release times and deadlines in which every job's time window $[r_j, d_j)$ has length exactly p_j. Interval scheduling is also the special case of non-preemptive scheduling with release times and deadlines in which every job's time window $[r_j, d_j)$ has length exactly p_j. Letting the windows grow recovers the general problem, and already on a single machine this yields $1|r_j|\sum w_j U_j$, whose parameterized complexity with respect to $p_{\max}$ is Open Problem 3 of the same list [MB18]. The *slack* $d_j - r_j - p_j$ [Cie+04] measures how far an instance is from the interval case, which makes it a natural parameter for lifting any interval scheduling algorithm to release times and deadlines. Indeed, deciding whether all jobs can be scheduled on m machines is fixed-parameter tractable with respect to m combined with the maximum slack [BNS17], although the problem is strongly NP-hard for every constant slack of at least two when m is part of the input [Cie+04]. In this sense, every interval scheduling problem, including those studied in this thesis, generalizes through slack to its non-preemptive counterpart with release times.

In Ohno's sense, then, interval scheduling is in good shape, for it has no shortage of problems, and slack is only one of the directions, alongside online arrivals, uncertain processing times, and additional resources, in which they extend. Every classification gives the next investigation a more precise starting point, identifying which restrictions are sufficient for tractability and which still leave the problem hard. This gradual refinement is what Toyota called *Kaizen* [SS09], continuous improvement through small, incremental changes, applied here not to a factory floor but to our understanding of where a problem's difficulty actually lives. The problems left open give this work its direction. To borrow Ohno's words once more, they are treasures.

Bibliography

[Her+24] Danny Hermelin, Yuval Itzhaki, Hendrik Molter, and Dvir Shabtay. "On the Parameterized Complexity of Interval Scheduling with Eligible Machine Sets". In: *Journal of Computer and System Sciences* 144 (2024), p. 103533. DOI: 10.1016/j.jcss.2024.103533.

[Hee+25] Klaus Heeger, Danny Hermelin, Yuval Itzhaki, Hendrik Molter, and Dvir Shabtay. "Fair Repetitive Interval Scheduling". In: *Algorithmica* 87 (2025), pp. 1340–1368. DOI: 10.1007/s00453-025-01322-y.

[Hee+26] Klaus Heeger, Danny Hermelin, Yuval Itzhaki, Baruch Schieber, and Dvir Shabtay. "Just-in-Time Scheduling in Two-Stage Flexible Flow Shops". In: *European Journal of Operational Research* (2026). DOI: 10.1016/j.ejor.2026.02.017.

[LC01] Chung-Yee Lee and Zhi-Long Chen. "Machine Scheduling with Transportation Considerations". In: *Journal of Scheduling* 4.1 (2001), pp. 3–24. DOI: 10.1002/1099-1425(200101/02)4:1%3C3::AID-JOS57%3E3.0.CO;2-D.

[Pin05] Michael Pinedo. *Planning and Scheduling in Manufacturing and Services*. Springer, 2005. DOI: 10.1007/b139030.

[ZP14] Christos Zacharias and Michael Pinedo. "Appointment Scheduling with No-Shows and Overbooking". In: *Production and Operations Management* 23.5 (2014), pp. 788–801. DOI: 10.1111/poms.12065.

[PS85] James L. Peterson and Abraham Silberschatz. *Operating System Concepts*. Addison-Wesley Longman Publishing Co., Inc., 1985.

[AK01] Hojjat Adeli and Asim Karim. *Construction Scheduling, Cost Optimization and Management*. CRC Press, 2001. DOI: 10.1201/9781482267686.

[Har97] Sönke Hartmann. *Scheduling Medical Research Experiments - An Application of Project Scheduling Methods*. Tech. rep. Manuskripte aus den Instituten für Betriebswirtschaftslehre der Universität Kiel, 1997. URL: https://hdl.handle.net/10419/177313.

[CDB10] Brecht Cardoen, Erik Demeulemeester, and Jeroen Beliën. "Operating Room Planning and Scheduling: A Literature Review". In: *European Journal of Operational Research* 201.3 (2010), pp. 921–932. DOI: 10.1016/j.ejor.2009.04.011.

[DT22] Giacomo Da Col and Erich C. Teppan. "Industrial-Size Job Shop Scheduling with Constraint Programming". In: *Operations Research Perspectives* 9 (2022), p. 100249. DOI: 10.1016/j.orp.2022.100249.

[Mal+59] Donald G. Malcolm, John H. Roseboom, Charles E. Clark, and Willard Fazar. "Application of a Technique for Research and Development Program Evaluation". In: *Operations Research* 7.5 (1959), pp. 646–669. DOI: 10.1287/opre.7.5.646.

[KW59] James E. Kelley Jr. and Morgan R. Walker. "Critical-Path Planning and Scheduling". In: *Papers Presented at the December 1-3, 1959, Eastern Joint IRE-AIEE-ACM Computer Conference*. 1959, pp. 160–173. DOI: 10.1145/1460299.1460318.

[CJ09] Jaewoo Chung and Jaejin Jang. "A WIP Balancing Procedure for Throughput Maximization in Semiconductor Fabrication". In: *IEEE Transactions on Semiconductor Manufacturing* 22.3 (2009), pp. 381–390. DOI: 10.1109/TSM.2009.2017666.

[Wei02] Lawrence M. Wein. "Scheduling Semiconductor Wafer Fabrication". In: *IEEE Transactions on Semiconductor Manufacturing* 1.3 (2002), pp. 115–130. DOI: 10.1109/66.4384.

[AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009. DOI: 10.1017/CBO9780511804090.

[Hoc97] Dorit S. Hochbaum. "Approximation Algorithms for NP-hard Problems". In: *ACM SIGACT News* 28.2 (1997), pp. 40–52. DOI: 10.1145/261342.571216.

[Kar91] Richard M. Karp. "An Introduction to Randomized Algorithms". In: *Discrete Applied Mathematics* 34.1-3 (1991), pp. 165–201. DOI: 10.1016/0166-218X(91)90086-C.

[Nie06] Rolf Niedermeier. *Invitation to Fixed-Parameter Algorithms*. Oxford University Press, 2006. DOI: 10.1093/acprof:oso/9780198566076.001.0001.

[DF13] Rodney G. Downey and Michael R. Fellows. *Fundamentals of Parameterized Complexity*. Springer, 2013. DOI: 10.1007/978-1-4471-5559-1.

[Cyg+15] Marek Cygan, Fedor V. Fomin, Łukasz Kowalik, Daniel Lokshtanov, Dániel Marx, Marcin Pilipczuk, Michał Pilipczuk, and Saket Saurabh. *Parameterized Algorithms*. Springer, 2015. DOI: 10.1007/978-3-319-21275-3.

[Lok+20] Daniel Lokshtanov, Maadapuzhi S. Ramanujan, Saket Saurab, and Meirav Zehavi. "Parameterized Complexity and Approximability of Directed Odd Cycle Transversal". In: *Proceedings of the 14th Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM. 2020, pp. 2181–2200. DOI: 10.1137/1.9781611975994.134.

[Cha+17] Parinya Chalermsook, Marek Cygan, Guy Kortsarz, Bundit Laekhanukit, Pasin Manurangsi, Danupon Nanongkai, and Luca Trevisan. "From Gap-ETH to FPT-Inapproximability: Clique, Dominating Set, and More". In: *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE. 2017, pp. 743–754. DOI: 10.1109/FOCS.2017.74.

[AYZ95] Noga Alon, Raphael Yuster, and Uri Zwick. "Color-Coding". In: *Journal of the ACM* 42.4 (1995), pp. 844–856. DOI: 10.1145/210332.210337.

[LRS18] Daniel Lokshtanov, Maadapuzhi Sridharan Ramanujan, and Saket Saurabh. "Linear Time Parameterized Algorithms for Subset Feedback Vertex Set". In: *ACM Transactions on Algorithms* 14.1 (2018), pp. 1–37. DOI: 10.1145/3155299.

[GMN17] Archontia C. Giannopoulou, George B. Mertzios, and Rolf Niedermeier. "Polynomial Fixed-Parameter Algorithms: A Case Study for Longest Path on Interval Graphs". In: *Theoretical Computer Science* 689 (2017), pp. 67–95. DOI: 10.1016/j.tcs.2017.05.017.

[AWW16] Amir Abboud, Virginia Vassilevska Williams, and Joshua Wang. "Approximation and Fixed Parameter Subquadratic Algorithms for Radius and Diameter in Sparse Graphs". In: *Proceedings of the 27th Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM. 2016, pp. 377–391. DOI: 10.1137/1.9781611974331.ch28.

[FC22] Henry Ford and Samuel Crowther. *My Life and Work*. Garden City, NY: Doubleday, Page and Company, 1922.

[Pet02] Peter B. Petersen. "The Misplaced Origin of Just-in-Time Production Methods". In: *Management Decision* 40.1 (2002), pp. 82–88. DOI: 10.5465/apbpp.1999.27614354.

[WJR90] James P. Womack, Daniel T. Jones, and Daniel Roos. *The Machine That Changed the World*. Simon and Schuster, 1990. DOI: 10.21061/jte.v5i2.a.7.

[Toy12] Toyota Motor Corporation. *Production Overview – 75 Years of Toyota*. Toyota Motor Corporation. 2012. URL: https://www.toyota-global.com/company/history_of_toyota/75years/data/automotive_business/production/production/overview/index.html.

[OB88] Taiichi Ohno and Norman Bodek. *Toyota Production System: Beyond Large-Scale Production*. New York: Productivity Press, 1988. DOI: 10.4324/9780429273018.

[Sug+77] Yutaka Sugimori, Ken Kusunoki, Fujio Cho, and Sjtijopr Uchikawa. "Toyota Production System and Kanban System Materialization of Just-in-Time and Respect-for-Human System". In: *International Journal of Production Research* 15.6 (1977), pp. 553–564. DOI: 10.1080/00207547708943149.

[Mon83] Yasuhiro Monden. *Toyota Production System - An Integrated Approach to Just-in-Time*. Industrial Engineering and Management Press, 1983. DOI: 10.1007/978-1-4615-9714-8.

[Näs08] Dag Näslund. "Lean, Six Sigma and Lean Sigma: Fads or Real Process Improvement Methods?" In: *Business Process Management Journal* 14.3 (2008), pp. 269–287. DOI: 10.1108/14637150810876634.

[IL89] Jin H. Im and Sang M. Lee. "Implementation of Just-in-Time Systems in US Manufacturing Firms". In: *International Journal of Operations & Production Management* 9.1 (Jan. 1989), pp. 5–14. DOI: 10.1108/EUM0000000001213.

[IM93] R. Anthony Inman and Satish Mehra. "Financial Justification of JIT Implementation". In: *International Journal of Operations & Production Management* 13.4 (Apr. 1993), pp. 32–39. DOI: 10.1108/01443579310027734.

[Inm+11] R. Anthony Inman, R. Samuel Sale, Kenneth W. Green, and Dwayne Whitten. "Agile Manufacturing: Relation to JIT, Operational Performance and Firm Performance". In: *Journal of Operations Management* 29.4 (2011), pp. 343–355. DOI: 10.1016/j.jom.2010.06.001.

[CFK00] Jeffrey L. Callen, Chris Fader, and Itzhak Krinsky. "Just-in-Time: A Cross-Sectional Plant Analysis". In: *International Journal of Production Economics* 63.3 (2000), pp. 277–301. DOI: 10.1016/S0925-5273(99)00025-0.

[Kub93] Wieslaw Kubiak. "Minimizing Variation of Production Rates in Just-in-Time Systems: A Survey". In: *European Journal of Operational Research* 66.3 (1993), pp. 259–271. DOI: 10.1016/0377-2217(93)90215-9.

[Sch82] Richard Schonberger. *Japanese Manufacturing Techniques: Nine Hidden Lessons in Simplicity*. Simon and Schuster, 1982.

[RR11] Roger Rios and Yasmín A. Ríos-Solís. *Just-in-Time Systems*. Vol. 60. Springer, 2011. DOI: 10.1007/978-1-4614-1123-9.

[ABF14] Enrico Angelelli, Nicola Bianchessi, and Carlo Filippi. "Optimal Interval Scheduling with a Resource Constraint". In: *Computers & Operations Research* 51 (2014), pp. 268–281. DOI: 10.1016/j.cor.2014.06.002.

[BS90] Kenneth R. Baker and Gary D. Scudder. "Sequencing with Earliness and Tardiness Penalties: A Review". In: *Operations Research* 38.1 (1990), pp. 22–36. DOI: 10.1287/opre.38.1.22.

[Bev+15] René van Bevern, Matthias Mnich, Rolf Niedermeier, and Mathias Weller. "Interval Scheduling and Colorful Independent Sets". In: *Journal of Scheduling* 18.5 (2015), pp. 449–469. DOI: 10.1007/s10951-014-0398-5.

[BE96] Khalid I. Bouzina and Hamilton Emmons. "Interval Scheduling on Identical Machines". In: *Journal of Global Optimization* 9.3 (1996), pp. 379–393. DOI: 10.1007/BF00121680.

[HLV02] Kunihiko Hiraishi, Eugene Levner, and Milan Vlach. "Scheduling of Parallel Identical Machines to Maximize the Weighted Number of Just-in-Time Jobs". In: *Computers & Operations Research* 29.7 (2002), pp. 841–848. DOI: 10.1016/S0305-0548(00)00086-1.

[LM96] Avital Lann and Gur Mosheiov. "Single Machine Scheduling to Minimize the Number of Early and Tardy Jobs". In: *Computers & Operations Research* 23.8 (1996), pp. 769–781. DOI: 10.1016/0305-0548(95)00078-X.

[Sha12] Dvir Shabtay. "The Just-in-Time Scheduling Problem in a Flow Shop Scheduling System". In: *European Journal of Operational Research* 216.3 (2012), pp. 521–532. DOI: 10.1016/j.ejor.2011.07.053.

[Spi99] Frits C. R. Spieksma. "On the Approximability of an Interval Scheduling Problem". In: *Journal of Scheduling* 2.5 (1999), pp. 215–227. DOI: 10.1002/(SICI)1099-1425(199909/10)2:5<215::AID-JOS27>3.0.CO;2-Y.

[SV05] Shao Chin Sung and Milan Vlach. "Maximizing Weighted Number of Just-in-Time Jobs on Unrelated Parallel Machines". In: *Journal of Scheduling* 8.5 (2005), pp. 453–460. DOI: 10.1007/s10951-005-2863-7.

[Yan+21] Jie Yang, Hongming Xie, Guangsheng Yu, and Mingyu Liu. "Achieving a Just-in-Time Supply Chain: The Role of Supply Chain Intelligence". In: *International Journal of Production Economics* 231 (2021), p. 107878. DOI: 10.1016/j.ijpe.2020.107878.

[Nug+20] Agustinus Nugroho, Budiman Christiananta, Fenika Wulani, and Ikbar Pratama. "Exploring the Association Among Just-in-Time, Total Quality and Supply Chain Management Influence on Firm Performance: Evidence from Indonesia". In: *Journal of Supply Chain Management* 9.2 (2020), p. 920. URL: http://dspace.uc.ac.id/handle/123456789/5333.

[Cho+23] Thomas Y. Choi, Torbjørn H. Netland, Nada Sanders, ManMohan S. Sodhi, and Stephan M. Wagner. "Just-in-Time for Supply Chains in Turbulent Times". In: *Production and Operations Management* 32.7 (2023), pp. 2331–2340. DOI: 10.1111/poms.13979.

[CYC10] Hing Kai Chan, Shizhao Yin, and Felix T. S. Chan. "Implementing Just-in-Time Philosophy to Reverse Logistics Systems: A Review". In: *International Journal of Production Research* 48.21 (2010), pp. 6293–6313. DOI: 10.1080/00207540903225213.

[Agy+21] Yaw Agyabeng-Mensah, Ebenezer Afum, Carin Agnikpe, Jiaxin Cai, Esther Ahenkorah, and Essel Dacosta. "Exploring the Mediating Influences of Total Quality Management and Just-in-Time Between Green Supply Chain Practices and Performance". In: *Journal of Manufacturing Technology Management* 32.1 (2021), pp. 156–175. DOI: 10.1108/JMTM-03-2020-0086.

[LC16] Kee-hung Lai and T. C. Edwin Cheng. *Just-in-Time Logistics*. Routledge, 2016.

[HZ21] Mohamed Hussein and Tarek Zayed. "Critical Factors for Successful Implementation of Just-in-Time Concept in Modular Integrated Construction: A Systematic Review and Meta-Analysis". In: *Journal of Cleaner Production* 284 (2021), p. 124716. DOI: 10.1016/j.jclepro.2020.124716.

[ATN20] Saman Abbasi, Katayoon Taghizade, and Esmatullah Noorzai. "BIM-Based Combination of Takt Time and Discrete Event Simulation for Implementing Just-in-Time in Construction Scheduling under Constraints". In: *Journal of Construction Engineering and Management* 146.12 (2020), p. 04020143. DOI: 10.1061/(ASCE)CO.1943-7862.0001940.

[Lyu+20] Zhongyuan Lyu, Peng Lin, Daqiang Guo, and George Q. Huang. "Towards Zero-Warehousing Smart Manufacturing from Zero-Inventory Just-in-Time Production". In: *Robotics and Computer-Integrated Manufacturing* 64 (2020), p. 101932. DOI: 10.1016/j.rcim.2020.101932.

[BLC19] Flora Bamana, Nadia Lehoux, and Caroline Cloutier. "Simulation of a Construction Project: Assessing Impact of Just-in-Time and Lean Principles". In: *Journal of Construction Engineering and Management* 145.5 (2019), p. 05019005. DOI: 10.1061/(ASCE)CO.1943-7862.0001654.

[LTC96] Shu Li, Tom Tang, and Donald W. Collins. "Minimum Inventory Variability Schedule with Applications in Semiconductor Fabrication". In: *IEEE Transactions on Semiconductor Manufacturing* 9.1 (1996), pp. 145–149. DOI: 10.1109/66.484296.

[Pin+18] José Luís Quesado Pinto, João Carlos O. Matias, Carina Pimentel, Susana Garrido Azevedo, and Kannan Govindan. "Just-in-Time Factory". In: *Management for Professionals* (2018). DOI: 10.1007/978-3-319-77016-1.

[Kum+22] Naveen Kumar, Syed Shahzeb Hasan, Kunal Srivastava, Rayhan Akhtar, Rakesh Kumar Yadav, and Vikas Kumar Choubey. "Lean Manufacturing Techniques and Its Implementation: A Review". In: *Materials Today: Proceedings* 64 (2022), pp. 1188–1192. DOI: 10.1016/j.matpr.2022.03.481.

[BM99] Paolo Brandimarte and M. Maiocco. "Job Shop Scheduling with a Non-Regular Objective: A Comparison of Neighbourhood Structures Based on a Sequencing/Timing Decomposition". In: *International Journal of Production Research* 37.8 (1999), pp. 1697–1715. DOI: 10.1080/002075499190969.

[RS93] David J. Robb and Edward A. Silver. "Scheduling in a Management Context: Uncertain Processing Times and Non-Regular Performance Measures". In: *Decision Sciences* 24.6 (1993), pp. 1085–1108. DOI: 10.1111/j.1540-5915.1993.tb00505.x.

[LW04] Volker Lauff and Frank Werner. "Scheduling with Common Due Date, Earliness and Tardiness Penalties for Multimachine Problems: A Survey". In: *Mathematical and Computer Modelling* 40.5-6 (2004), pp. 637–655. DOI: 10.1016/j.mcm.2003.05.019.

[HP91a] Nicholas G. Hall and Marc E. Posner. “Earliness-Tardiness Scheduling Problems, I: Weighted Deviation of Completion Times About a Common Due Date”. In: *Operations Research* 39.5 (1991), pp. 836–846. DOI: 10.1287/opre.39.5.836.

[HP91b] Nicholas G. Hall and Marc E. Posner. “Earliness–Tardiness Scheduling Problems, II: Deviation of Completion Times About a Restrictive Common Due Date”. In: *Operations Research* 39.5 (1991), pp. 847–856. DOI: 10.1287/opre.39.5.847.

[LKB77] Jan Karel Lenstra, Alexander H. G. Rinnooy Kan, and Peter Brucker. “Complexity of Machine Scheduling Problems”. In: *Annals of Discrete Mathematics*. Vol. 1. Elsevier, 1977, pp. 343–362. DOI: 10.1016/S0167-5060(08)70743-X.

[Law77] Eugene L. Lawler. “A “Pseudopolynomial” Algorithm for Sequencing Jobs to Minimize Total Tardiness”. In: *Annals of Discrete Mathematics*. Vol. 1. Elsevier, 1977, pp. 331–342. DOI: 10.1016/S0167-5060(08)70742-8.

[DL90] Jianzhong Du and Joseph Y. T. Leung. “Minimizing Total Tardiness on One Machine is NP-hard”. In: *Mathematics of Operations Research* 15.3 (1990), pp. 483–495. DOI: 10.1287/moor.15.3.483.

[GJ78] Michael R. Garey and David S. Johnson. ““Strong” NP-Completeness Results: Motivation, Examples, and Implications”. In: *Journal of the ACM* 25.3 (1978), pp. 499–508. DOI: 10.1145/322077.322090.

[SDK15] Dvir Shabtay, Omri Dover, and Moshe Kaspi. “Single-Machine Two-Agent Scheduling Involving a Just-in-Time Criterion”. In: *International Journal of Production Research* 53.9 (2015), pp. 2590–2604. DOI: 10.1080/00207543.2014.970706.

[HS05] Refael Hassin and Mati Shani. “Machine Scheduling with Earliness, Tardiness and Non-Execution Penalties”. In: *Computers & Operations Research* 32.3 (2005), pp. 683–705. DOI: 10.1016/j.cor.2003.08.012.

[SGK13] Dvir Shabtay, Nufar Gaspar, and Moshe Kaspi. “A Survey on Offline Scheduling with Rejection”. In: *Journal of Scheduling* 16.1 (2013), pp. 3–28. DOI: 10.1007/s10951-012-0303-z.

[COS12] Bahriye Cesaret, Ceyda Oğuz, and F. Sibel Salman. “A Tabu Search Algorithm for Order Acceptance and Scheduling”. In: *Computers & Operations Research* 39.6 (2012), pp. 1197–1205. DOI: 10.1016/j.cor.2010.09.018.

[Gra69] Ronald L. Graham. “Bounds on Multiprocessing Timing Anomalies”. In: *SIAM Journal on Applied Mathematics* 17.2 (1969), pp. 416–429. DOI: 10.1137/0117039.

[DF54] George B. Dantzig and Delbert R. Fulkerson. “Minimizing the Number of Tankers to Meet a Fixed Schedule”. In: *Naval Research Logistics Quarterly* 1.3 (1954), pp. 217–222. DOI: 10.1002/nav.3800010309.

[Haj57] György Hajós. "Über eine Art von Graphen". In: *Internationale Mathematische Nachrichten* 11.65 (1957), p. 34. URL: https://www.oemg.ac.at/db/IMN/47/48b.

[Dil50] Robert P. Dilworth. "A Decomposition Theorem for Partially Ordered Sets". In: *Annals of Mathematics* 51.1 (1950), pp. 161–166. DOI: 10.2307/1969503.

[Ben59] Seymour Benzer. "On the Topology of the Genetic Fine Structure". In: *Proceedings of the National Academy of Sciences* 45.11 (1959), pp. 1607–1620. DOI: 10.1073/pnas.45.11.1607.

[Bez48] Max Bezzel. "Proposal of the 8-Queens Problem". In: *Berliner Schachzeitung* 3 (1848), p. 363. URL: https://play.google.com/store/books/details?id=ql5fAAAAcAAJ&rdid=book-ql5fAAAAcAAJ&rdot=1.

[Cam77] Paul J. Campbell. "Gauss and the Eight Queens Problem: A Study in Miniature of the Propagation of Historical Error". In: *Historia Mathematica* 4.4 (1977), pp. 397–404. DOI: 10.1016/0315-0860(77)90076-3.

[Ber58] Claude Berge. *Théorie des Graphes et ses Applications*. Dunod, 1958.

[Kar72] Richard M. Karp. "Reducibility Among Combinatorial Problems". In: *Complexity of Computer Computations*. Plenum Press, 1972, pp. 85–103. DOI: 10.1007/978-1-4684-2001-2_9.

[FG65] Delbert Fulkerson and Oliver Gross. "Incidence Matrices and Interval Graphs". In: *Pacific Journal of Mathematics* 15.3 (1965), pp. 835–855. DOI: 10.2140/pjm.1965.15.835.

[BL76] Kellogg S. Booth and George S. Lueker. "Testing for the Consecutive Ones Property, Interval Graphs, and Graph Planarity Using PQ-tree Algorithms". In: *Journal of Computer and System Sciences* 13.3 (1976), pp. 335–379. DOI: 10.1016/S0022-0000(76)80045-1.

[LB62] Cornelis Lekkerkerker and Johan Boland. "Representation of a Finite Graph by a Set of Intervals on the Real Line". In: *Fundamenta Mathematicae* 51.1 (1962), pp. 45–64. DOI: 10.4064/FM-51-1-45-64.

[GH64] Paul C. Gilmore and Alan J. Hoffman. "A Characterization of Comparability Graphs and of Interval Graphs". In: *Canadian Journal of Mathematics* 16 (1964), pp. 539–548. DOI: 10.4153/CJM-1964-055-5.

[Ber61] Claude Berge. "Färbung von Graphen, deren sämtliche bzw. deren ungerade Kreise starr sind". In: *Wissenschaftliche Zeitschrift der Martin-Luther-Universität Halle-Wittenberg, Mathematisch-Naturwissenschaftliche Reihe* 10.1 (1961), pp. 114–115.

[HS58] András Hajnal and János Surányi. "Über die Auflösung von Graphen in vollständige Teilgraphen". In: *Annales Universitatis Scientiarum Budapestinensis de Rolando, Eötvös Nominatae Sectio Mathematica* 1 (1958), pp. 113–121. URL: https://www.researchgate.net/profile/Andras-Hajnal-3/publication/268655028_Uber_die_Auflosung_von_Graphen_in_vollstandige_Teilgraphen/links/56c9c87b08ae11063709a070/Ueber-die-Aufloesung-von-Graphen-in-vollstaendige-Teilgraphen.pdf.

[Dir61] Gabriel Andrew Dirac. "On Rigid Circuit Graphs". In: *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*. Vol. 25. 1. Springer. 1961, pp. 71–76. DOI: 10.1007/BF02992776.

[Ros70] Donald J. Rose. "Triangulated Graphs and the Elimination Process". In: *Journal of Mathematical Analysis and Applications* 32.3 (1970), pp. 597–609. DOI: 10.1016/0022-247X(70)90282-9.

[Gav72] Fănică Gavril. "Algorithms for Minimum Coloring, Maximum Clique, Minimum Covering by Cliques, and Maximum Independent Set of a Chordal Graph". In: *SIAM Journal on Computing* 1.2 (1972), pp. 180–187. DOI: 10.1137/0201013.

[RTL76] Donald J. Rose, Robert Endre Tarjan, and George S. Lueker. "Algorithmic Aspects of Vertex Elimination on Graphs". In: *SIAM Journal on Computing* 5.2 (1976), pp. 266–283. DOI: 10.1137/0205021.

[FF98] Rudolf Fritsch and Gerda Fritsch. *Four-Color Theorem*. Springer, 1998. DOI: 10.1007/978-1-4612-1720-6.

[YG87] Mihalis Yannakakis and Fănică Gavril. "The Maximum k-Colorable Subgraph Problem for Chordal Graphs". In: *Information Processing Letters* 24.2 (1987), pp. 133–137. DOI: 10.1016/0020-0190(87)90107-4.

[HK56] Alan J. Hoffman and Joseph B. Kruskal. "Integral Boundary Points of Convex Polyhedra". In: *Annals of Mathematics* (1956). DOI: 10.1007/978-3-540-68279-0_3.

[AS87] Esther M. Arkin and Ellen B. Silverberg. "Scheduling Jobs with Fixed Start and End Times". In: *Discrete Applied Mathematics* 18.1 (1987), pp. 1–8. DOI: 10.1016/0166-218X(87)90037-0.

[EK72] Jack Edmonds and Richard M. Karp. "Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems". In: *Journal of the ACM* 19.2 (1972), pp. 248–264. DOI: 10.1145/321694.321699.

[CL95] Martin C. Carlisle and Errol L. Lloyd. "On the k-Coloring of Intervals". In: *Discrete Applied Mathematics* 59.3 (1995), pp. 225–235. DOI: 10.1016/0166-218X(95)80003-M.

[LLP11] Kangbok Lee, Joseph Y. T. Leung, and Michael Pinedo. "Scheduling Jobs with Equal Processing Times Subject to Machine Eligibility Constraints". In: *Journal of Scheduling* 14.1 (2011), pp. 27–38. DOI: 10.1007/s10951-010-0190-0.

[LS08] Lu-Wen Liao and Gwo-Ji Sheen. “Parallel Machine Scheduling with Machine Availability and Eligibility Constraints”. In: *European Journal of Operational Research* 184.2 (2008), pp. 458–467. DOI: 10.1016/j.ejor.2006.11.027.

[MW14] Matthias Mnich and Andreas Wiese. “Scheduling and Fixed-Parameter Tractability”. In: *Mathematical Programming* 154.1 (2014), pp. 533–562. DOI: 10.1007/s10107-014-0830-9.

[MB18] Matthias Mnich and René van Bevern. “Parameterized Complexity of Machine Scheduling: 15 Open Problems”. In: *Computers & Operations Research* 100 (2018), pp. 254–261. DOI: 10.1016/j.cor.2018.07.020.

[BHT92] Miklós Biró, Mihály Hujter, and Zsolt Tuza. “Precoloring Extension. I. Interval Graphs”. In: *Discrete Mathematics* 100.1-3 (1992), pp. 267–279. DOI: 10.1016/0012-365X(92)90646-W.

[Mar06] Dániel Marx. “Precoloring Extension on Unit Interval Graphs”. In: *Discrete Applied Mathematics* 154.6 (2006), pp. 995–1002. DOI: 10.1016/j.dam.2005.10.008.

[CY07] Byung-Cheon Choi and Suk-Hun Yoon. “Maximizing the Weighted Number of Just-in-Time Jobs in Flow Shop Scheduling”. In: *Journal of Scheduling* 10 (2007), pp. 237–243. DOI: 10.1007/s10951-007-0030-z.

[SB12] Dvir Shabtay and Yaron Bensoussan. “Maximizing the Weighted Number of Just-in-Time Jobs in Several Two-Machine Scheduling Systems”. In: *Journal of Scheduling* 15.1 (2012), pp. 39–47. DOI: 10.1007/s10951-010-0204-y.

[ELT13] Amir Elalouf, Eugene Levner, and Huajun Tang. “An Improved FPTAS for Maximizing the Weighted Number of Just-in-Time Jobs in a Two-Machine Flow Shop Problem”. In: *Journal of Scheduling* 16.4 (2013), pp. 429–435. DOI: 10.1007/s10951-013-0320-6.

[HST19] Danny Hermelin, Dvir Shabtay, and Nimrod Talmon. “On the Parameterized Tractability of the Just-in-Time Flow Shop Scheduling Problem”. In: *Journal of Scheduling* 22 (2019), pp. 663–676. DOI: 10.1007/s10951-019-00617-7.

[Hee+23] Klaus Heeger, Danny Hermelin, George B. Mertzios, Hendrik Molter, Rolf Niedermeier, and Dvir Shabtay. “Equitable Scheduling on a Single Machine”. In: *Journal of Scheduling* 26.2 (2023), pp. 209–225. DOI: 10.1007/s10951-022-00754-6.

[Her+25] Danny Hermelin, Hendrik Molter, Rolf Niedermeier, Michael Pinedo, and Dvir Shabtay. “Fairness in Repetitive Scheduling”. In: *European Journal of Operational Research* (2025). DOI: 10.1016/j.ejor.2024.12.052.

[SPF24] Dvir Shabtay, Andrei Plotkin, and Yali Fink. “Algorithms for Fair Repetitive Scheduling”. In: *Proceedings of the 2024 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)*. IEEE. 2024, pp. 844–847. DOI: 10.1109/IEEM62345.2024.10857092.

[SFP25] Dvir Shabtay, Yali Fink, and Andrei Plotkin. "Algorithms for Fair Repetitive Scheduling Based on Total Completion Time Criterion". In: *Computers & Industrial Engineering* (2025), p. 111659. DOI: 10.1016/j.cie.2025.111659.

[Raw71] John Rawls. "A Theory of Justice". In: Harvard University Press, 1971.

[RL07] Božidar Radunović and Jean-Yves Le Boudec. "A Unified Framework for Max-Min and Min-Max Fairness with Applications". In: *IEEE/ACM Transactions on Networking* 15.5 (2007), pp. 1073–1083. DOI: 10.1109/TNET.2007.896231.

[Mou04] Hervé Moulin. *Fair Division and Collective Welfare*. MIT Press, 2004.

[NP08] Dritan Nace and Michał Pióro. "Max-Min Fairness and Its Applications to Routing and Load-Balancing in Communication Networks: A Tutorial". In: *IEEE Communications Surveys & Tutorials* 10.4 (2008), pp. 5–17. DOI: 10.1109/SURV.2008.080403.

[Bon+06] Thomas Bonald, Laurent Massoulié, Alexandre Proutière, and Jorma Virtamo. "A Queueing Analysis of Max-Min Fairness, Proportional Fairness and Balanced Fairness". In: *Queueing Systems* 53.1–2 (2006), pp. 65–84. DOI: 10.1007/s11134-006-7587-7.

[Fel+09] Michael R. Fellows, Danny Hermelin, Frances Rosamond, and Stéphane Vialette. "On the Parameterized Complexity of Multiple-Interval Graph Problems". In: *Theoretical Computer Science* 410.1 (2009), pp. 53–61. DOI: 10.1016/j.tcs.2008.09.065.

[Moo68] Michael J. Moore. "An n Job, One Machine Sequencing Algorithm for Minimizing the Number of Late Jobs". In: *Management Science* 15.1 (1968), pp. 102–109. DOI: 10.1287/mnsc.15.1.102.

[GD08] Diwakar Gupta and Brian Denton. "Appointment Scheduling in Health Care: Challenges and Opportunities". In: *IIE Transactions* 40.9 (2008), pp. 800–819. DOI: 10.1080/07408170802165880.

[Car+11] Adam P. Carpenter, Lawrence M. Leemis, Alan S. Papir, David J. Phillips, and Grace S. Phillips. "Managing Magnetic Resonance Imaging Machines: Support Tools for Scheduling and Planning". In: *Health Care Management Science* 14.2 (2011), pp. 158–173. DOI: 10.1007/s10729-011-9153-z.

[Ala+22] Ali Ala, Vladimir Simic, Dragan Pamucar, and Erfan Babaee Tirkolaee. "Appointment Scheduling Problem under Fairness Policy in Healthcare Services: Fuzzy Ant Lion Optimizer". In: *Expert Systems with Applications* 207 (2022), p. 117949. DOI: 10.1016/j.eswa.2022.117949.

[Len83] Hendrik W. Lenstra Jr. "Integer Programming with a Fixed Number of Variables". In: *Mathematics of Operations Research* 8.4 (1983), pp. 538–548. DOI: 10.1287/moor.8.4.538.

[CFG03] Yijia Chen, Jörg Flum, and Martin Grohe. "Bounded Nondeterminism and Alternation in Parameterized Complexity Theory". In: *18th IEEE Annual Conference on Computational Complexity*. IEEE. 2003, pp. 13–29. DOI: 10.1109/CCC.2003.1214407.

[FG06] Jörg Flum and Martin Grohe. *Parameterized Complexity Theory*. Springer, 2006. DOI: 10.1007/3-540-29953-X.

[Vaz13] Vijay V. Vazirani. *Approximation Algorithms*. Springer, 2013. DOI: 10.1007/978-3-662-04565-7.

[WS11] David P. Williamson and David B. Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011. DOI: 10.1017/CBO9780511921735.

[BBN19] Matthias Bentert, René van Bevern, and Rolf Niedermeier. "Inductive k-Independent Graphs and c-Colorable Subgraphs in Scheduling: A Review". In: *Journal of Scheduling* 22.1 (2019), pp. 3–20. DOI: 10.1007/s10951-018-0595-8.

[Tov84] Craig A. Tovey. "A Simplified NP-Complete Satisfiability Problem". In: *Discrete Applied Mathematics* 8.1 (1984), pp. 85–89. DOI: 10.1016/0166-218X(84)90081-7.

[IP01] Russell Impagliazzo and Ramamohan Paturi. "On the Complexity of k-SAT". In: *Journal of Computer and System Sciences* 62.2 (2001), pp. 367–375. DOI: 10.1006/jcss.2000.1727.

[IPZ01] Russell Impagliazzo, Ramamohan Paturi, and Francis Zane. "Which Problems Have Strongly Exponential Complexity?" In: *Journal of Computer and System Sciences* 63.4 (2001), pp. 512–530. DOI: 10.1006/jcss.2001.1774.

[LL19] T. S. Lee and Y. T. Loong. "A Review of Scheduling Problem and Resolution Methods in Flexible Flow Shop". In: *International Journal of Industrial Engineering Computations* 10 (2019), pp. 67–88. DOI: 10.5267/j.ijiec.2018.4.001.

[RV10] R. Ruiz and J. A. Vazquez-Rodríguez. "The Hybrid Flow Shop Scheduling Problem". In: *European Journal of Operational Research* 205 (2010), pp. 1–18. DOI: 10.1016/j.ejor.2009.09.024.

[TMT20] Ömür Tosun, Mariappan Kadarkarainadar Marichelvam, and Nedret Tosun. "A Literature Review on Hybrid Flow Shop Scheduling". In: *International Journal of Advanced Operations Management* 12.2 (2020), pp. 156–194. DOI: 10.1504/IJAOM.2020.108263.

[Sch98] Alexander Schrijver. *Theory of Linear and Integer Programming*. John Wiley & Sons, 1998. DOI: 10.1137/1030065.

[Vai89] Pravin M. Vaidya. "Speeding-Up Linear Programming Using Fast Matrix Multiplication (Extended Abstract)". In: *Proceedings of the 30th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society, 1989, pp. 332–337. DOI: 10.1109/SFCS.1989.63499.

[IK75] Oscar H. Ibarra and Chul E. Kim. "Fast Approximation Algorithms for the Knapsack and Sum of Subset Problems". In: *Journal of the ACM* 20.4 (1975), pp. 463–468. DOI: 10.1145/321906.321909.

[Joh+74] David S. Johnson, Alan Demers, Jeffrey D. Ullman, Michael R. Garey, and Ronald L. Graham. "Worst-case Performance Bounds for Simple One-Dimensional Packing Algorithms". In: *SIAM Journal on Computing* 3.4 (1974), pp. 299–325. DOI: 10.1137/0203025.

[Hom00] Willard Hom. *An Overview of Customer Satisfaction Models.* Tech. rep. California Community Colleges, 2000. URL: https://eric.ed.gov/?id=ed463825.

[LWC11] Chun-Chun Lin, Hsueh-Ying Wu, and Yong-Fu Chang. "The Critical Factors Impact on Online Customer Satisfaction". In: *Procedia Computer Science* 3 (2011), pp. 276–281. DOI: 10.1016/j.procs.2010.12.047.

[SPC17] Sang-Oh Shim, KyungBae Park, and SungYong Choi. "Innovative Production Scheduling with Customer Satisfaction Based Measurement for the Sustainability of Manufacturing Firms". In: *Sustainability* 9.12 (2017), p. 2249. DOI: 10.3390/su9122249.

[Che+11] Junliang Chen, Chen Wang, Bing Bing Zhou, Lei Sun, Young Choon Lee, and Albert Y. Zomaya. "Tradeoffs Between Profit and Customer Satisfaction for Service Provisioning in the Cloud". In: *Proceedings of the 20th International Symposium on High Performance Distributed Computing*. 2011, pp. 229–238. DOI: 10.1145/1996130.1996161.

[LLP97] Chung-Yee Lee, Lei Lei, and Michael Pinedo. "Current Trends in Deterministic Scheduling". In: *Annals of Operations Research* 70 (1997), pp. 1–41. DOI: 10.1023/A:1018909801944.

[Dou+07] Nikolaos D. Doulamis, Anastasios D. Doulamis, Emmanouel A. Varvarigos, and Theodora A. Varvarigou. "Fair Scheduling Algorithms in Grids". In: *IEEE Transactions on Parallel and Distributed Systems* 18.11 (2007), pp. 1630–1648. DOI: 10.1109/TPDS.2007.1053.

[Gup+26] Swati Gupta, Akhil Jalan, Gireeja Ranade, Helen Yang, and Simon Zhuang. "Too Many Fairness Metrics: Is There a Solution? Equity Across Demographic Groups for the Facility Location Problem". In: *Data Science and Optimization*. Springer, 2026, pp. 231–272. DOI: 10.1007/978-3-032-03844-9_10.

[Gol80] Martin Charles Golumbic. *Algorithmic Graph Theory and Perfect Graphs*. New York: Academic Press, 1980. DOI: 10.1016/C2013-0-10739-8.

[RS86] Neil Robertson and Paul D. Seymour. "Graph minors. II. Algorithmic Aspects of Tree-Width". In: *Journal of Algorithms* 7.3 (1986), pp. 309–322. DOI: 10.1016/0196-6774(86)90023-4.

[Bod93] Hans L. Bodlaender. "A Tourist Guide Through Treewidth". In: *Acta Cybernetica* 11.1-2 (1993), pp. 1–21.

[Klo94] Ton Kloks. *Treewidth: Computations and Approximations*. Springer, 1994. DOI: 10.1007/BFb0045375.

[AN02] Jochen Alber and Rolf Niedermeier. "Improved Tree Decomposition Based Algorithms for Domination-Like Problems". In: *Latin American Symposium on Theoretical Informatics*. Springer. 2002, pp. 613–627. DOI: 10.1007/3-540-45995-2_52.

[Pie03] Krzysztof Pietrzak. "On the Parameterized Complexity of the Fixed Alphabet Shortest Common Supersequence and Longest Common Subsequence Problems". In: *Journal of Computer and System Sciences* 67.4 (2003), pp. 757–771. DOI: 10.1016/S0022-0000(03)00078-3.

[APT79] Bengt Aspvall, Michael F. Plass, and Robert Endre Tarjan. "A Linear-Time Algorithm for Testing the Truth of Certain Quantified Boolean Formulas". In: *Information Processing Letters* 8.3 (1979), pp. 121–123. DOI: 10.1016/0020-0190(79)90002-4.

[HK73] John E. Hopcroft and Richard M. Karp. "An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs". In: *SIAM Journal on Computing* 2.4 (1973), pp. 225–231. DOI: 10.1137/0202019.

[BJ25] Boris Bukh and R. Amzi Jeffs. "Enumeration of Interval Graphs and d-Representable Complexes". In: *Israel Journal of Mathematics* (2025), pp. 1–18. DOI: 10.1007/s11856-025-2814-2.

[YP17] Joyce Yang and Nicholas Pippenger. "On the Enumeration of Interval Graphs". In: *Proceedings of the American Mathematical Society, Series B* 4.1 (2017), pp. 1–3. DOI: 10.1090/bproc/27.

[Bam+12] Evripidis Bampis, Christoph Dürr, Fadi Kacem, and Ioannis Milis. "Speed Scaling with Power Down Scheduling for Agreeable Deadlines". In: *Sustainable Computing: Informatics and Systems* 2.4 (2012), pp. 184–189. DOI: 10.1016/j.suscom.2012.10.003.

[Gil+21] Miri Gilenson, Dvir Shabtay, Liron Yedidsion, and Rohit Malshe. "Scheduling in Multi-Scenario Environment with an Agreeable Condition on Job Processing Times". In: *Annals of Operations Research* 307.1-2 (2021), pp. 153–173. DOI: 10.1007/s10479-021-04316-5.

[MY13] Ran Ma and Jinjiang Yuan. "Online Scheduling on a Single Machine with Rejection Under an Agreeable Condition to Minimize the Total Completion Time Plus the Total Rejection Cost". In: *Information Processing Letters* 113.17 (2013), pp. 593–598. DOI: 10.1016/j.ipl.2013.05.006.

[LNC07] L. L. Liu, Chi To Ng, and T. C. Edwin Cheng. "Scheduling Jobs with Agreeable Processing Times and Due Dates on a Single Batch Processing Machine". In: *Theoretical Computer Science* 374.1-3 (2007), pp. 159–169. DOI: 10.1016/j.tcs.2006.12.039.

[SG22] Dvir Shabtay and Miri Gilenson. "A State-of-the-Art Survey on Multi-Scenario Scheduling". In: *European Journal of Operational Research* (2022). DOI: 10.1016/j.ejor.2022.11.014.

[Pie14] Maciej Pieńkowski. "Waste Measurement Techniques for Lean Companies". In: *International Journal of Lean Thinking* 5.1 (2014), pp. 9–24. URL: https://www.researchgate.net/publication/343083406_Waste_Measurement_Techniques_For_Lean_Companies.

[Rad+24] Radin Zaid Radin Umar, Jing Yin Tiong, Nadiah Ahmad, and Jalaluddin Dahalan. "Development of Framework Integrating Ergonomics in Lean's Muda, Muri, and Mura Concepts". In: *Production Planning & Control* 35.12 (2024), pp. 1466–1474. DOI: 10.1080/09537287.2023.2189640.

[Kat17] Hiroshi Katayama. "Legend and Future Horizon of Lean Concept and Technology". In: *Procedia Manufacturing* 11 (2017), pp. 1093–1101. DOI: 10.1016/j.promfg.2017.07.227.

[LR72] E. M. Livshits and Y. I. Rublinetsky. "On Comparative Complexity of Some Discrete Optimization Problems". In: *Computational Mathematics and Computers* 3 (1972), pp. 78–85. URL: https://elementsofscheduling.nl/PDF/comparative_complexity_of_some_discrete_optimization_problems.pdf.

[LSV23] Jan Karel Lenstra, Vitaly A Strusevich, and Milan Vlach. "A Historical Note on the Complexity of Scheduling Problems". In: *Operations Research Letters* 51.1 (2023), pp. 1–2. DOI: 10.1016/j.orl.2022.11.006.

[GJ79] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, 1979. DOI: 10.1137/1024022.

[BF95] Hans L. Bodlaender and Michael R. Fellows. "W[2]-Hardness of Precedence Constrained k-Processor Scheduling". In: *Operations Research Letters* 18.2 (1995), pp. 93–97.

[FM03] Michael R Fellows and Catherine McCartin. "On the Parametric Complexity of Schedules to Minimize Tardy Tasks". In: *Theoretical Computer Science* 298.2 (2003), pp. 317–324. DOI: 10.1016/S0304-3975(02)00811-3.

[Mar11] Dániel Marx. *Fixed-Parameter Tractable Scheduling Problems*. Tech. rep. 2. Seminar 11091. Dagstuhl Reports, Schloss Dagstuhl – Leibniz Center for Informatics, 2011, pp. 67–86. DOI: 10.4230/DagRep.1.2.67.

[Her+18] Danny Hermelin, Shlomo Karhi, Michael Pinedo, and Dvir Shabtay. "New Algorithms for Minimizing the Weighted Number of Tardy Jobs on a Single Machine". In: *Annals of Operations Research* 298.1 (2018), pp. 271–287. DOI: 10.1007/s10479-018-2852-9.

[CA04] G. Centeno and Robert L. Armacost. "Minimizing Makespan on Parallel Machines with Release Time and Machine Eligibility Restrictions". In: *International Journal of Production Research* 42.6 (2004), pp. 1243–1256. DOI: 10.1080/00207540310001631584.

[Cie+04] Mark Cieliebak, Thomas Erlebach, Fabian Hennecke, Birgitta Weber, and Peter Widmayer. "Scheduling with Release Times and Deadlines on a Minimum Number of Machines". In: *Exploring New Frontiers of Theoretical Informatics*. Vol. 155. Springer, 2004, pp. 209–222. DOI: 10.1007/1-4020-8141-3_18.

[BNS17] René van Bevern, Rolf Niedermeier, and Ondřej Suchý. "A Parameterized Complexity View on Non-Preemptively Scheduling Interval-Constrained Jobs: Few Machines, Small Looseness, and Small Slack". In: *Journal of Scheduling* 20.3 (2017), pp. 255–265. DOI: 10.1007/s10951-016-0478-9.

[SS09] Jagdeep Singh and Harwinder Singh. "Kaizen Philosophy: A Review of Literature". In: *IUP journal of operations management* 8.2 (2009), p. 51. URL: https://cdn.chools.in/LEAN_PDF/Kaizen%20Toolkit/Kaizen_Philosophy_A_Review_of_Literature.pdf.

תוכן עניינים

1. מבוא 1
1.1 תזמון וסיבוכיות 1
1.2 ייצור בדיוק-בזמן 3
1.3 מודלים של תזמון בדיוק-בזמן 5
1.4 תזמון מקטעים 7
1.5 שלוש בעיות תזמון מקטעים 11
1.6 עבודות קודמות 15
1.7 תרומת המחקר 17
2. מושגים ומינוחים 23
3. תזמון עם רשימת מכונות כשירות 27
3.1 תוצאות קושי 29
3.2 תוצאות חיוביות 38
3.3 דיון וסיכום 42
4. תזמון זרימה דו-שלבי 43
4.1 תוצאות קושי 45
4.2 תוצאות חיוביות 53
4.3 דיון וסיכום 68
5. תזמון מחזורי הוגן 71
5.1 תוצאות קושי 73
5.2 תוצאות חיוביות 84
5.3 דיון וסיכום 92
מסקנות 95
ביבליוגרפיה 99
תקציר 116
תמצית 121

תקציר

תזמון עוסק בהקצאת משאבים למשימות בכפוף לאילוצי זמן, במטרה למטב מדד ביצוע נתון. עבודה זו חוקרת את הסיבוכיות החישובית של שלוש בעיות תזמון מקטעים, ומשלבת תוצאות קושי עם אלגוריתמים מדויקים וסכימות קירוב. הדגש הוא על זיהוי פרמטרים מבניים המאפשרים פתרון יעיל של מקרים מיוחדים, גם כאשר הבעיה הכללית קשה חישובית.

התמודדות עם בעיות NP-קשות מהווה מוטיב מרכזי במחקר האלגוריתמי. בעוד שגישות רבות נוקטות בקירוב של הפתרון האופטימלי [Hoc97] או בגישה הסתברותית [Kar91] כדי להימנע מזמני ריצה מעריכיים, מחקר זה שואף להציע פתרונות מדויקים ודטרמיניסטיים. לצורך כך אנו נעזרים בתורת הסיבוכיות הפרמטרית (Parameterized Complexity) [Nie06, DF13, Cyg+15]. גישה זו בוחנת פתרון מדויק בזמן ריצה התלוי באופן מעריכי רק בפרמטרים נומרים מסוימים של קלט הבעיה, בעוד שביחס לגודל הקלט הכולל, זמן הריצה נותר פולינומי. אלגוריתם המקיים תכונה זו נקרא אלגוריתם FPT (fixed-parameter tractable) - אלגוריתם יעיל ביחס לפרמטר קבוע. פורמלית, נגיד שאלגוריתם FPT ביחס לפרמטר k ירוץ בזמן $f(k) \cdot n^{O(1)}$, כאשר n הוא גודל הקלט. בכך הם מאפשרים להתגבר על הקושי החישובי המובנה של בעיות NP-קשות עבור מקרים בהם הפרמטר נותר קטן. בדומה לסיבוכיות הקלאסית, גם בסיבוכיות הפרמטרית קיים מושג של קושי פרמטרי. ניתן להראות שבעיה היא W[1]-קשה ביחס לפרמטר מסוים ובכך, תחת השערות מקובלות, לשלול את קיומו של אלגוריתם FPT ביחס לאותו פרמטר.

עבודה זו משלבת מתודולוגיות מתחום הסיבוכיות הקלאסית והפרמטרית לצורך ניתוחן של שלוש בעיות תזמון. בפרט, אנו מתמקדים בבעיות תזמון מקטעים (Interval Scheduling) תחום השואב את השראתו מתפיסות ניהוליות כגון 'ייצור רזה' (Lean Manufacturing) ו-'ייצור בדיוק בזמן' או JIT (Just-in-Time Production). פרדיגמות אלו מהוות אבן יסוד בהצלחות היסטוריות בתעשייה, והן מציבות אתגרים אלגוריתמיים מורכבים הדורשים פתרונות מדויקים.

ייצור בדיוק בזמן, JIT, היא אחת מפילוסופיות הניהול המשפיעות ביותר במאה ה-20. תפיסת JIT דוגלת בייצור מוצרים בדיוק בזמן בו הם נדרשים במטרה למזער עלויות החזקת מלאים. הפיתוח השיטתי של JIT והפיכתה לאסטרטגיית ייצור עולמית מיוחסים בעיקר ליצרנית הרכב היפנית טויוטה (Toyota Motor Corporation), אשר הטמיעה אותה כמרכיב ליבה בשיטת הניהול שלה – TPS (Toyota Production System). הטמעתה המוצלחת של פילוסופיית JIT בפסי הייצור של טויוטה לוותה בגידול דרמטי בנפח הייצור של החברה (ראו איור 1.1), ויש הטוענים כי היא הייתה הגורם המרכזי להצלחה הגלובלית של תעשיית הרכב היפנית [Pet02, FC22, WJR90].

מבחינה תזמונית, גישת JIT הופכת את הדיוק בזמן לאילוץ קשיח. על פיה, איחורים משבשים תהליכים עתידיים, אך באותה מידה, גם השלמת משימה בטרם עת גוררת עלויות נלוות, כגון עלויות החזקת מלאי. זאת בניגוד למודלים מסורתיים המבוססים על פונקציות מטרה רגולריות, אשר אינן "קונסות" סיום מוקדם של משימה ולפיכך מעדיפות תמיד הקדמה על פני איחור. מטרות רגולריות מצמצמות, למעשה, את בעיית התזמון לבעיית קביעת רצף (Sequencing) כלומר, מציאת סדר הפעולות האופטימלי עבור כל מכונה.

תוצאה חשובה של תכונת הרגולריות היא שבהינתן סדר ביצוע קבוע, ניתן להתחיל כל משימה במועד המוקדם ביותר המותר לפי אילוצי המערכת, בלי להרע את ערך פונקציית המטרה. ייתכנו זמני בטלה הנכפים על ידי אילוצים כגון זמני שחרור או יחסי קדימות, אך אין תועלת בדחייה נוספת של תחילת הביצוע. דוגמה בולטת למדד רגולרי היא מזעור סך האיחורים הממושקל,

$\sum_j w_j T_j$ במודל זה, האיחור של משימה j מוגדר כ- $T_j = \max\{0, C_j - d_j\}$ כאשר C_j הוא זמן הסיום של המשימה, d_j הוא המועד המבוקש (due date) ו w_j הוא המשקל המייצג את חשיבותה.

גישת JIT מאתגרת את תפיסה זו מאחר שהיא שואפת להימנע הן מעיכובים והן מהקדמות. תחת מטרת JIT, פתרונות אופטימליים עשויים לדרוש זמני בטלה יזומים על מנת להבטיח סיום במועד המדויק, דבר המגדיל משמעותית את מרחב הפתרונות האפשריים [BM99]. יתרה מכך, הידע התיאורטי שפותח עבור קריטריונים רגולריים לרוב אינו ניתן להכללה עבור מטרות לא-רגולריות מעין אלו [RS93].

עבודות מוקדמות בתחום תזמון JIT מגלמות עלויות באופן פרופורציונלי לגודל הסטייה מזמן הסיום המבוקש [BS90, LW04]. גישה זו שואפת למזער את סך הסטייה הממושקלת $\sum_j (w_j^T T_j + w_j^E E_j)$ כאשר T_j מייצג את האיחור (Tardiness) ו- E_j מייצג את ההקדמה (Earliness) של משימה j. מודלים אלו הוכחו כקשים מאוד לאופטימיזציה מבחינה חישובית [HP91a, HP91b], שכן הם מהווים הכללה של בעיות מזעור איחורים ממושקלים $\sum_j w_j T_j$, אשר כשלעצמן אינן מאפשרות אלגוריתמים יעילים ברוב המקרים [LKB77, Law77, DL90, GJ78]. עבודה מאוחרת יותר הציגה תפנית בגישה - בסביבות ייצור רבות (כגון תעשיית החלל, כימיה ומזון), גודל הסטייה מהמועד המבוקש אינו רלוונטי, שכן כל חריגה מחלון הזמן גוררת קנס קבוע ובלתי תלוי במשך העיכוב או ההקדמה [LM96].

תובנה זו הולידה את המדד הבינארי Z_j, לפיו משימה נחשבת למוצלחת $(Z_j = 1)$ אך ורק אם היא מסתיימת בדיוק במועד המבוקש $(C_j = d_j)$. כשם שפונקציה רגולרית מצמצמת את בעיית התזמון לבעיית מציאת סדר ביצוע למשימות (Sequencing), כך מדד הביצוע הבינארי מצמצם את בעיית התזמון לבעיית בחירת משימות (Job Selection). תחת מדד זה, ותחת ההנחה שלא ניתן לעצור משימה ברגע שהחלה (non-preemptive), נגזר זמן התחלה קשיח $d_j - p_j$ לביצוע המשימה. מכך מתקבל מקטע זמן יחיד וקשיח, $[d_j - p_j, d_j)$, המייצג את חלון הביצוע עבור משימה j. האתגר החישובי הופך למציאת תת-קבוצה אופטימלית של מקטעים שאינם חופפים, מודל המכונה בספרות 'תזמון מקטעים' (Interval Scheduling).

את בעיית תזמון המקטעים ניתן לייצג באמצעות גרף הקונפליקט של המשימות. גרפים הם אובייקטים מתמטיים הבנויים מצמתים וקשתות ומשמשים למידול יחסים. בייצוג זה, אנו מגדירים צומת עבור כל משימה, ומותחים קשת בין כל זוג צמתים שבין המקטעים שלהם קיימת חפיפה בזמן (ראו איור 1.2). המטרה של בחירת מספר מקסימלי של משימות ללא חפיפה שקולה למציאת הקבוצה הבלתי תלויה המקסימלית (Maximum Independent Set). בעיה זו היא בעיה קלאסית בתורת הגרפים והיא הופיעה בחידות כבר במאה ה-19 (ראו איור 1.3). בעוד בעיית הקבוצה הבלתי תלויה בגרפים כלליים היא בעיה NP-קשה [Kar72], במקרה שלנו לא מדובר בגרף כללי אלא בגרף בו יחס הקונפליקט בין משימות הוא בדיוק יחס החפיפה בין מקטעיהן.

גרפים המקיימים תנאי זה נקראים גרפים מקטעיים (Interval Graphs). מאז הגדרתם על ידי המתמטיקאי יורג האיוש ב-1957 [Haj57] נחקרו לעומק תכונותיהם המבניות הייחודיות [LB62, GH64, FG65]. אפיון מרכזי של גרפים אלו הוא היותם תת-קבוצה של גרפים מיתריים (Chordal Graphs) - גרפים שבהם לכל מעגל באורך 4 לפחות קיים מיתר המחבר בין זוג צמתים שאינם עוקבים - אפיון זה אפשר לפניקה גבריל לרתום את המחקר שכבר נעשה על גרפים מיתריים [Ber61, HS58, Dir61, Ros70] ולפתח אלגוריתם יעיל לבעיית הקבוצה הבלתי תלויה בגרף מקטעי [Gav72].

בעיית גרפים קלאסית נוספת, המשיקה ישירות לעולם התזמון, היא בעיית הצביעה (Coloring). בבעיה זו אנו נדרשים לצבוע את צמתי הגרף במספר מינימלי של צבעים כך שכל זוג צמתים שכנים צבועים בצבעים שונים. בדומה לבעיית הקבוצה הבלתי תלויה, גם בעיה קלאסית זו עניינה מתמטיקאים כבר במאה ה-19 (ראו איור 1.5) וגם היא הוכחה כבעיה NP-קשה על גרפים כלליים [Kar72]. כמו כן, גם עבורה פיתח גבריל אלגוריתם פולינומי המכריע אותה על גרף מקטעי [Gav72], ובהמשך הכליל אותה לבעיית מציאת תת-גרף מקסימלי הניתן לצביעה עם k צבעים [YG87].

במונחי תזמון בעיה זו שקולה לבעיית מקסום התפוקה (Throughput Maximization): מציאת קבוצת המשימות הגדולה ביותר שיכולה להתבצע באופן JIT על k מכונות זהות. במודל זה כל מכונה מיוצגת על ידי צבע, משימה המשויכת לצבע מסוים מתבצעת על המכונה המתאימה לו, ואף זוג משימות בעלות מקטעים חופפים אינו יכול להתבצע על אותה מכונה. עבור בעיית מקסום התפוקה פותחו בהדרגה אלגוריתמים יעילים יותר ויותר [AS87, EK72, CL95]. אחד המחקרים הציע הכללה של הבעיה, שבה עבור כל משימה מפורטת רשימת מכונות כשירות — תת-קבוצה של מכונות אשר רק הן מסוגלות לבצע את המשימה. זוהי גם בעיית תזמון המקטעים הראשונה שעבודה זו בוחנת.

תזמון עם רשימת מכונות כשירות (Interval Scheduling with Eligible Machines). החוקרות אסתר ארקין ואלן סילברברג הוכיחו שבעיה זו היא NP-קשה [AS87]. בנוסף, הן פירטו אלגוריתם הפותר אותה בזמן $O(n^{m+1})$ תוצאה שלא השתפרה עד היום. בסקירה מקיפה מ-2018 סימנו ואן-בורן ומניך את שאלת קיומו של אלגוריתם יעיל ביחס למספר המכונות m כאחת מ-15 השאלות הפתוחות החשובות בתורת התזמון [MB18]. בעבודה זו אנו משיבים על שאלתם בשלילה ומראים שהבעיה W[1]-קשה ביחס לפרמטר m (Theorem 1.1). אנו עושים זאת באמצעות רדוקציה פרמטרית מבעיית הקליקה הצבעונית [Fel+09]. תוצאתנו שוללת אלגוריתם שזמן ריצתו $f(m) \cdot n^{O(1)}$ לכל פונקציה חישיבה f, תחת ההשערה FPT $\neq$ W[1].

בהמשך אנו בוחנים את הפרמטר $p_{\max}$, זמן העיבוד המקסימלי מבין כל המשימות. אנו מוכיחים כי הבעיה NP-קשה גם במקרה שפרמטר זה חסום בקבוע (Theorem 1.2). לבסוף, לצד תוצאות קושי אלו, השוללות קיומו של אלגוריתם יעיל ביחס לכל אחד מהפרמטרים בנפרד, אנו מציגים אלגוריתם יעיל ביחס לפרמטר המשולב $p_{\max} + m$ (Theorem 1.3). האלגוריתם מתבסס על הליך סינון משימות נחותות, שאינן נחוצות עבור פתרון אופטימלי, ואחריו תוכנית דינמית המוצאת שיבוץ אופטימלי לקבוצת המשימות הנותרת. בכך אנו מראים כי הסיבוכיות של הבעיה גדלה בצורה מתונה כששני הפרמטרים קטנים יחסית.

תזמון זרימה דו-שלבי (Flexible Flow Shop Interval Scheduling). בבעיות מסוג תזמון זרימה (Flow Shop) כל משימה מורכבת מסדרת תהליכונים שיש לבצע בסדר קבוע, כאשר לכל שלב מוקדשת מכונה ייעודית. החוקרים צ'וי ויון הראו כי, עבור פונקציית המטרה $\sum w_j Z_j$, מקסום התפוקה במערכת דו-שלבית הוא בעיה NP-קשה אפילו כאשר לכל המשימות אותו זמן עיבוד בשלב השני [CY07]. עבודות קודמות מראות כי המקרה הלא-ממושקל של הבעיה פתיר בזמן $O(n^3)$ [Sha12] ואילו המקרה הממושקל פתיר בזמן פסודו-פולינומי [SB12] ואף מעמיד סכימת קירוב בזמן פולינומי מלא (FPTAS) [ELT13].

אנו בוחנים הכללה של תזמון דו-שלבי עבור מערכת גמישה (Flexible Flow Shop), שבה השלב השני כולל מספר מכונות זהות הפועלות במקביל. סקירות ספרותיות מראות כי בעיות אלו

מאתגרות חישובית וכי מרבית המאמרים בתחום מציעים היוריסטיקות לפתרון ולא אלגוריתמים מדויקים [LL19, TMT20]. תרומתנו המרכזית לנושא זה היא ההוכחה כי הוספת גמישות למערכת — בדמות ריבוי מכונות בשלב השני — הופכת את הבעיה לקשה משמעותית מבחינה חישובית ונותנת חסמים תחתונים חזקים לזמן הריצה של כל אלגוריתם מדויק. בפרט, אנו מראים כי הבעיה הלא ממושקלת NP-קשה במובן החזק ובנוסף W[2]-קשה ביחס למספר המכונות בשלב השני (Theorem 1.4). תוצאה זו שוללת את קיומו של אלגוריתם FPT ביחס לפרמטר זה, תחת ההשערה כי W[2]≠FPT.

כדי להתמודד עם קושי זה, אנו מפתחים שלוש תוכניות דינמיות בזמן פסודו-פולינומי הפותרות מקרים פרטיים של הבעיה (Theorem 1.5 & Theorem 1.6). בנוסף אנו מראים, באמצעות שינוי קנה מידה ועיגול המשקלים, כיצד להמיר כל אחת מהתוכניות בסכימת קירוב בזמן פולינומי מלא (Theorem 1.8). התוכנית הראשונה יעילה למקרה בו מספר המכונות קבוע ומראה כי המקרה הלא משוקלל ולא גמיש פתיר בזמן $O(n^2)$, שיפור של האלגוריתם המהיר ביותר למקרה זה. שתי התוכניות האחרות פותרות את הבעיה בזמן פסודו-פולינומי כאשר זמן העיבוד המקסימלי של התהליכונים בשלב השני חסום בקבוע, או כאשר מספר הצביעה של גרף הקונפליקט בשלב השני הוא $O(\log n)$, כאשר n הוא מספר המשימות.

בנוסף אנו בוחנים את המקרה בו זמני העיבוד של התהליכונים בשלב הראשון זהים. אנו מפתחים אלגוריתמים יעילים תחת כל אחת משתי הגבלות נוספות בנפרד (Theorem 1.7). בראשונה גם משקלי המשימות זהים ובשנייה אף מקטע של תהליכון בשלב השני לא מכיל במלואו מקטע של משימה אחרת.

תזמון מחזורי הוגן (Fair Repetitive Interval Scheduling). בשנים האחרונות שביעות רצון הלקוחות נהפכה ליעד מרכזי אצל נותני שירות [Hom00, LWC11], דבר שהוביל למגמה גוברת של תעדוף הוגנות על פני מטרות אחרות [SPC17, Che+11, LLP97, Dou+07]. אך למרות השאיפה להקצאה הוגנת, חסרה בספרות הגדרה אחידה למושג; מה שנחשב הוגן תלוי בנסיבות ובהעדפות ספציפיות [Gup+26]. בהיעדר הגדרה כזו, כל מקרה נדרש למודל משלו, ואין מסגרת כללית התומכת בקבלת החלטות הוגנת במערכות שירות.

עבודה זו מבקשת לצמצם פער זה באמצעות ניתוח מעמיק של מודל תזמון מחזורי שנחקר לאחרונה [Hee+23, Her+25, SPF24, SFP25]. גישה זו הוצעה בהשראת מסך הבערות של רולס [Raw71], במטרה למקסם הוגנות במובן של מיקסום התועלת המינימלית. בכל יום מבקש כל לקוח לבצע משימה המוגדרת באמצעות מקטע זמן, וכל יום מהווה מקרה של תזמון מקטעים על מכונה בודדת. שביעות הרצון של לקוח נמדדת במספר משימותיו שבוצעו לאורך התקופה, ואנו מבקשים להכריע אם קיים שיבוץ שבו מבוצעות לפחות k משימות לכל לקוח.

אנו מראים כי הבעיה NP-קשה כבר עבור שלושה ימים ודרישת שירות של משימה אחת לכל לקוח. באופן כללי, אנו מוכיחים קושי לכל זוג קבוע של ערכים $q \geq 3$ ו-$1 \leq k \leq q-2$. בנוסף אנו מראים כי עבור $k = q-1$, הבעיה פתירה בזמן פולינומי (Theorem 1.9). בנוסף אנו מראים כי הבעיה NP-קשה גם כאשר זמני העיבוד של כל המשימות בכל הימים זהים. למרות זאת, כאשר מגבילים את זמני העיבוד לגודל יחידה אנו מראים כי הבעיה פתירה בזמן פולינומי. מקרה זה שכיח במערכות בריאות ציבוריות, בהן פגישות מתוזמנות לאורך זמן אחיד (Theorem 1.10).

אנו בוחנים בנוסף את המקרה הטבעי שבו זמן הסיום הרצוי של כל לקוח קבוע ואינו משתנה מיום ליום (Theorem 1.11). אנו מראים כי גם מקרה זה NP-קשה אך מזהים שני תתי-מקרים הניתנים לפתרון יעיל: כאשר מספר הימים קבוע או כאשר זמני העיבוד אינם תלויים ביום —

ובמילים אחרות, גרף הקונפליקט זהה בכל הימים.

בנוסף אנו מראים כי הבעיה קשה ביחס לרוחב העץ (treewidth) של גרף הקונפליקט המצרפי (ראו הגדרה 2 והגדרה 3). בפרט אנו מראים כי הבעיה NP-קשה גם כאשר רוחב העץ ω קבוע. מאחר שהבעיה קשה אף עבור מספר קבוע של ימים, אנו משלימים את התמונה באמצעות אלגוריתם יעיל ביחס לפרמטר המשולב $q + \omega$ (Theorem 1.12).

תמצית

תזמון מקטעים (Interval Scheduling) הוא תת-תחום מחקרי מרכזי של בעיות תזמון המונע באופן טיפוסי ממתודולוגיות תעשייתיות כמו ניהול רזה (Lean Manufacturing) ופרדיגמת תזמון המכונה "תזמון בדיוק-בזמן" (Just-in-Time Scheduling). פרדיגמה זו דוגלת בייצור מוצרים בדיוק בזמן שבו הם נדרשים במטרה למזער עלויות החזקת מלאי. בבעיות מסוג זה, משימות מוגדרות באמצעות מקטעי זמן, ומציאת לוחות זמנים ישימים מתפרשת כבחירת תתי-קבוצות של מקטעים ללא חפיפה הכפופות לאילוצים נוספים. למרות המבנה הפשוט שלהן, הכללות טבעיות רבות של תזמון מקטעים הן קשות חישובית, בפרט הבעיות הנחקרות בעבודה זו אשר הן NP-קשות בצורתן הכללית. מחקר זה מתמקד בפיתוח אלגוריתמים מדויקים ודטרמיניסטיים. התמקדות זו מובילה באופן טבעי לשימוש בתורת הסיבוכיות הפרמטרית, המאפשרת ניתוח מעמיק של יעילות האלגוריתמים מעבר למדדי הסיבוכיות הקלאסיים. באמצעות תורה זו ניתן לזהות פרמטרים נומרים בקלט הבעיה אשר מאפשרים פיתוח אלגוריתמים יעילים כשערכם קטן משמעותית מאורך הקלט, המינוח הטכני לאלגוריתמים מסוג זה הוא אלגוריתמים יעילים ביחס לפרמטר קבוע (fixed-parameter tractable) ומחלקת הבעיות הפרמטריות הניתנות להכרעה על ידי אלגוריתמים אלו נקראת FPT. יתר על כן, תורת הסיבוכיות הפרמטרית מספקת כלים ומתודולוגיות להוכחת קושי פרמטרי. הוכחות אלו יכולות להראות כי עבור בעיה מסוימת ופרמטר מסוים, קיומו של אלגוריתם FPT עומד בסתירה להשערות יסוד מקובלות בתורת הסיבוכיות, ועל כן שוללות את קיומו בהנחה שהשערות אלו מתקיימות. הפרמטרים שאנו חוקרים כוללים, בין היתר, את מספר המכונות, זמני העיבוד של המשימות, ותכונות מתורת הגרפים הנובעות מיחס החיתוך של המקטעים. במסגרת עבודה זו, אנו חוקרים את סיבוכיותן של שלוש בעיות תזמון מקטעים:

תזמון עם רשימת מכונות כשירות (Interval Scheduling with Eligible Machines). תזמון על מכונות זהות הפועלות במקביל ניתן להכללה באמצעות שיוך של כל משימה לרשימת מכונות כשירות, כך שכל משימה יכולה להתבצע על אחת המכונות המופיעות ברשימה שלה. מודל זה ניתן גם לפירוש כמקרה פרטי של תזמון מקטעים על מכונות בלתי-תלויות, בעיית תזמון קלאסית בה זמני העיבוד של כל משימה הם תלויי-מכונה, ואינם נגזרים מיחס מהירות קבוע בין המכונות השונות. הסיבוכיות הפרמטרית של בעיה קלאסית זו ביחס למספר המכונות נכללה ברשימה המשפיעה של 15 הבעיות הפתוחות בתורת התזמון של החוקרים ואן בוורן ומניך. אנו משיבים על שאלתם בשלילה על-ידי הוכחה כי הבעיה קשה עבור המחלקה W[1] ביחס לפרמטר מספר המכונות. בנוסף, אנו מוכיחים קושי למחלקת הסיבוכיות NP עבור המקרה שבו כל זמני העיבוד חסומים בקבוע. אנו משלימים תוצאות אלו באמצעות אלגוריתם יעיל ביחס לפרמטר המשולב של מספר המכונות וזמן העיבוד המקסימלי.

תזמון זרימה דו-שלבי (Flexible Flow Shop Interval Scheduling). בבעיות מסוג תזמון זרימה (Flow Shop) כל משימה מורכבת מסדרת תהליכונים שיש לבצע בסדר קבוע, כאשר

לכל שלב מוקדשת מכונה ייעודית. אנו מתמקדים במערכת דו-שלבית גמישה. כלומר, השלב השני מורכב ממספר מכונות זהות הפועלות במקביל. לכל משימה משויך משקל המבטא את חשיבותה, והמטרה היא למקסם את סכום המשקלים של המשימות המסתיימות בדיוק במועדן. עבודות קודמות הראו כי בעיה זו NP-קשה כבר כאשר מכונה בודדת משמשת לעיבוד כל שלב, אפילו כאשר לכל המשימות אותו זמן עיבוד בשלב השני. אנו חוקרים את הסיבוכיות החישובית של הבעיה הגמישה ומציגים הן תוצאות קושי והן תוצאות אלגוריתמיות. בפרט, אנו מראים כי הבעיה נשארת NP-קשה אפילו כאשר לכל המשימות משקל זהה וכל ערכי הקלט מקודדים אונרית. בנוסף אנו מראים כי הבעיה W[2]-קשה ביחס למספר המכונות בשלב השני ובכך שוללים קיום אלגוריתם יעיל ביחס לפרמטר זה תחת ההשערה כי W[2]≠FPT.

בהמשך אנו מפרטים שלוש תוכניות דינמיות הניתנות לחישוב בזמן פסודו-פולינומי, כל אחת פותרת מקרה פרטי אחר של הבעיה. בנוסף אנו מראים כיצד ניתן, באמצעות שינוי קנה מידה ועיגול המשקלים, להמיר כל אחת מהתוכניות בסכימת קירוב בזמן פולינומי מלא (Fully Polynomial-Time Approximation Scheme). התוכנית הראשונה יעילה למקרה בו מספר המכונות קבוע ומשפרת את האלגוריתם המהיר ביותר למקרה שבו בשלב השני רק מכונה אחת. שתי התוכניות האחרות פותרות את הבעיה בזמן פסודו-פולינומי כאשר זמן העיבוד המקסימלי של התהליכונים בשלב השני חסום בקבוע, או כאשר מספר הצביעה של גרף הקונפליקט בשלב השני הוא $O(\log n)$, כאשר n הוא מספר המשימות.

בנוסף אנו בוחנים את המקרה בו זמני העיבוד של התהליכונים בשלב הראשון זהים. אנו מפתחים אלגוריתמים יעילים למקרה זה תחת שתי הגבלות נוספות. בראשונה בנוסף לזמני העיבוד הזהים בשלב הראשון גם משקלי המשימות זהים ובשנייה אף מקטע של תהליכון בשלב השני לא מכיל במלואו מקטע של משימה אחרת.

תזמון מחזורי הוגן (Fair Repetitive Interval Scheduling). אנו חוקרים מודל תזמון מחזורי שבו אותה קבוצת לקוחות מבקשת שירות במקטעי זמן לאורך מספר ימים. כל יום מהווה מקרה של תזמון מקטעים על מכונה בודדת, ושביעות רצון הלקוחות נמדדת על-פי מספר המשימות שבוצעו לאורך כל התקופה. המטרה היא להבטיח שירות הוגן בטווח הארוך, והיא מוגדרת כמקסום התועלת עבור הלקוח הכי פחות מרוצה. אנו מוכיחים שהבעיה היא NP-קשה גם תחת מגבלות מחמירות, לרבות מספר קבוע של ימים וזמני עיבוד אחידים. כדי לזהות מקרים הניתנים לפתרון יעיל, אנו מנתחים את הסיבוכיות הפרמטרית של הבעיה ביחס לפרמטרים כגון מספר הימים, שביעות הרצון הנדרשת, וכן רוחב-העץ (treewidth) ומספר הצביעה של גרף הקונפליקט בין המשימות.

חיבור זה מבוסס על המאמרים [Her+24, Hee+25, Hee+26] והוא פרי של שיתוף פעולה עם קלאוס הגר, הנדריק מולטר, ברוך שיבר, דני הרמלין ודביר שבתאי.

אוניברסיטת בן-גוריון בנגב
הפקולטה להנדסה
המחלקה להנדסת תעשייה וניהול

תזמון עם מקטעים

חיבור זה מהווה חלק מהדרישות לקבלת התואר דוקטור לפילוסופיה (PhD)

יובל יצחקי

בהנחיית **פרופסור דני הרמלין** ו**פרופסור דביר שבתאי**

מרץ 2026